Time Series Analysis with R

Teodor-Florin Fortis

Table of contents

1. Time Series with R: A Naive Introduction **1**

Introduction **2**

I. Foundations **3**

2. Basics **4**
2.1. Introduction . . . 4
2.2. Recognize What Qualifies as a Time Series . . . 4
2.2.1. Example1: simple time series objects . . . 5
2.3. The Different Approaches of Time Series . . . 6
2.3.1. Example: different forms, same values . . . 7
2.4. Identify Common Uses of Time Series . . . 10
2.4.1. Description, monitoring, and a simple forecast . . . 11
2.5. Formulate the Objectives of Time Series Analysis . . . 13
2.5.1. Example: Basic operations matching these objectives . . . 13
2.6. Why Time Matters . . . 16
2.6.1. Example: Compare an ordered series with a shuffled version . . . 16
2.7. Original . . . 16
2.8. Shuffled . . . 18
2.9. Exploring more sample datasets . . . 20
2.10. Finance . . . 20
2.11. Health . . . 21
2.12. Temperature . . . 22
2.13. Multiple measurements (stock market) . . . 23
2.14. Summary . . . 24

3. Time Series Representations **26**
3.1. Introduction . . . 26
3.2. Time Series From an R Package . . . 27
3.2.1. `gtemp_both_ts` . . . 27
3.2.2. `gtemp_land_ts` . . . 27
3.2.3. `gtemp_ocean_ts` . . . 28
3.2.4. Functions . . . 29
3.3. From Plain Data … . . . 30
3.3.1. Vector . . . 30

3.3.2. Data frame . . . 30
3.3.3. Tibble . . . 31
3.3.4. Plot the data . . . 31
3.4. Time Series in Various Formats . . . 35
3.4.1. `ts` . . . 35
3.4.2. `zoo` . . . 36
3.4.3. `xts` . . . 36
3.4.4. `tsibble` . . . 37
3.5. `tsbox` . . . 38
3.5.1. Plot the `ts` object with `ggplot2::autoplot()` . . . 38
3.6. Import Datasets from the File System . . . 40
3.6.1. From CSV . . . 40
3.6.2. From RDS . . . 40
3.6.3. Plain text & `scan()` . . . 41
3.6.4. Excel (template) . . . 41
3.7. Convert Data into Time-Series Objects . . . 42
3.7.1. `ts` . . . 42
3.7.2. `zoo` . . . 42
3.7.3. `xts` . . . 42
3.7.4. `tsibble` . . . 43
3.8. Dealing with Multiple Measurements at Each Time Point . . . 44
3.8.1. Wide data frame . . . 44
3.8.2. Multivariate `ts` . . . 44
3.8.3. Multivariate `xts` . . . 45
3.8.4. Long/keyed `tsibble` . . . 45
3.8.5. Plotting… . . . 46
3.9. Back to Conversions . . . 47
3.9.1. `tsbox` to a tidy table . . . 47
3.9.2. `tsbox` to `xts` . . . 48
3.9.3. `tsbox` to `tsibble` . . . 49
3.9.4. `tsibble` back to another format . . . 49
3.10. The Right Representation . . . 50
3.10.1. `ts, when...` . . . 50
3.10.2. `zoo, xts``, when...` . . . 50
3.10.3. `tsibble``, when...` . . . 50
3.10.4. data frames `tsbox``, when...` . . . 50
3.11. Summary . . . 51

4. Plotting Time Series **52**
4.1. Introduction . . . 52
4.2. Your First Plot of a Time Series . . . 54
4.2.1. Plotting (`ts`) . . . 54
4.2.2. Plotting (`df`), `ggplot2` . . . 55
4.2.3. Plotting (`ts`), `ggplot2::autoplot()` . . . 56

4.3. plot.ts() and Variants . . . 57
4.3.1. univariate . . . 57
4.3.2. multivariate, one plot . . . 58
4.3.3. multivariate, several plots . . . 59
4.3.4. multivariate, columns (nc) . . . 60
4.3.5. time window . . . 61
4.3.6. aggregation() . . . 62
4.3.7. X–Y / lag-style . . . 63
4.4. ts.plot() for Common-Axis . . . 64
4.4.1. Multi-plots . . . 64
4.4.2. With gpars . . . 65
4.5. zoo and xts indexed time series . . . 67
4.5.1. zoo . . . 67
4.5.2. plot.xts() . . . 67
4.5.3. xts (multiple TS) . . . 68
4.5.4. multi.panel = TRUE . . . 69
4.5.5. multi.panel=value . . . 70
4.5.6. time subset . . . 72
4.5.7. Alternative tick marks . . . 73
4.5.8. Log scale . . . 74
4.6. Plot Tidy Time Series with tsibble . . . 75
4.6.1. tsibble (ggplot2) . . . 76
4.6.2. Faceted tsibble (1/2) . . . 77
4.6.3. Multiple series together with color mapping . . . 80
4.7. Find a Trend . . . 81
4.7.1. Simple moving average . . . 81
4.7.2. Smooth (ggplot2) . . . 82
4.7.3. Original vs smoothed . . . 83
4.8. Visualize Distribution and Variation Over Time . . . 84
4.8.1. Bars . . . 84
4.8.2. Histogram . . . 85
4.8.3. Boxplots . . . 86
4.9. Compare Multiple Plots . . . 87
4.9.1. Basics . . . 87
4.9.2. facet_wrap() . . . 88
4.9.3. Option 8C: Vertical comparison by reshaping . . . 90
4.10. Simple Plotting Guidelines . . . 91
4.10.1. Observations . . . 92
4.11. Summary . . . 92

II. Exercises **94**

5. Exercises **95**
5.1. Discover datasets . . . 95

5.2. Base R 95
5.3. `timeSeriesDataSets` 95
5.4. tsibbledata 96
5.5. Recognize time series objects 96
5.6. Play with a time series dataset (basic) 97
5.7. Play with a dataset (`timeSeriesDataSets`) 98
5.8. Play with a dataset (`tsibbledata`) 99
5.9. Alternative representations, same same data 99
5.10. Why time matters 100
5.11. A simple moving average 101
5.12. A forecasting experiment 102
5.13. Search and play with custom time series dataset 102
5.14. Student choice mini-investigation (recommeded) 103
5.15. Compare several time series from different sources 104

1. Time Series with R: A Naive Introduction

Introduction

These pages introduce time series analysis in R through a recipe-based approach.

Each chapter includes a short conceptual introduction, usually followed by practical, self-contained recipes focused on common tasks.

A version of this textbook is available at https://nxrolinf.github.io/QuartoTSB1

Part I.

Foundations

2. Basics

2.1. Introduction

Time series analysis studies data observed through time. In the most practical sense, a time series is a sequence of observations indexed by time, often collected at regular intervals of time (seconds, minutes, hours, days, months, or even years).

From a statistical perspective, time series are different from ordinary unordered data because observations are usually dependent across time. This means that nearby observations often influence one another, which is why time series require specialized methods.

It is in our intention to introduce some core notions on time series. The goal is not to build sophisticated models, but to understand what a time series are, how they differ from other data structures, what are their uses, and what kinds of questions time series analysis is designed to answer.

> This chapter is intentionally introductory. The goal is not yet to model or forecast a series, but to build the conceptual vocabulary needed for the rest of the book.

```
# We use mostly built-in functionality in this chapter.
# The forecast package is used in one example for a simple forecast.
library(forecast)
library(ggplot2)
library(ggfortify)
```

2.2. Recognize What Qualifies as a Time Series

A time series is a sequence of values recorded over time for the same variable or process. There are many examples, such as daily temperatures, monthly unemployment, hourly electricity demand, stock prices, annual GDP, daily solar power, and many others.

Usually, observations are assumed to occur at regular intervals of time. This regularity is commonly described by the frequency of the series. For example: - frequency = 1 for yearly data - frequency = 4 for quarterly data - frequency = 12 for monthly data

A useful rule of thumb is this: if the order of the observations matters, and especially if adjacent observations are expected to be related, then the data should be treated as a time series.

2.2.1. Example1: simple time series objects

```
# Yearly data
yearly_sales <- ts(c(120, 135, 142, 160, 175), start = 2020, frequency = 1)
yearly_sales
```

```
Time Series:
Start = 2020
End = 2024
Frequency = 1
[1] 120 135 142 160 175
```

```
# Quarterly data
quarterly_demand <- ts(c(50, 55, 53, 60, 62, 64, 63, 70),
                       start = c(2023, 1), frequency = 4)
quarterly_demand
```

```
     Qtr1 Qtr2 Qtr3 Qtr4
2023   50   55   53   60
2024   62   64   63   70
```

```
# Monthly data using a built-in dataset
AirPassengers
```

```
     Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1949 112 118 132 129 121 135 148 148 136 119 104 118
1950 115 126 141 135 125 149 170 170 158 133 114 140
1951 145 150 178 163 172 178 199 199 184 162 146 166
1952 171 180 193 181 183 218 230 242 209 191 172 194
1953 196 196 236 235 229 243 264 272 237 211 180 201
1954 204 188 235 227 234 264 302 293 259 229 203 229
1955 242 233 267 269 270 315 364 347 312 274 237 278
1956 284 277 317 313 318 374 413 405 355 306 271 306
1957 315 301 356 348 355 422 465 467 404 347 305 336
1958 340 318 362 348 363 435 491 505 404 359 310 337
1959 360 342 406 396 420 472 548 559 463 407 362 405
1960 417 391 419 461 472 535 622 606 508 461 390 432
```

```
frequency(AirPassengers)
```

```
[1] 12
```

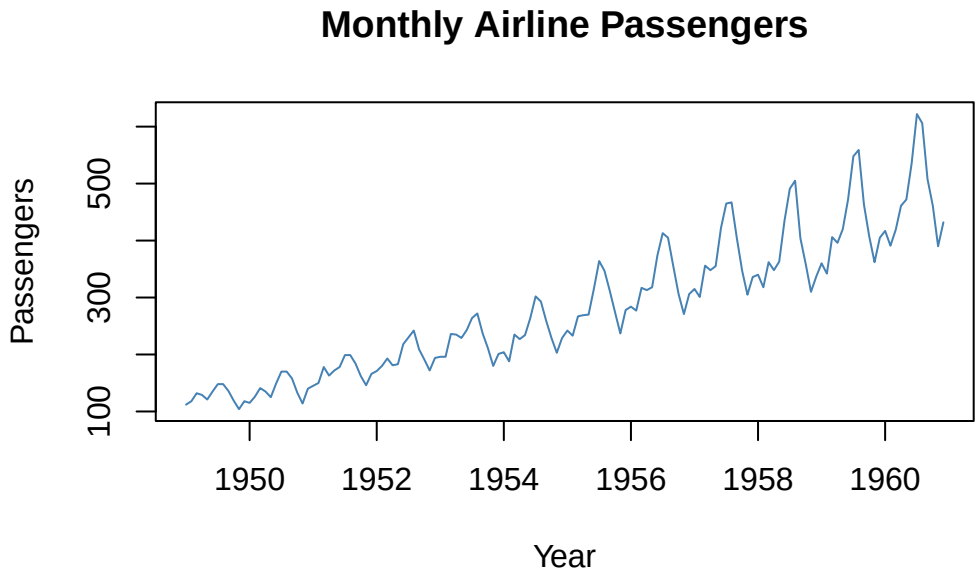
```r
start(AirPassengers)
```

```
[1] 1949    1
```

```r
end(AirPassengers)
```

```
[1] 1960   12
```

```r
plot(AirPassengers,
     main = "Monthly Airline Passengers",
     ylab = "Passengers",
     xlab = "Year",
     col = "steelblue")
```



These examples illustrate a defining feature of time series work in R: the data are not just values, but values interpreted together with temporal location and frequency. Even when the values are simple numeric observations, attaching a time structure allows them to be analyzed and visualized as a sequence evolving through time.

2.3. The Different Approaches of Time Series

There are several complementary ways to deal with a time series.

- A time series – as a set of measurements collected over time for the same process or unit of observation (**practical** approach).
- A time series as a realization of a stochastic process. This emphasizes that observations are typically not independent and that their ordering in time is central to the analysis (**statistical** approach).
- For the case of R, time series are usually stored in special objects that preserve temporal structure and enable specific methods for plotting, transforming, and analyzing the data (**computational** approach, focused on representation).

2.3.1. Example: different forms, same values

```
library(ggfortify)
values <- c(120, 135, 142, 160, 175)

# 1. Plain numeric vector
values
```

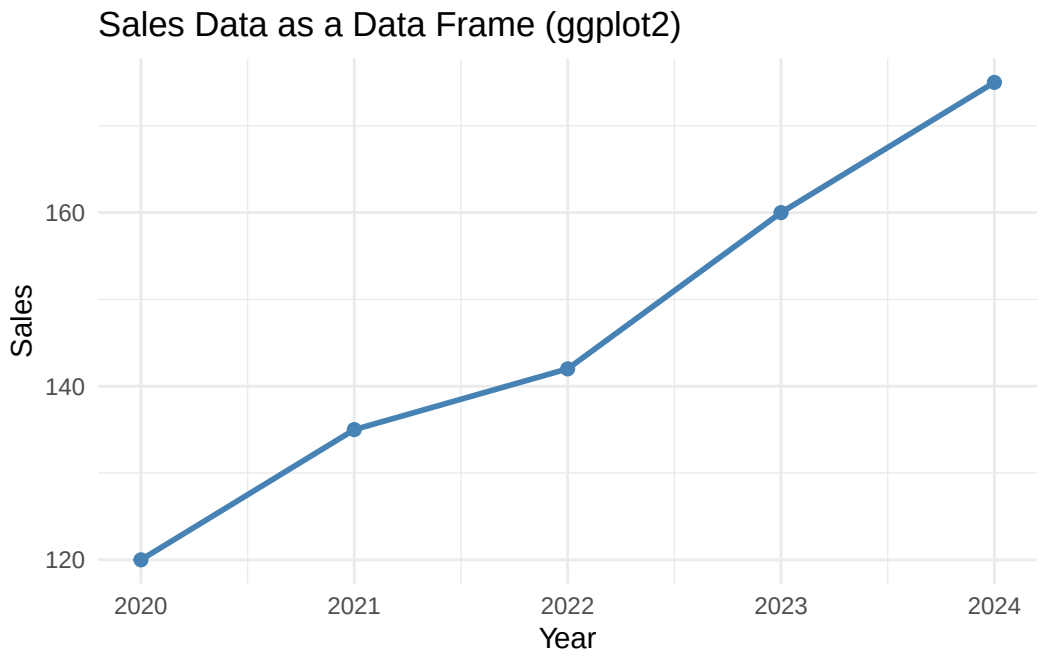
```
[1] 120 135 142 160 175
```

```
# 2. Data frame with explicit year column
sales_df <- data.frame(
  year = 2020:2024,
  sales = values
)
sales_df
```

```
  year sales
1 2020   120
2 2021   135
3 2022   142
4 2023   160
5 2024   175
```

```
# Autoplot with specified columns
ggplot(sales_df, aes(x = year, y = sales)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(color = "steelblue", size = 2) +
  labs(
    title = "Sales Data as a Data Frame (ggplot2)",
    x = "Year",
```

```
    y = "Sales"
  ) +
  theme_minimal()
```




```
# 3. Time series object
sales_ts <- ts(values, start = 2020, frequency = 1)
sales_ts
```

```
Time Series:
Start = 2020
End = 2024
Frequency = 1
[1] 120 135 142 160 175
```

```
# Compare the structures
class(values)
```

```
[1] "numeric"
```

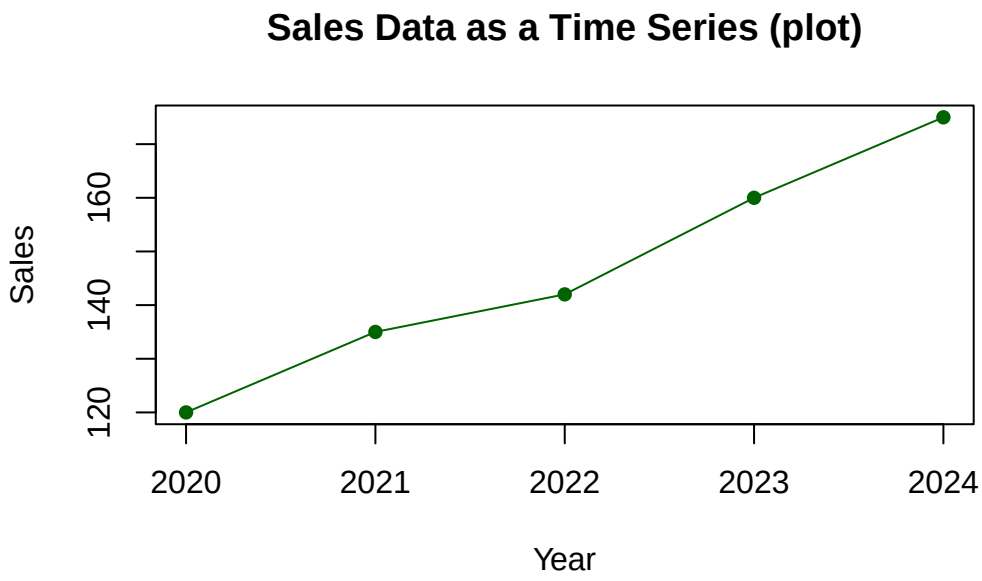
```
class(sales_df)
```

```
[1] "data.frame"
```

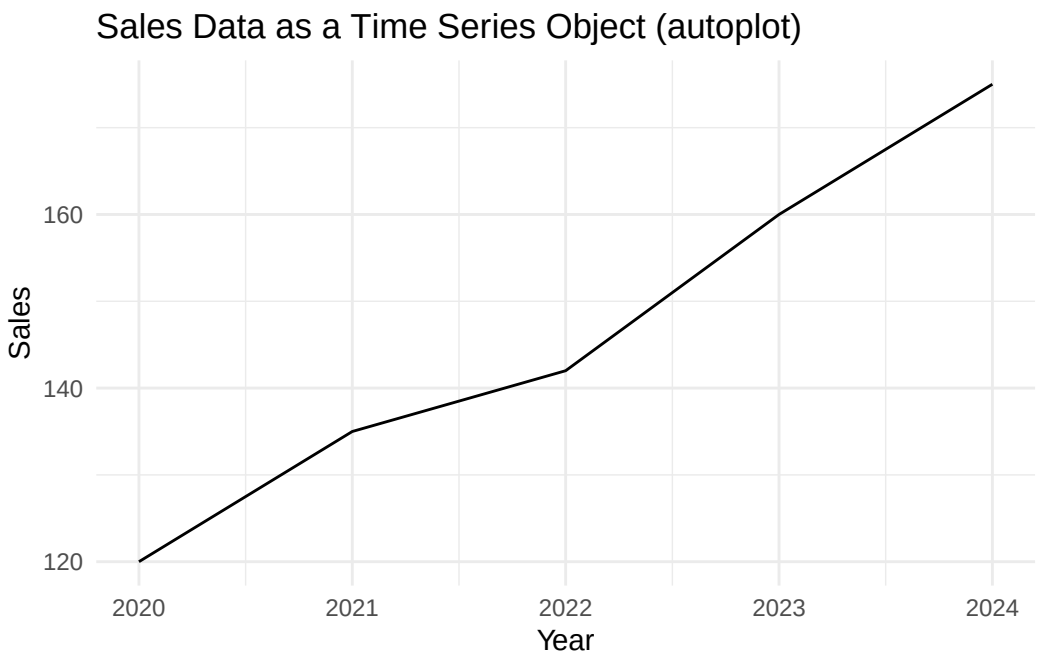
```
class(sales_ts)
```

```
[1] "ts"
```

```
# Plot the time series version
plot(sales_ts,
     type = "o",
     pch = 16,
     col = "darkgreen",
     main = "Sales Data as a Time Series (plot)",
     ylab = "Sales",
     xlab = "Year")
```




```
# Alternative plot for the time series version
autoplot(sales_ts) +
  labs(
    title = "Sales Data as a Time Series Object (autoplot)",
    x = "Year",
    y = "Sales"
  ) +
  theme_minimal()
```



- The numeric vector contains only values.
- The data frame adds an explicit year variable.
- The `ts` object adds temporal semantics that R can use for time-aware analysis and plotting.

While the representations use similar information, they may support different workflows and methods. This distinction becomes important when we're dealing with various time series representations in R.

2.4. Identify Common Uses of Time Series

Time series are used in many domains because they help us understand how a variable evolves through time.

One major use is **forecasting**, where past data are used to predict future values.

A second use is **description and monitoring**, where the goal is to inspect the evolution of the series and identify broad patterns such as growth, seasonality, or sudden changes.

A third use is **explanation**, where the objective is to understand temporal relationships, delays, and persistence.

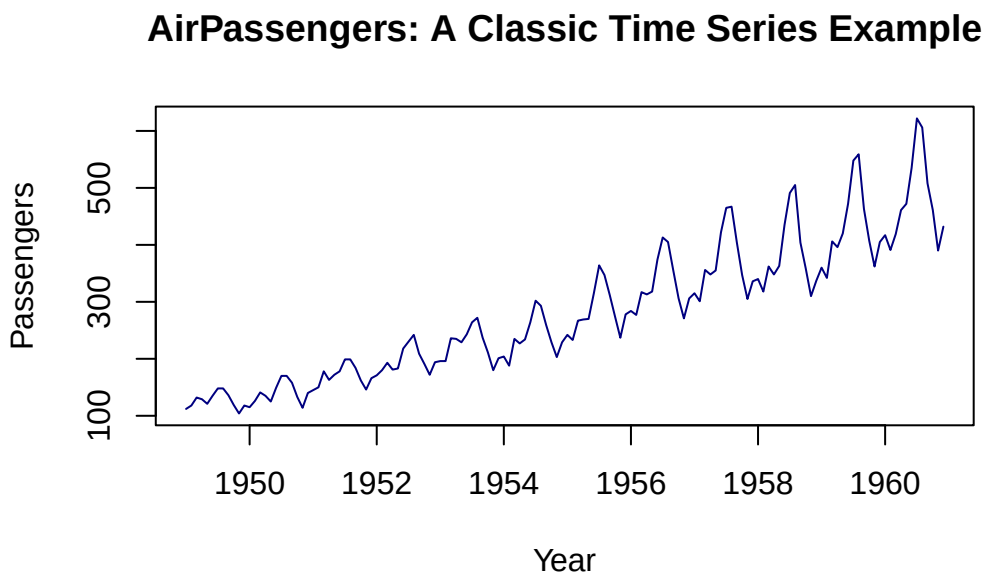
A fourth use is **decision support**, where time series help guide planning, operations, and control.

2.4.1. Description, monitoring, and a simple forecast

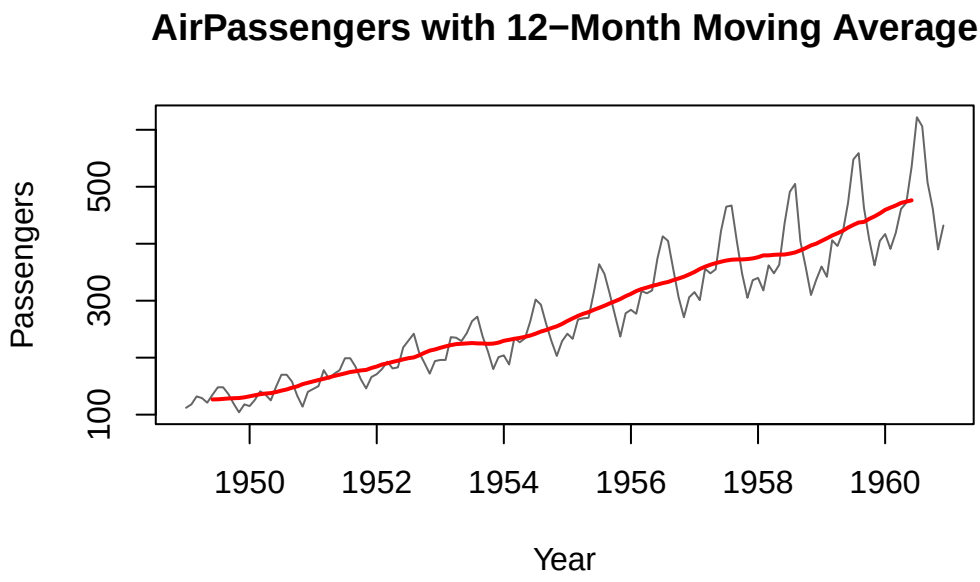
Time series datasets

An example based on the `AirPassengers` dataset, already available in R (the `dataset` package).
Check the package `timeSeriesDataSets` for more datasets: you'll have to install the package(s) first.

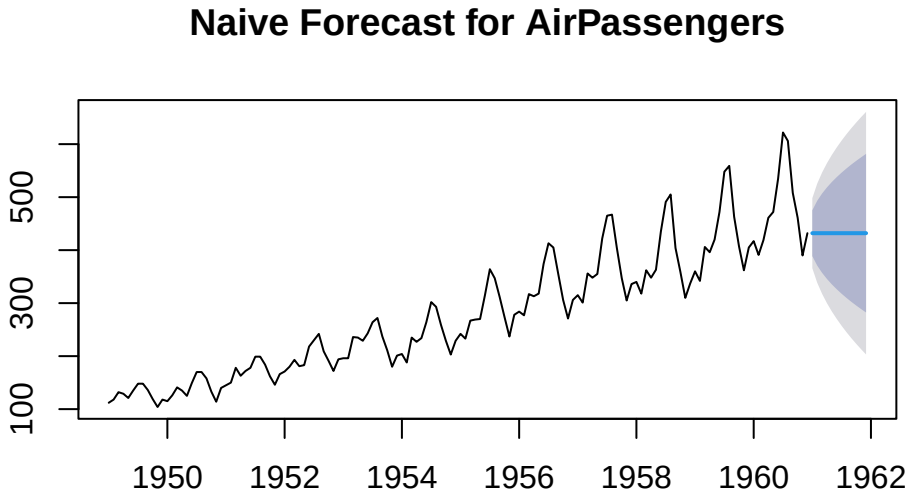
```r
# Plot the series for visual inspection
plot(AirPassengers,
     main = "AirPassengers: A Classic Time Series Example",
     ylab = "Passengers",
     xlab = "Year",
     col = "navy")
```




```r
# Add a simple moving average to highlight long-term movement
air_ma <- stats::filter(AirPassengers, rep(1/12, 12), sides = 2)

plot(AirPassengers,
     main = "AirPassengers with 12-Month Moving Average",
     ylab = "Passengers",
     xlab = "Year",
     col = "gray40")
lines(air_ma, col = "red", lwd = 2)
```




```
# A very simple forecast as an illustration of the forecasting objective
air_naive_fc <- naive(AirPassengers, h = 12)
plot(air_naive_fc,
     main = "Naive Forecast for AirPassengers")
```



Exercise

You can check other simple forecasting functions, like

```
# library (forecast) is required!

air_naive_fc1 <- forecast(AirPassengers, h = 12)
air_naive_fc2 <- rwf (AirPassengers, h = 12)
```

or do some other investigations

```
# Drift forecast
rwf(AirPassengers, drift = TRUE) |> autoplot()

# Seasonal naive forecasts
snaive(AirPassengers) |> autoplot()
```

Discussion

The first plot supports visual description. The *moving average* helps reveal long-term movement by smoothing short-term variation. The naive forecast illustrates the forecasting objective in its simplest form: future values are projected from past behavior.

2.5. Formulate the Objectives of Time Series Analysis

The objectives of time series analysis can be grouped into several broad categories:

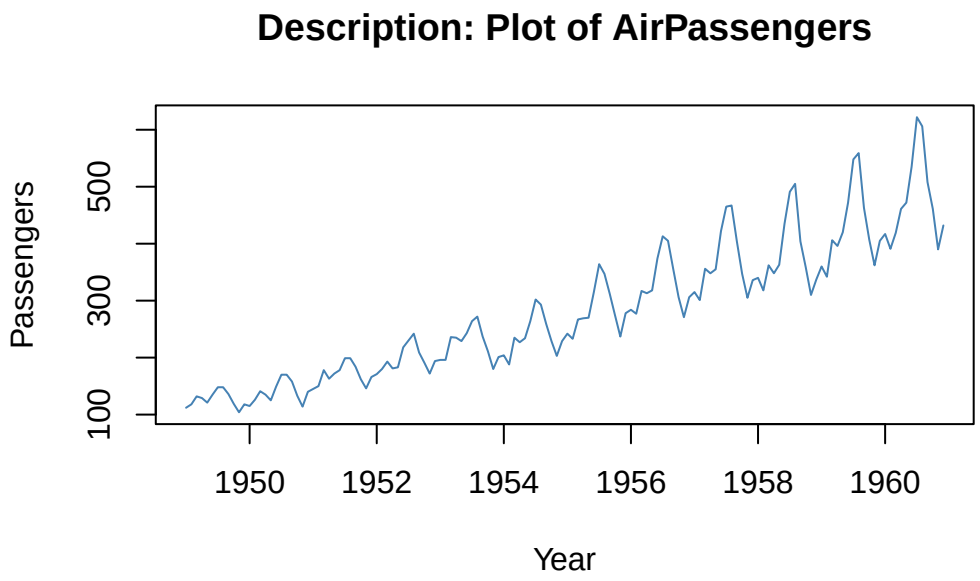
1. **Description:** Summarize and visualize the structure of a series.
2. **Explanation:** Understand the mechanisms or relationships underlying the observed behavior.
3. **Forecasting:** Predict future values using past observations.
4. **Detection:** Identify unusual events, changes, anomalies, or structural breaks.
5. **Support for decision making:** Use time-dependent information to guide planning and intervention.

2.5.1. Example: Basic operations matching these objectives

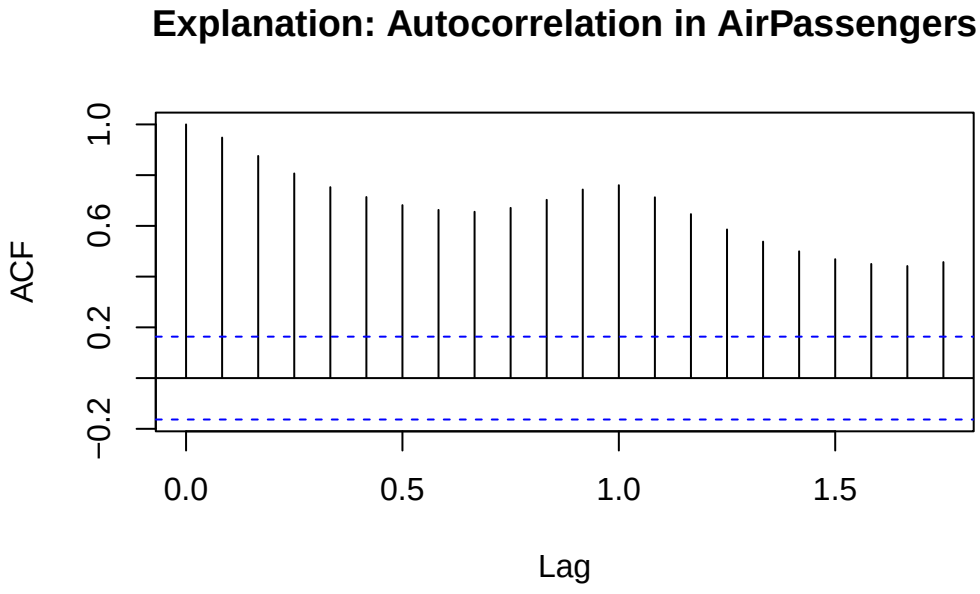
```
# Description: simple summary and plot
summary(AirPassengers)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  104.0   180.0   265.5   280.3   360.5   622.0
```

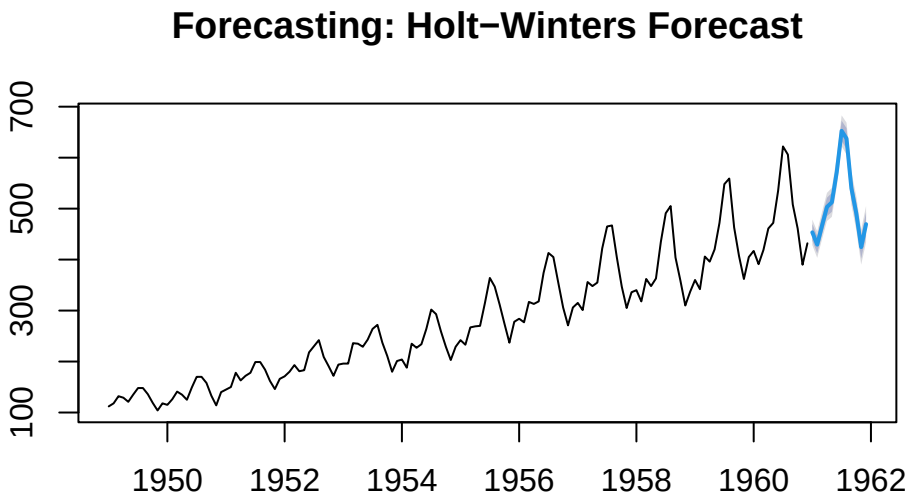
```r
plot(AirPassengers,
     main = "Description: Plot of AirPassengers",
     ylab = "Passengers",
     xlab = "Year",
     col = "steelblue")
```



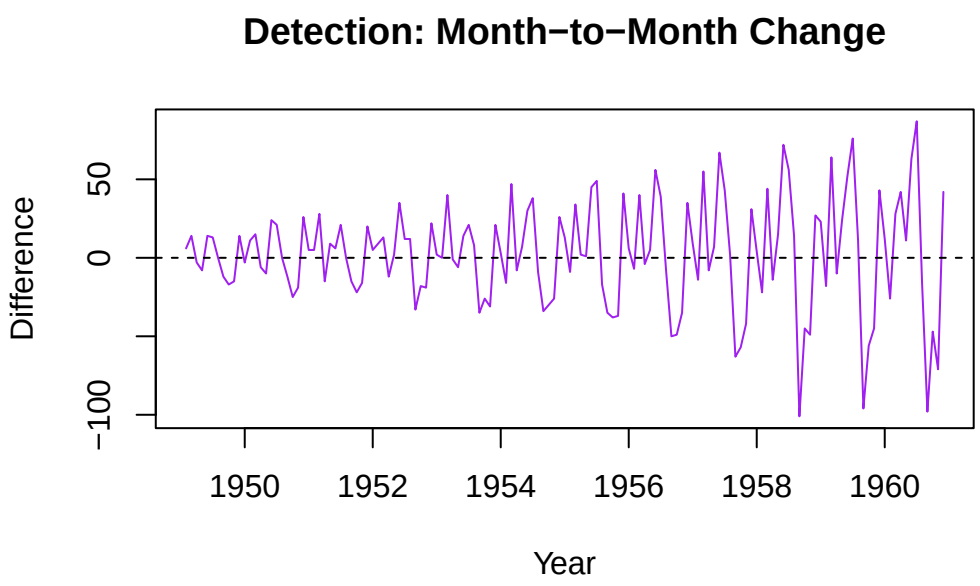

```r
# Explanation: inspect dependence through the autocorrelation function
acf(AirPassengers,
    main = "Explanation: Autocorrelation in AirPassengers")
```



```r
# Forecasting: produce a forecast from an exponential smoothing model
air_hw <- HoltWinters(AirPassengers)
air_hw_fc <- forecast(air_hw, h = 12)

plot(air_hw_fc,
     main = "Forecasting: Holt-Winters Forecast")
```




```r
# Detection: identify unusually large month-to-month changes
air_diff <- diff(AirPassengers)
plot(air_diff,
     main = "Detection: Month-to-Month Change",
     ylab = "Difference",
     xlab = "Year",
     col = "purple")
abline(h = 0, lty = 2)
```



These examples illustrate how the same series can support multiple analytical goals. Visualization and summary serve description, autocorrelation helps reveal internal temporal structure, forecasting projects future values, and differencing can highlight changes or unusually large movements between consecutive periods.

2.6. Why Time Matters

Time changes the structure of the data because it introduces order and, often, dependence.

In many classical statistical methods, observations are assumed to be independent. Time series often violate this assumption because adjacent observations tend to be correlated.

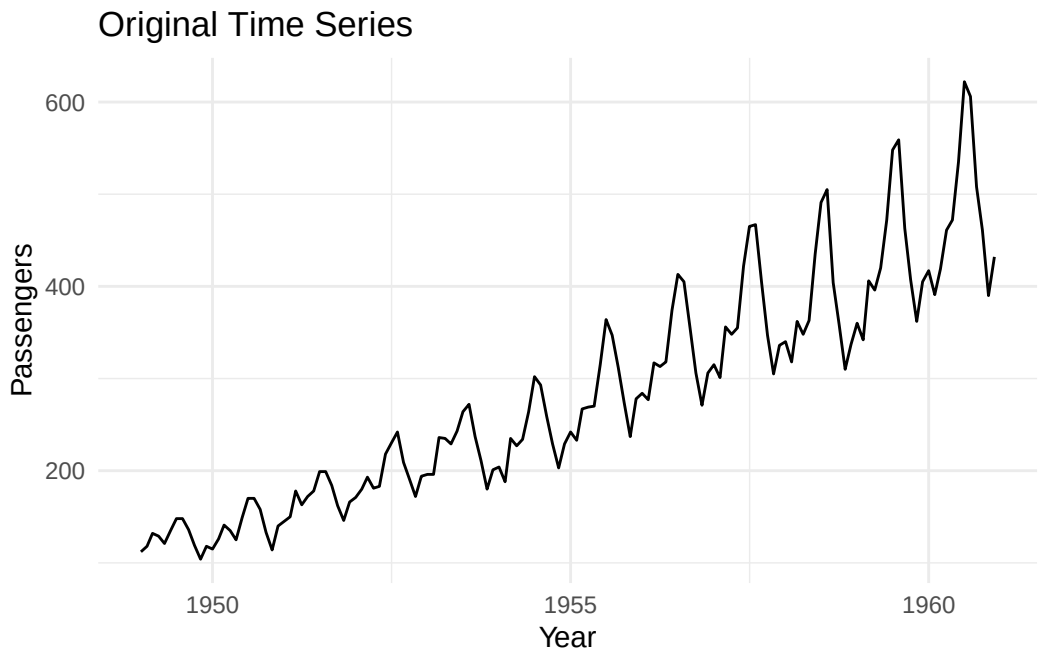
This means that the order of the observations is not arbitrary. Shuffling a time series destroys part of the information we care about, namely its temporal structure.

2.6.1. Example: Compare an ordered series with a shuffled version

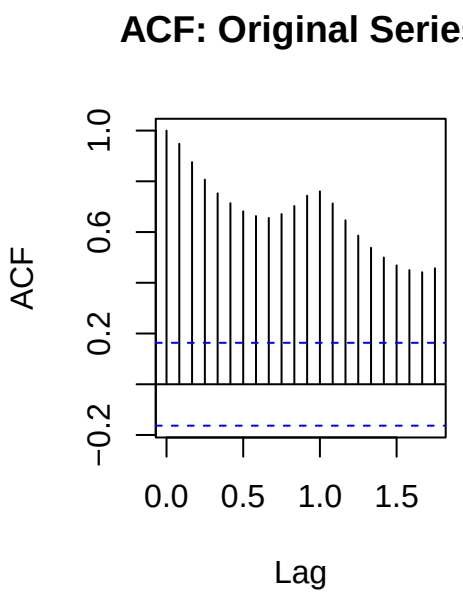
2.7. Original

```
set.seed(123)

# Original series
original_series <- AirPassengers

# Shuffled version: same values, wrong order
```

```
shuffled_series <- ts(sample(as.numeric(AirPassengers)),
                      start = start(AirPassengers),
                      frequency = frequency(AirPassengers))

par(mfrow = c(1, 2))

autoplot(original_series) +
  labs(
    title = "Original Time Series",
    x = "Year",
    y = "Passengers"
  ) +
  theme_minimal()
```

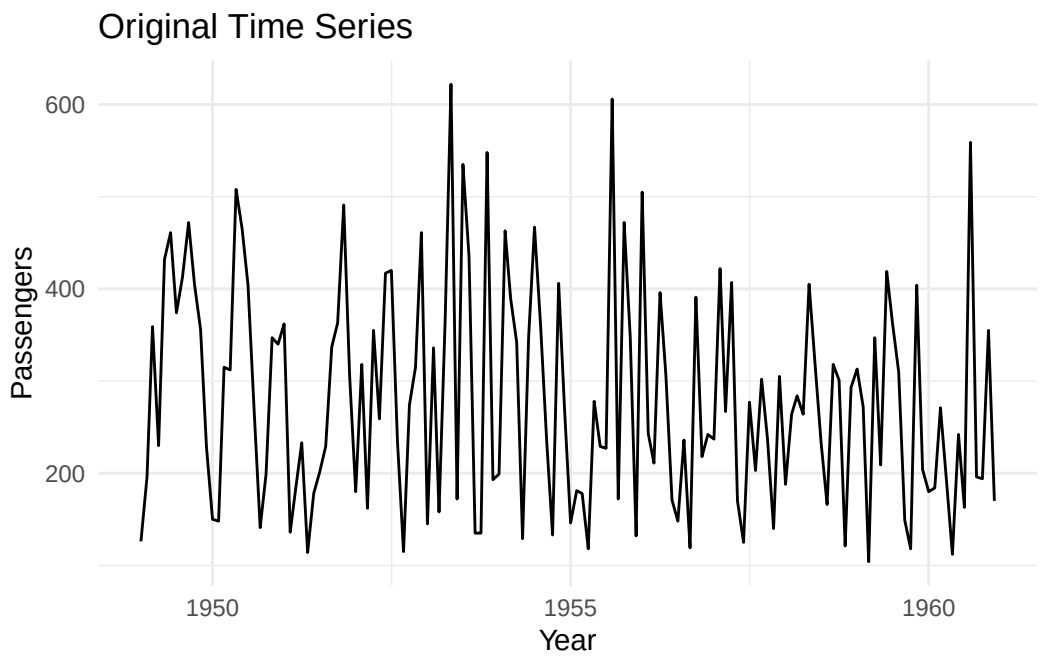



```
par(mfrow = c(1, 1))

# Compare autocorrelation
par(mfrow = c(1, 2)) # two column plots

acf(original_series, main = "ACF: Original Series")

par(mfrow = c(1, 1)) # back to one column
```

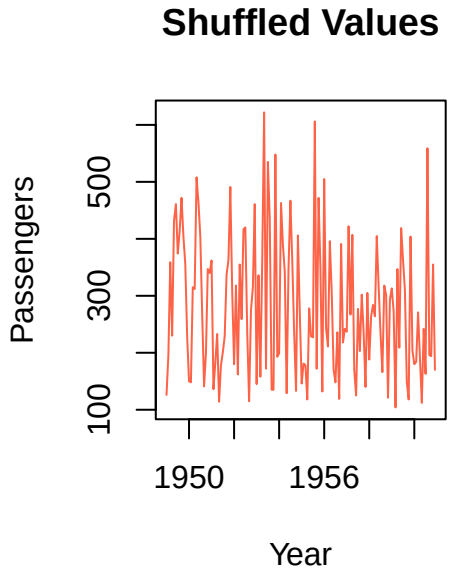


2.8. Shuffled

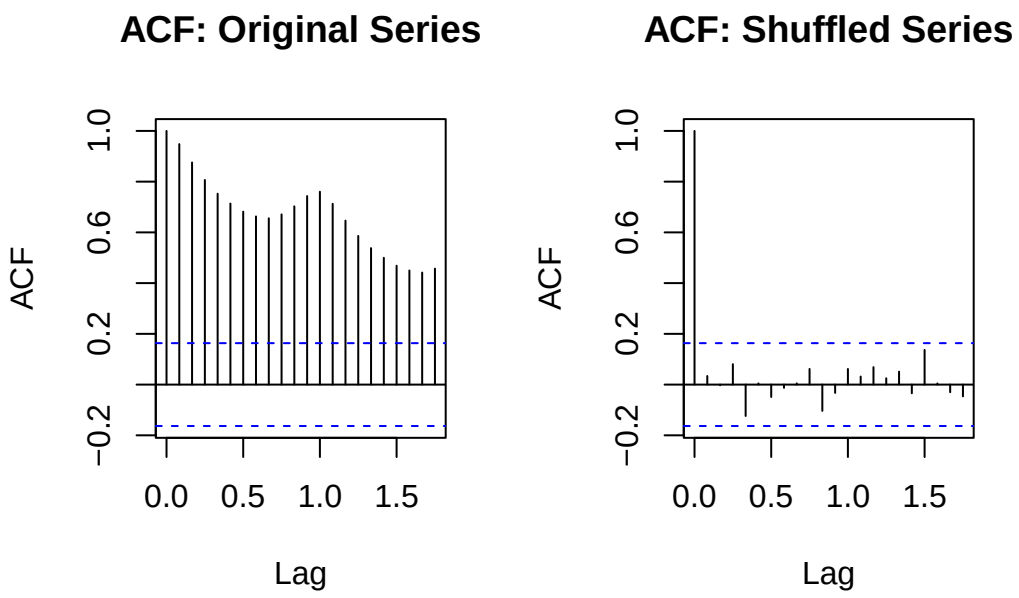
```r
set.seed(123)

# Original series
original_series <- AirPassengers

# Shuffled version: same values, wrong order
shuffled_series <- ts(sample(as.numeric(AirPassengers)),
                      start = start(AirPassengers),
                      frequency = frequency(AirPassengers))

par(mfrow = c(1, 2))

autoplot(shuffled_series) +
  labs(
    title = "Original Time Series",
    x = "Year",
    y = "Passengers",
    color = "tomato"
  ) +
  theme_minimal()
```




```
plot(shuffled_series,
     main = "Shuffled Values",
     ylab = "Passengers",
     xlab = "Year",
     col = "tomato")

par(mfrow = c(1, 1))
```



```r
# Compare autocorrelation
par(mfrow = c(1, 2))

acf(original_series, main = "ACF: Original Series")
acf(shuffled_series, main = "ACF: Shuffled Series")
```




```r
par(mfrow = c(1, 1))
```

The shuffled series contains the same values as the original series, but it no longer preserves their temporal order. As a result, the visible structure is lost and the autocorrelation pattern is altered. This simple experiment shows why time series analysis must take ordering seriously.

2.9. Exploring more sample datasets

2.10. Finance

The `JohnsonJohnson` dataset is a quarterly time series giving earnings per share for Johnson & Johnson. It is useful as a compact example of a financial time series with a clear long-term trend and visible fluctuations across quarters.

```
autoplot(
  JohnsonJohnson, ts.colour = "steelblue", ts.size = 0.8,
  lwd = 2) +
  labs (
    title = "Johnson & Johnson Quarterly Earnings",
    y = "Earnings per share",
    x = "Year") + theme_minimal()
```

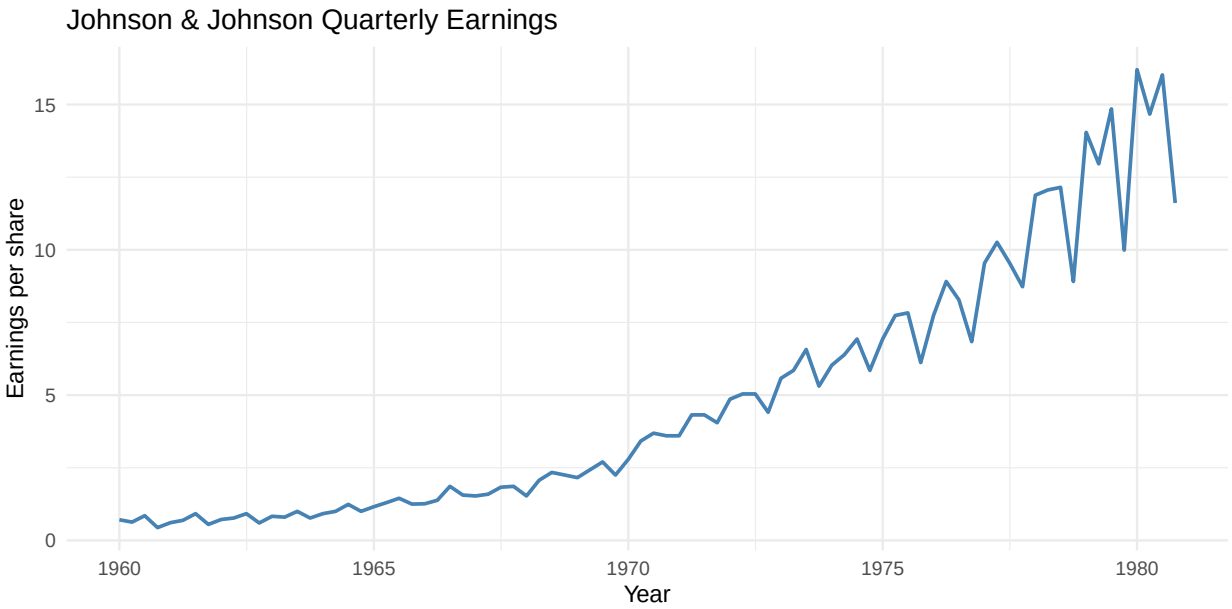


Figure 2.1.: Financial time series example: the built-in `JohnsonJohnson` dataset shows quarterly earnings per Johnson & Johnson share. This figure illustrates a univariate time series from the financial domain, where the horizontal axis represents time in quarters and the vertical axis represents earnings per share.

2.11. Health

The `ldeaths` dataset records monthly deaths from lung diseases in the United Kingdom. It provides a health-related example in which repeated measurement over time can reveal long-term changes and seasonal patterns.

```
autoplot(ldeaths, ts.colour = "firebrick", ts.size = 0.8) +
  labs (
    title = "Monthly Deaths from Lung Diseases in the UK",
    y = "Number of deaths",
    x = "Year") + theme_minimal()
```

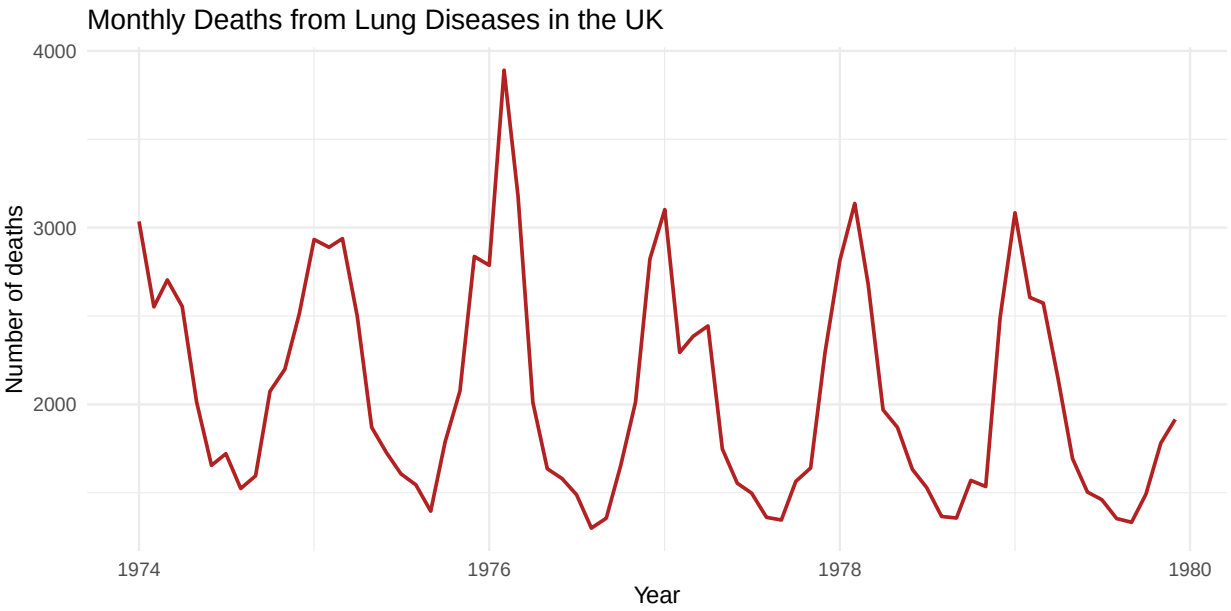


Figure 2.2.: Health time series example: the built-in `ldeaths` dataset records monthly deaths from lung diseases in the UK. This figure illustrates a univariate health-related time series, where each observation corresponds to a monthly count of deaths.

2.12. Temperature

The `nottem` dataset contains average monthly air temperatures recorded in Nottingham from 1920 to 1939. It is a classical example of a climatic time series and is especially useful for showing seasonality.

```
autoplot(nottem, ts.colour = "darkorange", ts.size = 0.8) +
  labs(
    title = "Average Monthly Temperatures at Nottingham",
    x = "Year",
    y = "Temperature"
  ) + theme_minimal()
```

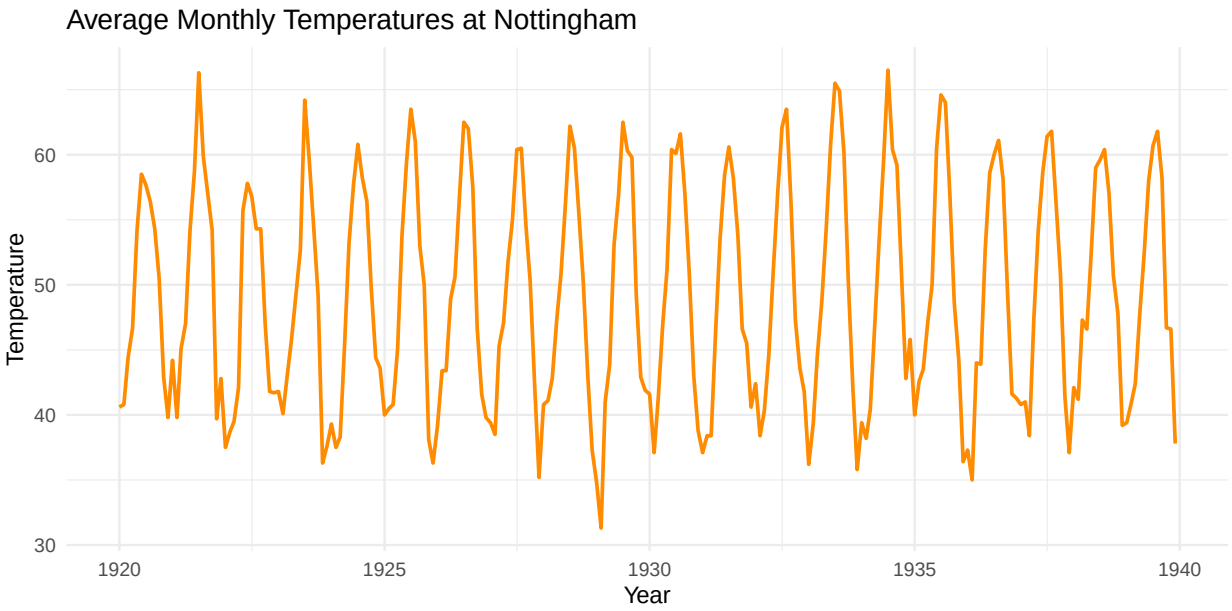


Figure 2.3.: Temperature time series example: the built-in `nottem` dataset contains average monthly temperatures measured in Nottingham from 1920 to 1939. The recurring annual pattern makes it a useful illustration of seasonality in an environmental time series.

2.13. Multiple measurements (stock market)

The `EuStockMarkets` dataset is a multivariate time series containing daily closing prices of several major European stock indices. Unlike the previous examples, which are univariate, this dataset stores multiple measurements observed at the same time points.

```
autoplot (EuStockMarkets) + labs (x="Year", y="Index Value", title = "European Stock Market In

#matplot(
#  time(EuStockMarkets),
#  EuStockMarkets,
#  type = "l",
#  lty = 1,
#  lwd = 1.5,
#  col = c("steelblue", "firebrick", "darkgreen", "goldenrod"),
#  xlab = "Year",
#  ylab = "Index value",
#  main = "European Stock Market Indices"
#)

#legend(
```

```
#  "topleft",
#  legend = colnames(EuStockMarkets),
#  col = c("steelblue", "firebrick", "darkgreen", "goldenrod"),
#  lty = 1,
#  lwd = 1.5,
#  bty = "n"
#)
```

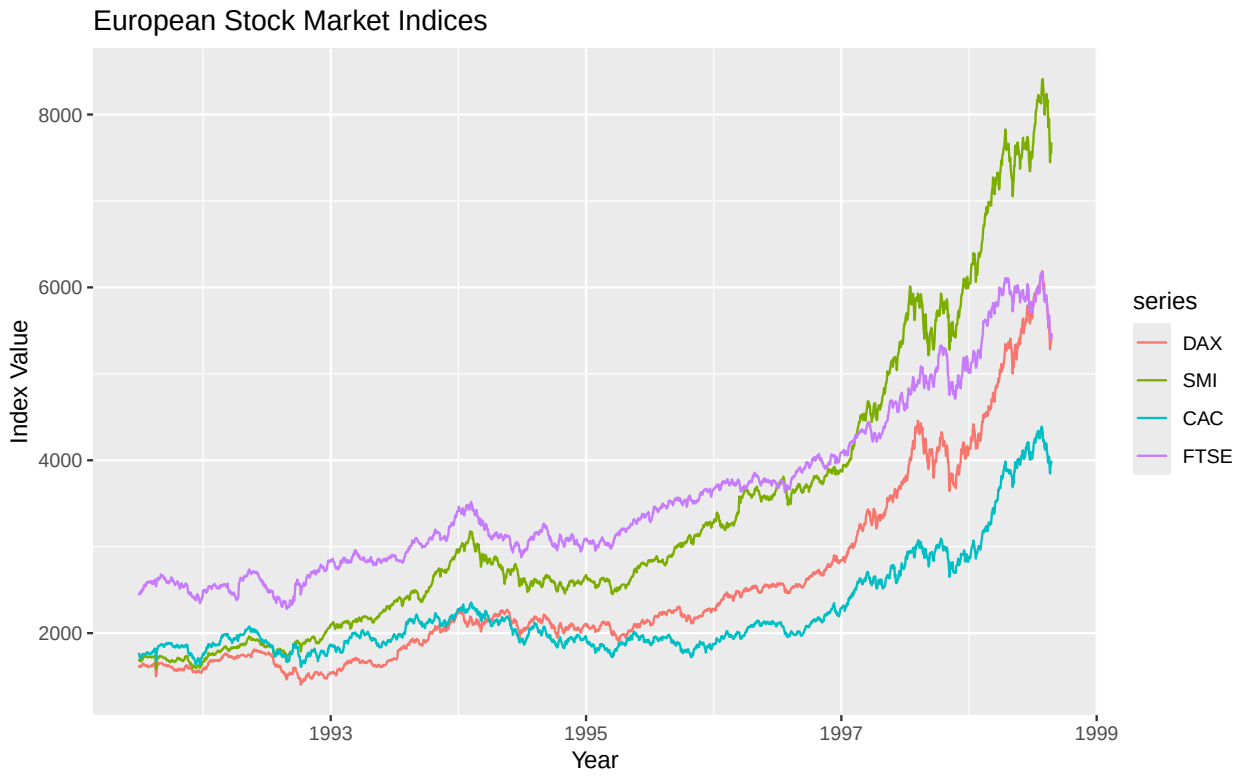


Figure 2.4.: Multivariate time series example: the built-in `EuStockMarkets` dataset contains daily closing prices for several major European stock indices. This figure illustrates a time series with multiple measurements observed at each time point, making it suitable for discussing multivariate or multiple-series time series data.

2.14. Summary

In this chapter, we introduced the basic conceptual foundations of time series analysis. A time series is more than a list of values with timestamps: it is data whose ordering in time matters and whose observations are often related across time.

We distinguished practical, statistical, and computational views of time series, discussed common uses such as description, monitoring, forecasting, and decision support, and outlined the major

objectives of time series analysis.

The next chapter moves from concepts to implementation by examining how time series can be represented in R using several different object types and data paradigms.

3. Time Series Representations

3.1. Introduction

A short presentation of the main main ways in which time series can be represented in R, with a practical approach: the same underlying time series data can be stored as a plain vector, a data frame, a classical `ts` object, a `zoo`/`xts` indexed object, or a `tsibble`. Additionally real work often requires importing data from files or packages and converting between various formats.

For an easier comparison, we're using the **global temperature anomaly datasets**: `gtemp_both_ts`, `gtemp_land_ts`, and `gtemp_ocean_ts` from the `timeSeriesDataSets` package.

> **Note**
>
> This chapter is organized by representational task rather than by package. Each recipe uses a **tabset** so the reader can compare alternative approaches side by side.

```
library(ggplot2)
library(ggfortify)
library(zoo)
library(xts)
library(tsibble)
library(tsbox)
library(dplyr)
library(tibble)
library(tidyr)
library(timeSeriesDataSets)

# Running example datasets
# gtemp_both_ts  : global land + ocean temperature
# gtemp_land_ts  : global land temperature
# gtemp_ocean_ts : global ocean temperature

#
# additional datasets are available in R (`datasets` package)
```

3.2. Time Series From an R Package

One of the simplest import path: load a time series object directly from an installed package (`timeSeriesDataSets`).

3.2.1. gtemp_both_ts

```
data(gtemp_both_ts, package = "timeSeriesDataSets")
class(gtemp_both_ts)
```

```
[1] "ts"
```

```
start(gtemp_both_ts)
```

```
[1] 1850    1
```

```
end(gtemp_both_ts)
```

```
[1] 2023    1
```

```
frequency(gtemp_both_ts)
```

```
[1] 1
```

```
head(gtemp_both_ts)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
[1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

3.2.2. gtemp_land_ts

```
data(gtemp_land_ts, package = "timeSeriesDataSets")
class(gtemp_land_ts)
```

```
[1] "ts"
```

```
start(gtemp_land_ts)
```

```
[1] 1850    1
```

```
end(gtemp_land_ts)
```

```
[1] 2023    1
```

```
frequency(gtemp_land_ts)
```

```
[1] 1
```

```
head(gtemp_land_ts)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
[1] -0.5 -0.6 -0.5 -0.5 -0.2 -0.5
```

3.2.3. gtemp_ocean_ts

```
data(gtemp_ocean_ts, package = "timeSeriesDataSets")
class(gtemp_ocean_ts)
```

```
[1] "ts"
```

```
start(gtemp_ocean_ts)
```

```
[1] 1850    1
```

```
end(gtemp_ocean_ts)
```

```
[1] 2023    1
```

```
frequency(gtemp_ocean_ts)
```

```
[1] 1
```

```
head(gtemp_ocean_ts)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
[1] -0.12 -0.08 -0.14  0.04  0.04  0.00
```

3.2.4. Functions

```
# use data() to load `dataset_name` -- this does not exist!
data(dataset_name, package = "timeSeriesDataSets")

# simple class information, should return `ts`
class(dataset_name)

# Time of first observation (e.g., year) for a time-series
start(dataset_name)

# Time of last observation (e.g., year) for a time-series
end(dataset_name) # Last recorded index (year)

# The number of samples per unit time
frequency(dataset_name)
# Also check `time`, `cycle`, `deltat`

#First recorded data; Use `tail` for last recorded data
head(dataset_name)
```

! Notice

- All three datasets are already `ts` objects;
- The datasets from `timeSeriesDataSets` are immediately ready for time-series-aware plotting and modeling;
- Using package-provided datasets, as they are convenient for teaching.

3.3. From Plain Data ...

Start from existing datasets, reconstruct them from more primitive forms.

3.3.1. Vector

As a numeric vector only. Extract the numeric values (`as.numeric`), and check the content.

```
gtemp_vec <- as.numeric(gtemp_both_ts)
head(gtemp_vec)
```

```
[1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

```
class(gtemp_vec)
```

```
[1] "numeric"
```

3.3.2. Data frame

As data frames, with explicit year column. Extract both dimensions (with `time` and `as.numeric`), reconstruct with tibble.

```
gtemp_df <- tibble(
  year = time(gtemp_both_ts),
  anomaly = as.numeric(gtemp_both_ts)
)
# also, year = as.numeric(time(gtemp_both_ts))

gtemp_df
```

```
# A tibble: 174 x 2
    year anomaly
   <dbl>   <dbl>
 1  1850   -0.24
 2  1851   -0.25
 3  1852   -0.27
 4  1853   -0.15
 5  1854   -0.05
 6  1855   -0.16
 7  1856   -0.29
 8  1857   -0.32
```

```
 9  1858   -0.19
10  1859   -0.04
# i 164 more rows
```

3.3.3. Tibble

Tibble, with explicit year column (using `as_tibble`)

```
gtemp_tbl <- as_tibble(gtemp_df)
gtemp_tbl
```

```
# A tibble: 174 x 2
    year anomaly
   <dbl>   <dbl>
 1  1850   -0.24
 2  1851   -0.25
 3  1852   -0.27
 4  1853   -0.15
 5  1854   -0.05
 6  1855   -0.16
 7  1856   -0.29
 8  1857   -0.32
 9  1858   -0.19
10  1859   -0.04
# i 164 more rows
```

3.3.4. Plot the data

```
ggplot(gtemp_df, aes(x = year, y = anomaly)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  labs(
    title = "Global Temperature Anomalies as a Data Frame",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

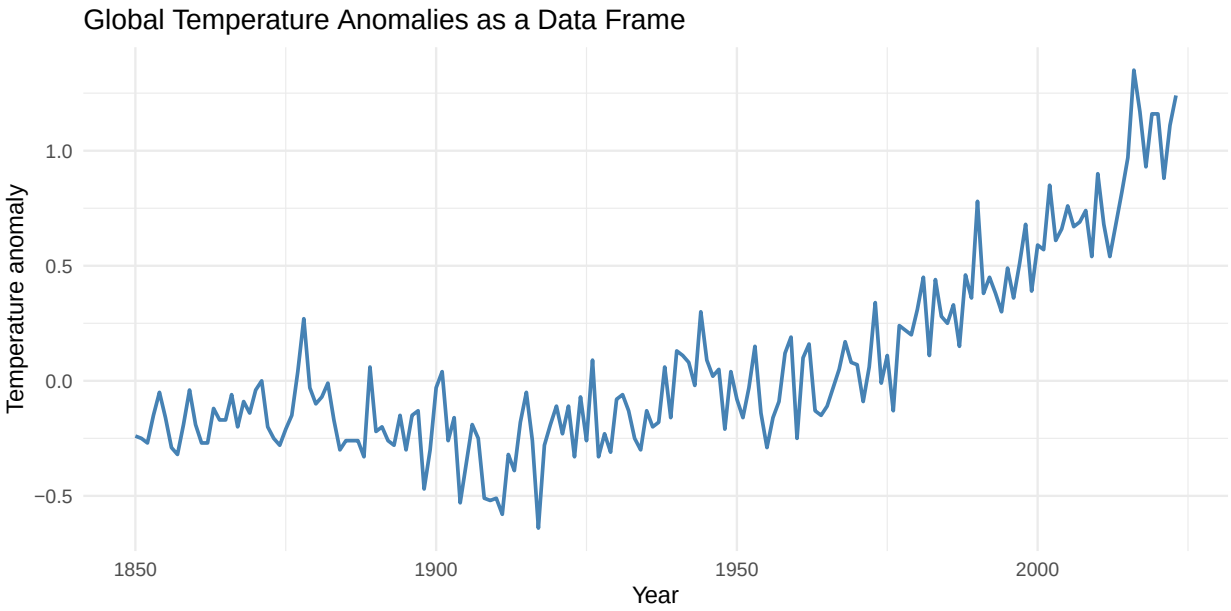


Figure 3.1.: Global temperature anomalies represented as a plain data frame and plotted with `ggplot2`. Time is stored explicitly in the `year` column rather than in a specialized time-series object.

Comment

These options illustrate the baseline representations that are easiest to inspect, reshape, and export. The trade-off is that temporal semantics are not enforced automatically. If unsure, consider the `tsbox` library.

Additional work

Pick another dataset and play with the dataset. There are more exercises in the final chapter.

```
#! eval: false
# This code is only informative!
library(tsbox)
library(tibbletime)
data(gtemp_land_ts, package = "timeSeriesDataSets")
data(gtemp_ocean_ts, package = "timeSeriesDataSets")
x.ts <- ts_c(gtemp_land_ts, gtemp_ocean_ts)
head(x.ts)
```

```
Time Series:
Start = 1850
End = 1855
```

```
Frequency = 1
     gtemp_land_ts gtemp_ocean_ts
1850          -0.5          -0.12
1851          -0.6          -0.08
1852          -0.5          -0.14
1853          -0.5           0.04
1854          -0.2           0.04
1855          -0.5           0.00
```

```
x.xts <- ts_xts(x.ts)
head(x.xts)
```

```
           gtemp_land_ts gtemp_ocean_ts
1850-01-01          -0.5          -0.12
1851-01-01          -0.6          -0.08
1852-01-01          -0.5          -0.14
1853-01-01          -0.5           0.04
1854-01-01          -0.2           0.04
1855-01-01          -0.5           0.00
```

```
x.df <- ts_df(x.xts)
head(x.df)
```

```
             id       time value
1 gtemp_land_ts 1850-01-01  -0.5
2 gtemp_land_ts 1851-01-01  -0.6
3 gtemp_land_ts 1852-01-01  -0.5
4 gtemp_land_ts 1853-01-01  -0.5
5 gtemp_land_ts 1854-01-01  -0.2
6 gtemp_land_ts 1855-01-01  -0.5
```

```
x.dt <- ts_dt(x.df)
head(x.dt)
```

```
              id       time value
          <char>     <Date> <num>
1: gtemp_land_ts 1850-01-01  -0.5
2: gtemp_land_ts 1851-01-01  -0.6
3: gtemp_land_ts 1852-01-01  -0.5
4: gtemp_land_ts 1853-01-01  -0.5
5: gtemp_land_ts 1854-01-01  -0.2
6: gtemp_land_ts 1855-01-01  -0.5
```

```
x.tbl <- ts_tbl(x.dt)
head(x.tbl)
```

```
# A tibble: 6 x 3
  id            time       value
  <chr>         <date>     <dbl>
1 gtemp_land_ts 1850-01-01  -0.5
2 gtemp_land_ts 1851-01-01  -0.6
3 gtemp_land_ts 1852-01-01  -0.5
4 gtemp_land_ts 1853-01-01  -0.5
5 gtemp_land_ts 1854-01-01  -0.2
6 gtemp_land_ts 1855-01-01  -0.5
```

```
x.zoo <- ts_zoo(x.tbl)
head(x.zoo)
```

```
           gtemp_land_ts gtemp_ocean_ts
1850-01-01          -0.5          -0.12
1851-01-01          -0.6          -0.08
1852-01-01          -0.5          -0.14
1853-01-01          -0.5           0.04
1854-01-01          -0.2           0.04
1855-01-01          -0.5           0.00
```

```
x.tsibble <- ts_tsibble(x.zoo)
head(x.tsibble)
```

```
# A tsibble: 6 x 3 [1D]
# Key:       id [1]
  id            time       value
  <chr>         <date>     <dbl>
1 gtemp_land_ts 1850-01-01  -0.5
2 gtemp_land_ts 1851-01-01  -0.6
3 gtemp_land_ts 1852-01-01  -0.5
4 gtemp_land_ts 1853-01-01  -0.5
5 gtemp_land_ts 1854-01-01  -0.2
6 gtemp_land_ts 1855-01-01  -0.5
```

```
x.tibbletime <- ts_tibbletime(x.tsibble)
head(x.tibbletime)
```

```
# A time tibble: 6 x 3
# Index:         time
  time       gtemp_land_ts gtemp_ocean_ts
  <date>             <dbl>          <dbl>
1 1850-01-01          -0.5          -0.12
2 1851-01-01          -0.6          -0.08
3 1852-01-01          -0.5          -0.14
4 1853-01-01          -0.5           0.04
5 1854-01-01          -0.2           0.04
6 1855-01-01          -0.5           0
```

```
x.timeSeries <- ts_timeSeries(x.tibbletime)
head(x.timeSeries)
```

```
GMT
           gtemp_land_ts gtemp_ocean_ts
1850-01-01          -0.5          -0.12
1851-01-01          -0.6          -0.08
1852-01-01          -0.5          -0.14
1853-01-01          -0.5           0.04
1854-01-01          -0.2           0.04
1855-01-01          -0.5           0.00
```

```
all.equal(ts_ts(x.timeSeries), x.ts)  # TRUE
```

```
[1] TRUE
```

3.4. Time Series in Various Formats

Using the same global temperature series, `gtemp_both_ts`, and storing them in several R time-series representations.

3.4.1. `ts`

```
gtemp_ts <- gtemp_both_ts
class(gtemp_ts)
```

```
[1] "ts"
```

```
head(gtemp_ts)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
[1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

Notice that this is the native format for `timeSeriesDataSets`!

3.4.2. `zoo`

```
gtemp_zoo <- zoo(
  as.numeric(gtemp_both_ts),
  order.by = as.numeric(time(gtemp_both_ts))
)
class(gtemp_zoo)
```

```
[1] "zoo"
```

```
head(gtemp_zoo)
```

```
 1850  1851  1852  1853  1854  1855
-0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

For `zoo` and `xts`, we have to manipulate the different dimensions of data.

3.4.3. `xts`

```
gtemp_years <- floor(as.numeric(time(gtemp_both_ts)))
gtemp_xts <- xts(
  as.numeric(gtemp_both_ts),
  order.by = as.Date(paste0(gtemp_years, "-01-01"))
)
class(gtemp_xts)
```

```
[1] "xts" "zoo"
```

```
head(gtemp_xts)
```

```
            [,1]
1850-01-01 -0.24
1851-01-01 -0.25
1852-01-01 -0.27
1853-01-01 -0.15
1854-01-01 -0.05
1855-01-01 -0.16
```

3.4.4. tsibble

```
gtemp_tsibble <- tibble(
  year = gtemp_years,
  anomaly = as.numeric(gtemp_both_ts)
) |>
  as_tsibble(index = year)

class(gtemp_tsibble)
```

```
[1] "tbl_ts"     "tbl_df"     "tbl"        "data.frame"
```

```
gtemp_tsibble
```

```
# A tsibble: 174 x 2 [1Y]
    year anomaly
   <dbl>   <dbl>
 1  1850   -0.24
 2  1851   -0.25
 3  1852   -0.27
 4  1853   -0.15
 5  1854   -0.05
 6  1855   -0.16
 7  1856   -0.29
 8  1857   -0.32
 9  1858   -0.19
10  1859   -0.04
# i 164 more rows
```

3.5. tsbox

With tsbox, you have some handy conversions, from/to any format (supported by the package). Just check if your results are the same, for the various options.

```
tsb_tsibble <- ts_tsibble(gtemp_both_ts)
class(tsb_tsibble)
```

```
[1] "tbl_ts"     "tbl_df"     "tbl"        "data.frame"
```

```
tsb_tsibble
```

```
# A tsibble: 174 x 2 [1D]
   time       value
   <date>     <dbl>
 1 1850-01-01 -0.24
 2 1851-01-01 -0.25
 3 1852-01-01 -0.27
 4 1853-01-01 -0.15
 5 1854-01-01 -0.05
 6 1855-01-01 -0.16
 7 1856-01-01 -0.29
 8 1857-01-01 -0.32
 9 1858-01-01 -0.19
10 1859-01-01 -0.04
# i 164 more rows
```

3.5.1. Plot the ts object with ggplot2::autoplot()

```
# Use ggfortify method via ggplot2::autoplot
# This is more Quarto-safe than relying on a plain autoplot() call.
ggplot2::autoplot(
  gtemp_ts,
  ts.colour = "darkorange",
  ts.size = 0.8,
  main = "Global Temperature Anomalies as a ts Object",
  xlab = "Year",
  ylab = "Temperature anomaly"
) +
  theme_minimal()
```

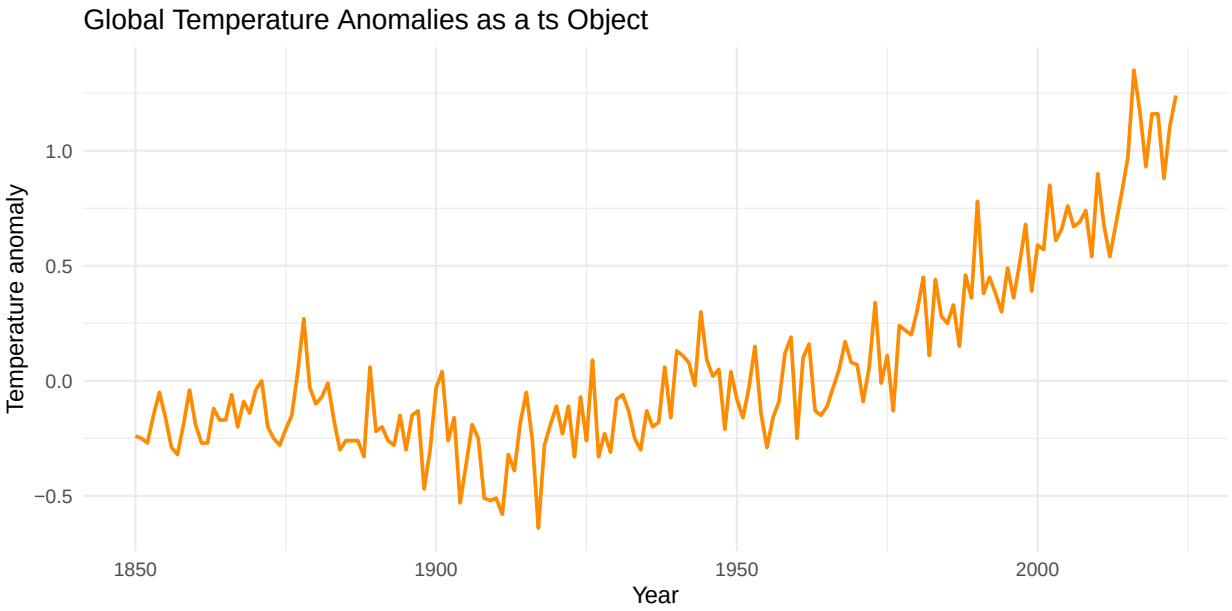


Figure 3.2.: The `gtemp_both_ts` series plotted as a classical `ts` object. In this case, the temporal structure is embedded directly in the object and handled automatically by time-series-aware plotting methods.

Additional work

Check the various conversion formats supported by `tsbox` and repeat previous transformations. Try to plot the auto-generated outputs (check if `plot`, `ggplot`, or `autoplot` is available for your dataset)
There are more exercises in the final chapter.

! Notice

We can view the major representational philosophies for time series:

- `ts`: classical approach (and compact);
- `xts` (also, `zoo`): indexed and especially useful when time stamp matter;
- `tsibble`: tidy and explicit, well suited to modern pipelines.
- `tsbox`: quick and dirty, when you're happy with automatic conversions.

3.6. Import Datasets from the File System

In practice, data rarely start as ready-made R time series objects. Let's inspect some of the available options (based on the temperature example).

3.6.1. From CSV

```r
# Simulate a CSV import using the running example
csv_path <- tempfile(fileext = ".csv")
# Notice that gtemp_df is from a previous example (data frames)
write.csv(gtemp_df, csv_path, row.names = FALSE)
# Ignore previous lines, if you already have a CSV file

gtemp_from_csv <- read.csv(csv_path)
head(gtemp_from_csv)
```

```
  year anomaly
1 1850   -0.24
2 1851   -0.25
3 1852   -0.27
4 1853   -0.15
5 1854   -0.05
6 1855   -0.16
```

3.6.2. From RDS

```r
# Simulate a RDS import using the running example
rds_path <- tempfile(fileext = ".rds")
# Notice that gtemp_df is from a previous example (data frames)
saveRDS(gtemp_df, rds_path)
# Ignore previous lines, if you already have a CSV file

gtemp_from_rds <- readRDS(rds_path)
head(gtemp_from_rds)
```

```
# A tibble: 6 x 2
   year anomaly
  <dbl>   <dbl>
1  1850   -0.24
2  1851   -0.25
3  1852   -0.27
```

```
4  1853   -0.15
5  1854   -0.05
6  1855   -0.16
```

3.6.3. Plain text & `scan()`

```
# Simulate a simple file import using the running example
text_path <- tempfile(fileext = ".txt")
# Notice that gtemp_both_ts has to be loaded (`data (...)`)
write(as.numeric(gtemp_both_ts), text_path)
# Ignore previous lines, if you already have a CSV file

gtemp_from_text <- scan(text_path, quiet = TRUE)
head(gtemp_from_text)
```

```
[1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

3.6.4. Excel (template)

```
# Library readxl is needed (and a real Excel file in the project)
library(readxl)

gtemp_from_excel <- read_excel("data/gtemp_both.xlsx")
head(gtemp_from_excel)
```

Discussion

We may distinguish two important steps:

1. **importing the raw data** from some storage format;
2. **converting the imported object** into a time-series representation appropriate for analysis.
   (You can use `tsbox` when easy conversions are sufficient).

Notice

There are more options for supporting data imports. Check the `readr` package (already included in `tidyverse`): `read_delim()`, `read_tsv()`, `read_csv`, `read_table()`, `read_fwf()`. You can check the available cheatsheet for `readr` and other options, on https://github.com/rstudio/cheatsheets/

Additional work

Repeat one of previous exercises from this chapter, for the imported data.
There are more exercises in the final chapter.

3.7. Convert Data into Time-Series Objects

Once the data are loaded into R, the next step is usually to convert them into a dedicated time-series representation.

3.7.1. `ts`

```
gtemp_ts_from_df <- ts(
  gtemp_from_csv$anomaly,
  start = min(gtemp_from_csv$year),
  frequency = 1
)
head(gtemp_ts_from_df)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
[1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

3.7.2. `zoo`

```
gtemp_zoo_from_df <- zoo(
  gtemp_from_csv$anomaly,
  order.by = gtemp_from_csv$year
)
head(gtemp_zoo_from_df)
```

```
 1850  1851  1852  1853  1854  1855
-0.24 -0.25 -0.27 -0.15 -0.05 -0.16
```

3.7.3. `xts`

```r
gtemp_xts_from_df <- xts(
  gtemp_from_csv$anomaly,
  order.by = as.Date(paste0(gtemp_from_csv$year, "-01-01"))
)
head(gtemp_xts_from_df)
```

```
           [,1]
1850-01-01 -0.24
1851-01-01 -0.25
1852-01-01 -0.27
1853-01-01 -0.15
1854-01-01 -0.05
1855-01-01 -0.16
```

3.7.4. `tsibble`

```r
gtemp_tsibble_from_df <- as_tsibble(gtemp_from_csv, index = year)
gtemp_tsibble_from_df
```

```
# A tsibble: 174 x 2 [1Y]
    year anomaly
   <int>   <dbl>
 1  1850   -0.24
 2  1851   -0.25
 3  1852   -0.27
 4  1853   -0.15
 5  1854   -0.05
 6  1855   -0.16
 7  1856   -0.29
 8  1857   -0.32
 9  1858   -0.19
10  1859   -0.04
# i 164 more rows
```

> **Comments**
>
> These conversions make the difference between **tabular storage** and **time-aware analysis** explicit. The same imported values can support very different workflows depending on how they are encoded after import.

3.8. Dealing with Multiple Measurements at Each Time Point

The global temperature datasets also allow a natural comparison between univariate and multiple-series representations.

3.8.1. Wide data frame

```
gtemp_multi_df <- tibble(
  year = gtemp_years,
  land_ocean = as.numeric(gtemp_both_ts),
  land = as.numeric(gtemp_land_ts),
  ocean = as.numeric(gtemp_ocean_ts)
)

head(gtemp_multi_df)
```

```
# A tibble: 6 x 4
   year land_ocean  land ocean
  <dbl>      <dbl> <dbl> <dbl>
1  1850      -0.24  -0.5 -0.12
2  1851      -0.25  -0.6 -0.08
3  1852      -0.27  -0.5 -0.14
4  1853      -0.15  -0.5  0.04
5  1854      -0.05  -0.2  0.04
6  1855      -0.16  -0.5  0
```

3.8.2. Multivariate `ts`

```
gtemp_multi_ts <- ts(
  cbind(
    land_ocean = as.numeric(gtemp_both_ts),
    land = as.numeric(gtemp_land_ts),
    ocean = as.numeric(gtemp_ocean_ts)
  ),
  start = start(gtemp_both_ts),
  frequency = frequency(gtemp_both_ts)
)

head(gtemp_multi_ts)
```

```
Time Series:
Start = 1850
End = 1855
Frequency = 1
     land_ocean land ocean
1850      -0.24 -0.5 -0.12
1851      -0.25 -0.6 -0.08
1852      -0.27 -0.5 -0.14
1853      -0.15 -0.5  0.04
1854      -0.05 -0.2  0.04
1855      -0.16 -0.5  0.00
```

3.8.3. Multivariate `xts`

```
gtemp_multi_xts <- xts(
  gtemp_multi_df[, c("land_ocean", "land", "ocean")],
  order.by = as.Date(paste0(gtemp_multi_df$year, "-01-01"))
)

head(gtemp_multi_xts)
```

```
           land_ocean land ocean
1850-01-01      -0.24 -0.5 -0.12
1851-01-01      -0.25 -0.6 -0.08
1852-01-01      -0.27 -0.5 -0.14
1853-01-01      -0.15 -0.5  0.04
1854-01-01      -0.05 -0.2  0.04
1855-01-01      -0.16 -0.5  0.00
```

3.8.4. Long/keyed `tsibble`

```
gtemp_multi_tsibble <- gtemp_multi_df |>
  pivot_longer(
    cols = c(land_ocean, land, ocean),
    names_to = "series",
    values_to = "anomaly"
  ) |>
  as_tsibble(index = year, key = series)

head(gtemp_multi_tsibble)
```

```
# A tsibble: 6 x 3 [1Y]
# Key:       series [1]
   year series anomaly
  <dbl> <chr>    <dbl>
1  1850 land      -0.5
2  1851 land      -0.6
3  1852 land      -0.5
4  1853 land      -0.5
5  1854 land      -0.2
6  1855 land      -0.5
```

3.8.5. Plotting...

Plot multiple measurements, use one of the available data formats.

```
ggplot2::ggplot(gtemp_multi_tsibble, aes(x = year, y = anomaly, color = series)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Global Temperature Anomaly Series",
    x = "Year",
    y = "Temperature anomaly",
    color = "Series"
  ) +
  theme_minimal()
```

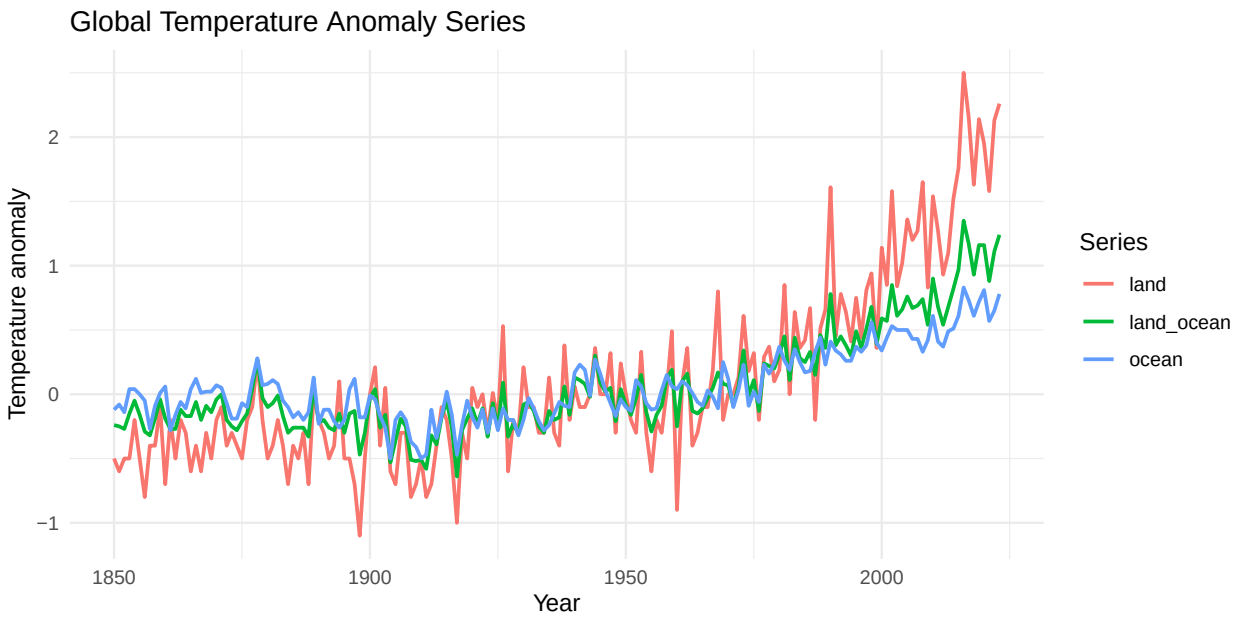


Figure 3.3.: Three related temperature anomaly series represented together: combined land-and-ocean, land-only, and ocean-only. This illustrates a case where each time point carries multiple measurements rather than a single scalar value.

3.8.6.

> **Notice**
>
> This section will allow you to distinguishing:
>
> - **multivariate storage** (`mts`, wide `xts`),
> - **multiple-series tidy storage** (keyed `tsibble`), and
> - a simple wide tabular representation intended for interoperability.

3.9. Back to Conversions

Time-series analysis in R often requires jumping between frameworks. Usually a handy tool, like `tsbox`, will solve this kind of problem.

3.9.1. `tsbox` to a tidy table

```
tsbox::ts_tbl(gtemp_both_ts)
```

```
# A tibble: 174 x 2
   time       value
   <date>     <dbl>
 1 1850-01-01 -0.24
 2 1851-01-01 -0.25
 3 1852-01-01 -0.27
 4 1853-01-01 -0.15
 5 1854-01-01 -0.05
 6 1855-01-01 -0.16
 7 1856-01-01 -0.29
 8 1857-01-01 -0.32
 9 1858-01-01 -0.19
10 1859-01-01 -0.04
# i 164 more rows
```

3.9.2. `tsbox` to `xts`

```
tsbox::ts_xts(gtemp_both_ts)
```

```
           value
1850-01-01 -0.24
1851-01-01 -0.25
1852-01-01 -0.27
1853-01-01 -0.15
1854-01-01 -0.05
1855-01-01 -0.16
1856-01-01 -0.29
1857-01-01 -0.32
1858-01-01 -0.19
1859-01-01 -0.04
       ...
2014-01-01  0.82
2015-01-01  0.97
2016-01-01  1.35
2017-01-01  1.17
2018-01-01  0.93
2019-01-01  1.16
2020-01-01  1.16
2021-01-01  0.88
2022-01-01  1.11
2023-01-01  1.24
```

3.9.3. `tsbox` to `tsibble`

```
tsbox::ts_tsibble(gtemp_both_ts)
```

```
# A tsibble: 174 x 2 [1D]
   time       value
   <date>     <dbl>
 1 1850-01-01 -0.24
 2 1851-01-01 -0.25
 3 1852-01-01 -0.27
 4 1853-01-01 -0.15
 5 1854-01-01 -0.05
 6 1855-01-01 -0.16
 7 1856-01-01 -0.29
 8 1857-01-01 -0.32
 9 1858-01-01 -0.19
10 1859-01-01 -0.04
# i 164 more rows
```

3.9.4. `tsibble` back to another format

```
gtemp_tsibble_simple <- tsbox::ts_tsibble(gtemp_both_ts)
tsbox::ts_ts(gtemp_tsibble_simple)
```

```
Time Series:
Start = 1850
End = 2023
Frequency = 1
  [1] -0.24 -0.25 -0.27 -0.15 -0.05 -0.16 -0.29 -0.32 -0.19 -0.04 -0.19 -0.27
 [13] -0.27 -0.12 -0.17 -0.17 -0.06 -0.20 -0.09 -0.14 -0.04  0.00 -0.20 -0.25
 [25] -0.28 -0.21 -0.15  0.04  0.27 -0.03 -0.10 -0.07 -0.01 -0.17 -0.30 -0.26
 [37] -0.26 -0.26 -0.33  0.06 -0.22 -0.20 -0.26 -0.28 -0.15 -0.30 -0.15 -0.13
 [49] -0.47 -0.30 -0.03  0.04 -0.26 -0.16 -0.53 -0.36 -0.19 -0.25 -0.51 -0.52
 [61] -0.51 -0.58 -0.32 -0.39 -0.18 -0.05 -0.26 -0.64 -0.28 -0.19 -0.11 -0.23
 [73] -0.11 -0.33 -0.07 -0.26  0.09 -0.33 -0.23 -0.31 -0.08 -0.06 -0.13 -0.25
 [85] -0.30 -0.13 -0.20 -0.18  0.06 -0.16  0.13  0.11  0.08 -0.02  0.30  0.09
 [97]  0.02  0.05 -0.21  0.04 -0.08 -0.16 -0.03  0.15 -0.14 -0.29 -0.16 -0.09
[109]  0.12  0.19 -0.25  0.10  0.16 -0.13 -0.15 -0.11 -0.03  0.05  0.17  0.08
[121]  0.07 -0.09  0.06  0.34 -0.01  0.11 -0.13  0.24  0.22  0.20  0.31  0.45
[133]  0.11  0.44  0.28  0.25  0.33  0.15  0.46  0.36  0.78  0.38  0.45  0.38
[145]  0.30  0.49  0.36  0.51  0.68  0.39  0.59  0.57  0.85  0.61  0.66  0.76
[157]  0.67  0.69  0.74  0.54  0.90  0.68  0.54  0.68  0.82  0.97  1.35  1.17
[169]  0.93  1.16  1.16  0.88  1.11  1.24
```

> **Notice**
>
> Conversion is often unavoidable when combining packages.
> A representation is not only a storage choice, but also a gateway to a particular computational ecosystem.

3.10. The Right Representation

Choosing the right representation is not an easy task. Just some hints on the different advantages you can get from the various options.

3.10.1. `ts, when...`

- the data are regular;
- the focus is on classical time series methods;
- compact storage is sufficient.

3.10.2. `zoo, xts``, when...`

- the index should be explicit;
- dates or date-times matter;
- irregularity is possible;
- the workflow is finance- or index-oriented.

3.10.3. `tsibble``, when...`

- the workflow is tidyverse-oriented;
- multiple series must be managed together;
- explicit keys and indices are desirable;
- modern forecasting pipelines are expected.

3.10.4. data frames `tsbox``, when...`

- import/export interoperability is central;
- the original files are not (yet) in a time-series structure;
- conversion between formats is part of the workflow.

> **Notice**
>
> A useful decision sequence is:
>
> 1. Where does the data come from?

2. Is the index regular or irregular?
3. Are there one or many series?
4. Which ecosystem will be used next: base R, indexed objects, or tidy forecasting tools?

3.11. Summary

we offered a naive comparison for the different ways of representing time series in R (usually around the global temperature example). The different alternatives were identified, either for representations (direct package datasets, plain vectors, and data frames), or by mechanisms (`ts` objects, `zoo`/`xts`, tidy `tsibble`s, and `tsbox` conversions).

In fact, representation choices begin **at import time**. Reading data from CSV, RDS, text, JSON, or Excel is only the first step; the crucial second step is converting the imported values into a structure that supports the subsequent activities.

4. Plotting Time Series

4.1. Introduction

First step in understanding a time series is vizualization. While time series analysis offer most valuable information, it is useful to inspect the data visually in order to identify and understand some structural patterns such as long-term movement, seasonality, irregular fluctuations, abrupt changes, or unusual observations.

We're going to explore several options for plotting time series in R. We're using similar examples with those from previous chapters. The goal is not only to produce plots, but also to show how the choice of representation affects plotting syntax and graphical flexibility.

> **Note**
>
> Our comparison will cover basic aspects from time series frameworks, especially base `ts`, `zoo`/`xts`, and tidy plotting pipelines built around `ggplot2` and `tsibble`.

```
# The libraries (the full set, as previously identified)
library(ggplot2)
library(ggfortify)
library(zoo)
library(xts)
library(tsibble)
library(dplyr)
library(tibble)
library(tidyr)
library(timeSeriesDataSets)
library(tsbox)

# Running and loading example datasets
data(gtemp_both_ts, package = "timeSeriesDataSets")
data(gtemp_land_ts, package = "timeSeriesDataSets")
data(gtemp_ocean_ts, package = "timeSeriesDataSets")

# Common helper objects
gtemp_years <- floor(as.numeric(time(gtemp_both_ts)))
```

```
# gtemp as dataframes (explicit conversion, also check tsbox options)
gtemp_df <- tibble(
  year = as.numeric(time(gtemp_both_ts)),
  anomaly = as.numeric(gtemp_both_ts)
)

# gtemp as tsiblle objects (explicit conversion, also check tsbox options)
gtemp_tsibble <- tibble(
  year = gtemp_years,
  anomaly = as.numeric(gtemp_both_ts)
) |>
  as_tsibble(index = year)

# gtemp as zoo (explicit conversion, also check tsbox options)
gtemp_zoo <- zoo(
  as.numeric(gtemp_both_ts),
  order.by = as.numeric(time(gtemp_both_ts))
)

# gtemp as xts (explicit conversion, also check tsbox options)
gtemp_xts <- xts(
  cbind(
    land_ocean = as.numeric(gtemp_both_ts),
    land = as.numeric(gtemp_land_ts),
    ocean = as.numeric(gtemp_ocean_ts)
  ),
  order.by = as.Date(paste0(gtemp_years, "-01-01"))
)

# gtemp as multivariate ts (explicit conversion, also check tsbox options)
gtemp_multi_ts <- ts(
  cbind(
    land_ocean = as.numeric(gtemp_both_ts),
    land = as.numeric(gtemp_land_ts),
    ocean = as.numeric(gtemp_ocean_ts)
  ),
  start = start(gtemp_both_ts),
  frequency = frequency(gtemp_both_ts)
)

# gtemp as multivariate dataframes (explicit conversion, also check tsbox options)
gtemp_multi_df <- tibble(
  year = gtemp_years,
  land_ocean = as.numeric(gtemp_both_ts),
```

```
  land = as.numeric(gtemp_land_ts),
  ocean = as.numeric(gtemp_ocean_ts)
)

# gtemp as multivariate tsibble (explicit conversion, also check tsbox options)
gtemp_multi_tsibble <- gtemp_multi_df |>
  pivot_longer(
    cols = c(land_ocean, land, ocean),
    names_to = "series",
    values_to = "anomaly"
  ) |>
  as_tsibble(index = year, key = series)

# Additional multivariate examples for panel layouts
four_ts <- EuStockMarkets[, 1:4]
# eu_xts  <- as.xts(EuStockMarkets)
eu_xts  <- ts_xts(EuStockMarkets)
```

4.2. Your First Plot of a Time Series

In fact, you already have some experiences with time series plots. Just a reminder for the direct plotting options.

4.2.1. Plotting (`ts`)

```
plot(
  gtemp_both_ts,
  col = "steelblue",
  lwd = 2,
  main = "Global Temperature Anomalies (base plot)",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
```

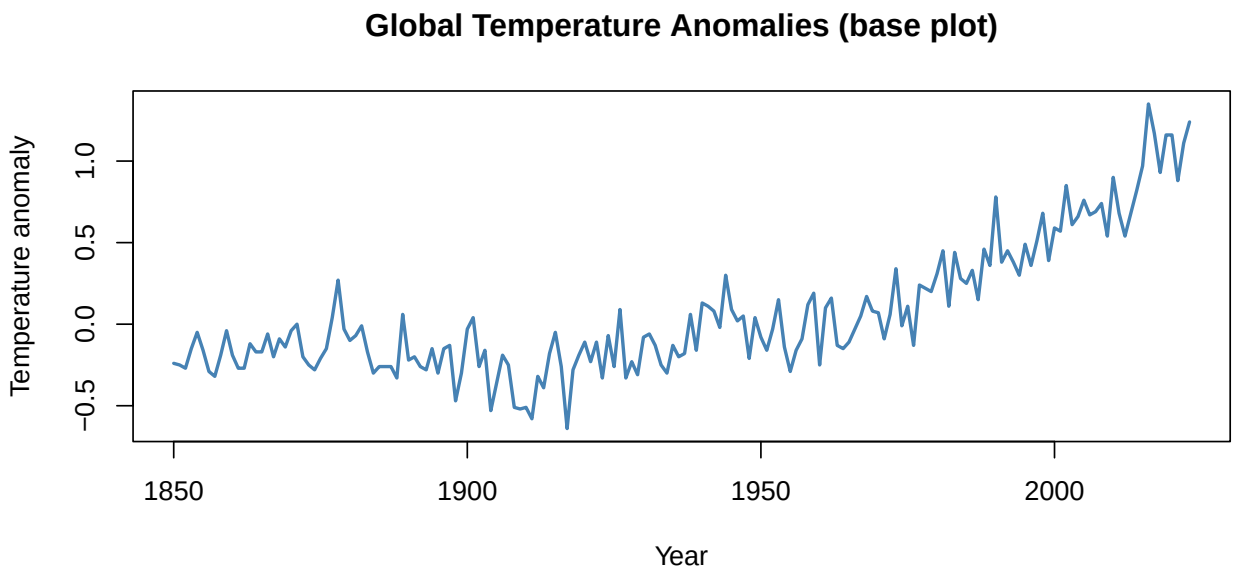


Figure 4.1.: A first plot of the global temperature anomaly series using base R on a classical `ts` object.

4.2.2. Plotting (`df`), `ggplot2`

```r
ggplot(gtemp_df, aes(x = year, y = anomaly)) +
  geom_line(color = "steelblue", linewidth = 0.9) +
  labs(
    title = "Global Temperature Anomalies (ggplot2)",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

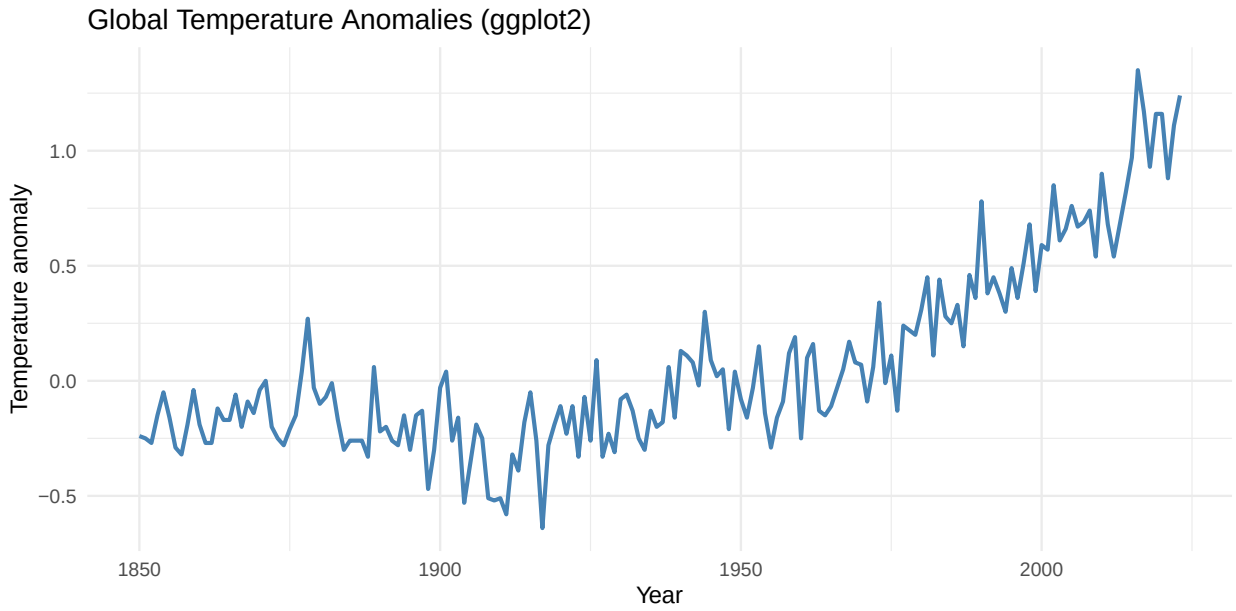


Figure 4.2.: The same series plotted from a plain data frame using `ggplot2`.

4.2.3. Plotting (`ts`), `ggplot2::autoplot()`

```
ggplot2::autoplot(
  gtemp_both_ts,
  ts.colour = "darkorange",
  ts.size = 0.8,
  main = "Global Temperature Anomalies (autoplot)",
  xlab = "Year",
  ylab = "Temperature anomaly"
) +
  theme_minimal()
```

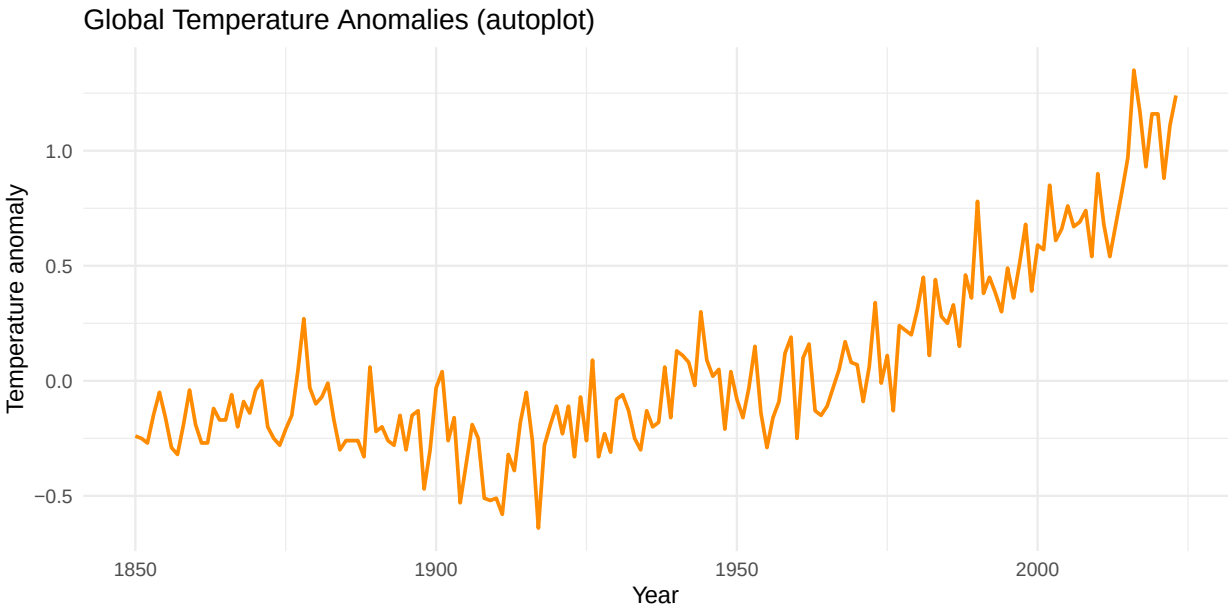


Figure 4.3.: Autoplot of the global temperature anomaly series from a `ts` object.

> **Observation**
>
> The three alternatives were already used in previous recipes: direct base plotting, explicit `ggplot2` plotting from tabular data, and automatic plotting from a time-series object. Please notice that `ggplot` and `autoplot` use a simplified syntax in these examples. Check the available options (see `aes` – aesthetics, `labs` – labels, and other options).

4.3. `plot.ts()` and Variants

The `plot.ts()` method can be used explicitly, covering both uni- and multi-variate time series.

4.3.1. univariate

```
plot.ts(
  gtemp_both_ts,
  col = "steelblue",
  lwd = 2,
  main = "Global Temperature Anomalies with plot.ts()",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
```

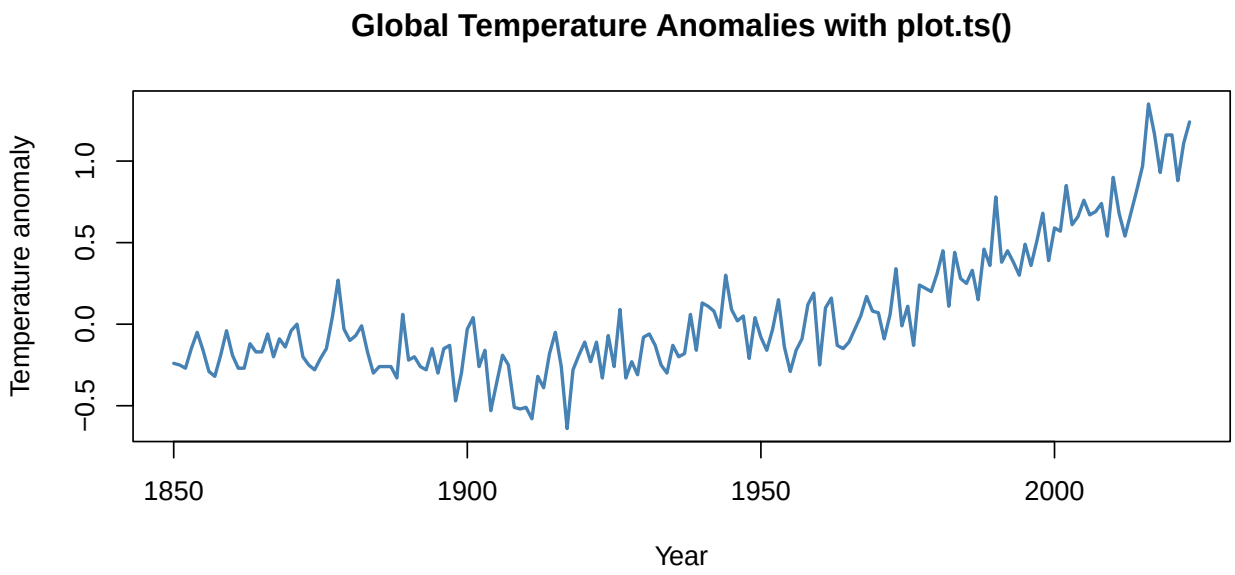


Figure 4.4.: Explicit use of `plot.ts()` for a univariate `ts` object.

4.3.2. multivariate, one plot

```
plot.ts(
  gtemp_multi_ts,
  plot.type = "single",
  col = c("steelblue", "firebrick", "darkgreen"),
  lty = 1:3,
  lwd = 2,
  main = "Temperature Series on a Single Plot",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
legend(
  "topleft",
  legend = colnames(gtemp_multi_ts),
  col = c("steelblue", "firebrick", "darkgreen"),
  lty = 1:3,
  lwd = 2,
  bty = "n"
)
```

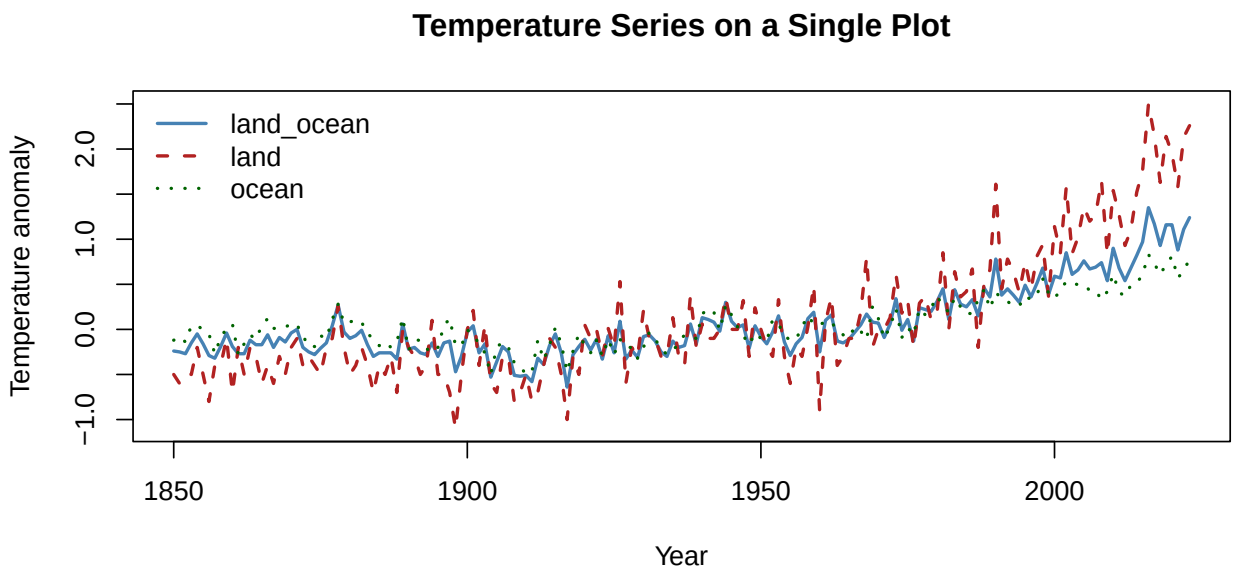


Figure 4.5.: A multivariate `ts` object displayed on a single plot with `plot.ts(plot.type = "single")`.

4.3.3. multivariate, several plots

```
plot.ts(
  gtemp_multi_ts,
  plot.type = "multiple",
  col = c("steelblue", "firebrick", "darkgreen"),
  lwd = 2,
  main = "Temperature Series in Separate Panels",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
```

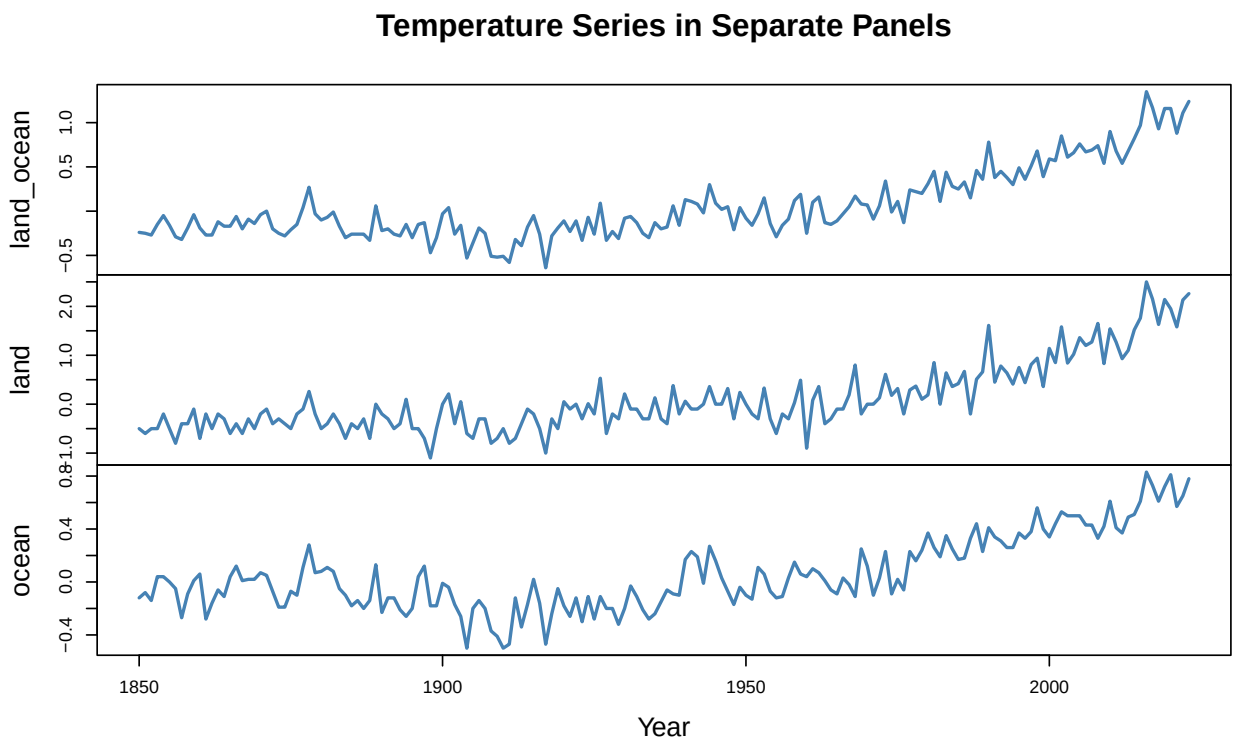


Figure 4.6.: A multivariate `ts` object displayed with separate panels via `plot.type = "multiple"`.

4.3.4. multivariate, columns (`nc`)

```
plot.ts(
  four_ts,
  plot.type = "multiple",
  nc = 2,
  col = 2:5,
  lwd = 1.5,
  main = "EuStockMarkets in a 2-Column Layout",
  xlab = "Year",
  ylab = "Index level"
)
```

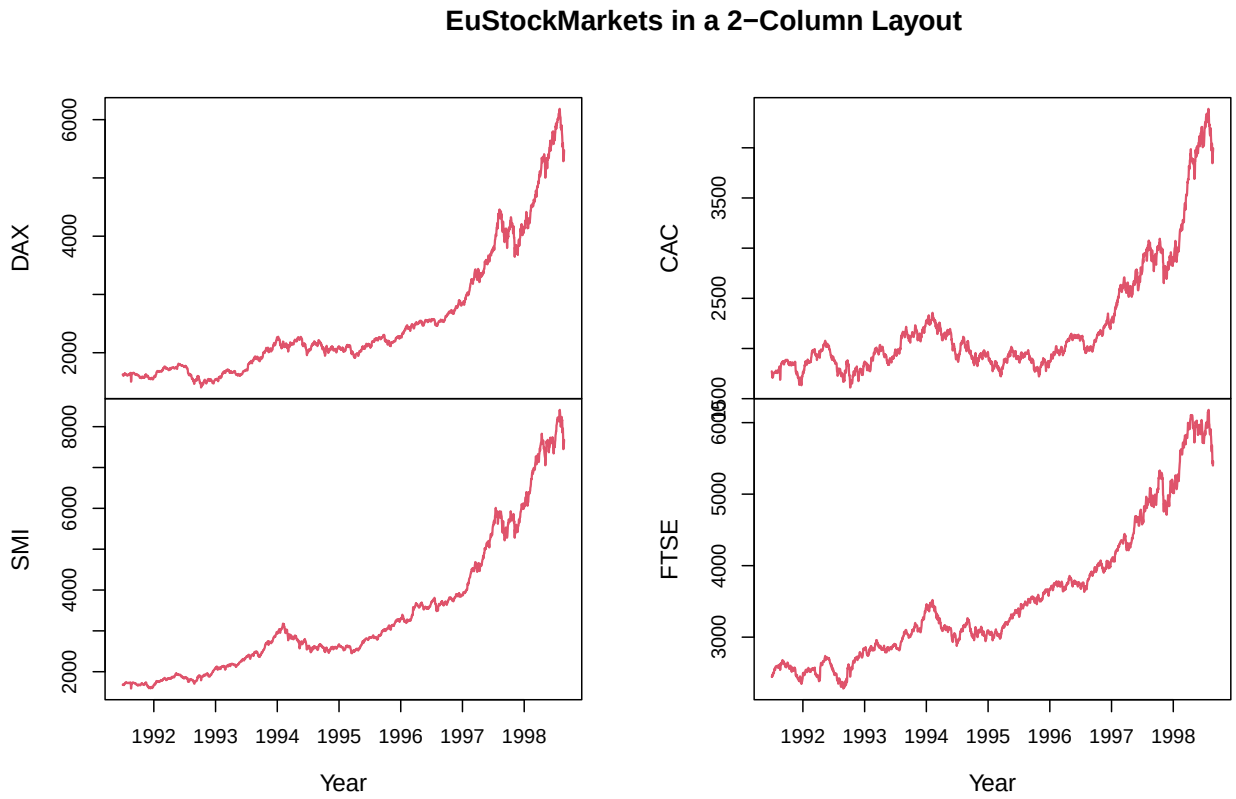


Figure 4.7.: A multivariate `ts` object arranged with multiple panels and a custom number of columns using `nc = 2`.

4.3.5. time window

```r
gtemp_window <- window(gtemp_both_ts, start = 1950, end = 2000)

plot.ts(
  gtemp_window,
  col = "darkorange",
  lwd = 2,
  main = "Temperature Anomalies, 1900-2000",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
```

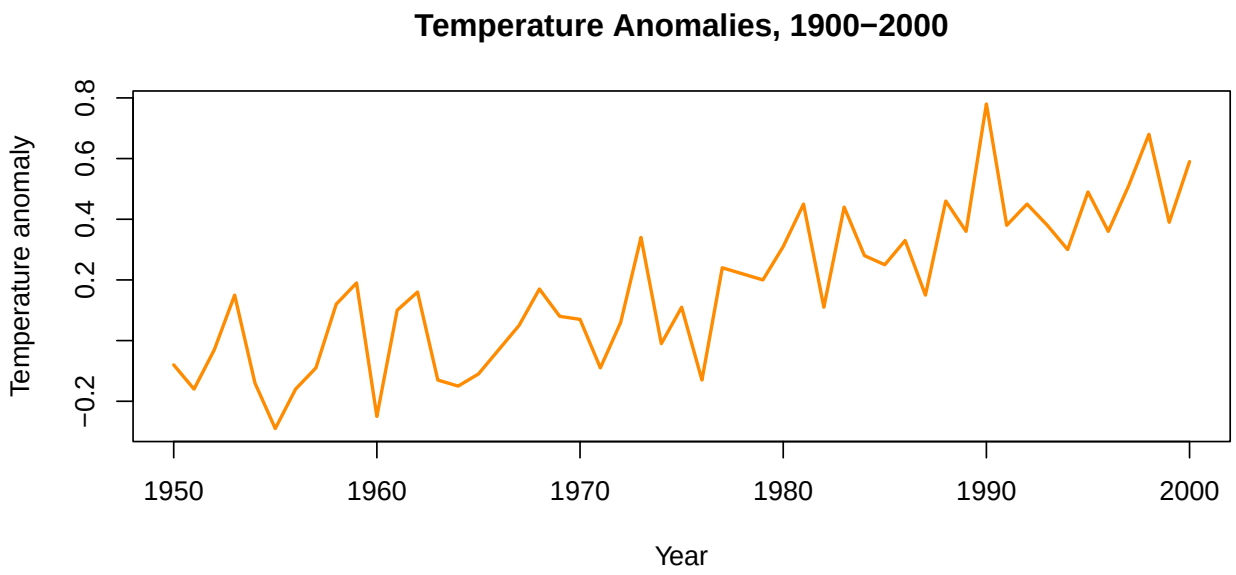


Figure 4.8.: A restricted time window of the temperature anomaly series, obtained with `window()` before plotting.

Notice the use of the `window()` function. A new frequency may be specified, if needed.

4.3.6. `aggregation()`

```
air_annual <- aggregate(AirPassengers, nfrequency = 1, FUN = mean)

plot.ts(
  air_annual,
  col = "purple",
  lwd = 2,
  main = "AirPassengers Aggregated to Annual Frequency",
  xlab = "Year",
  ylab = "Average passengers"
)
```

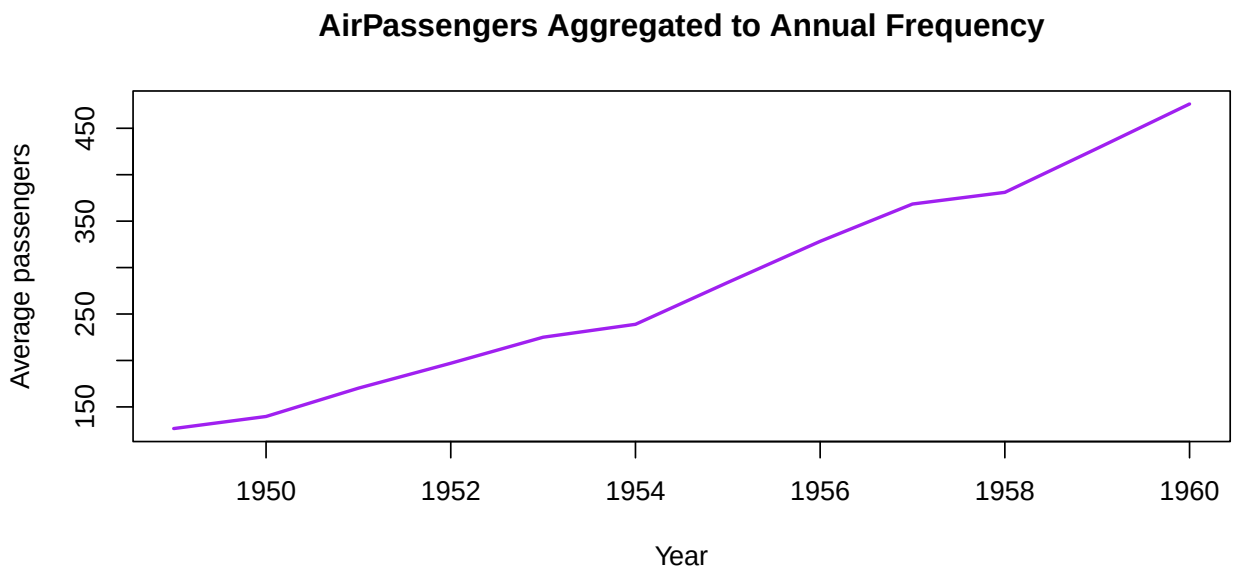


Figure 4.9.: A `ts` plot after changing the series frequency by aggregation. Here, monthly AirPassengers data are converted to annual averages before plotting.

`aggregate` was used to change the frequency (previous frequency was `frequency(AirPassengers)`), while applying the specified function.

4.3.7. X–Y / lag-style

```
plot.ts(
  stats::lag(gtemp_both_ts, -1),
  gtemp_both_ts,
  pch = 19,
  col = "navy",
  cex = 0.8,
  main = "Lag Plot of Global Temperature Anomalies",
  xlab = "Lagged anomaly",
  ylab = "Current anomaly"
)
```

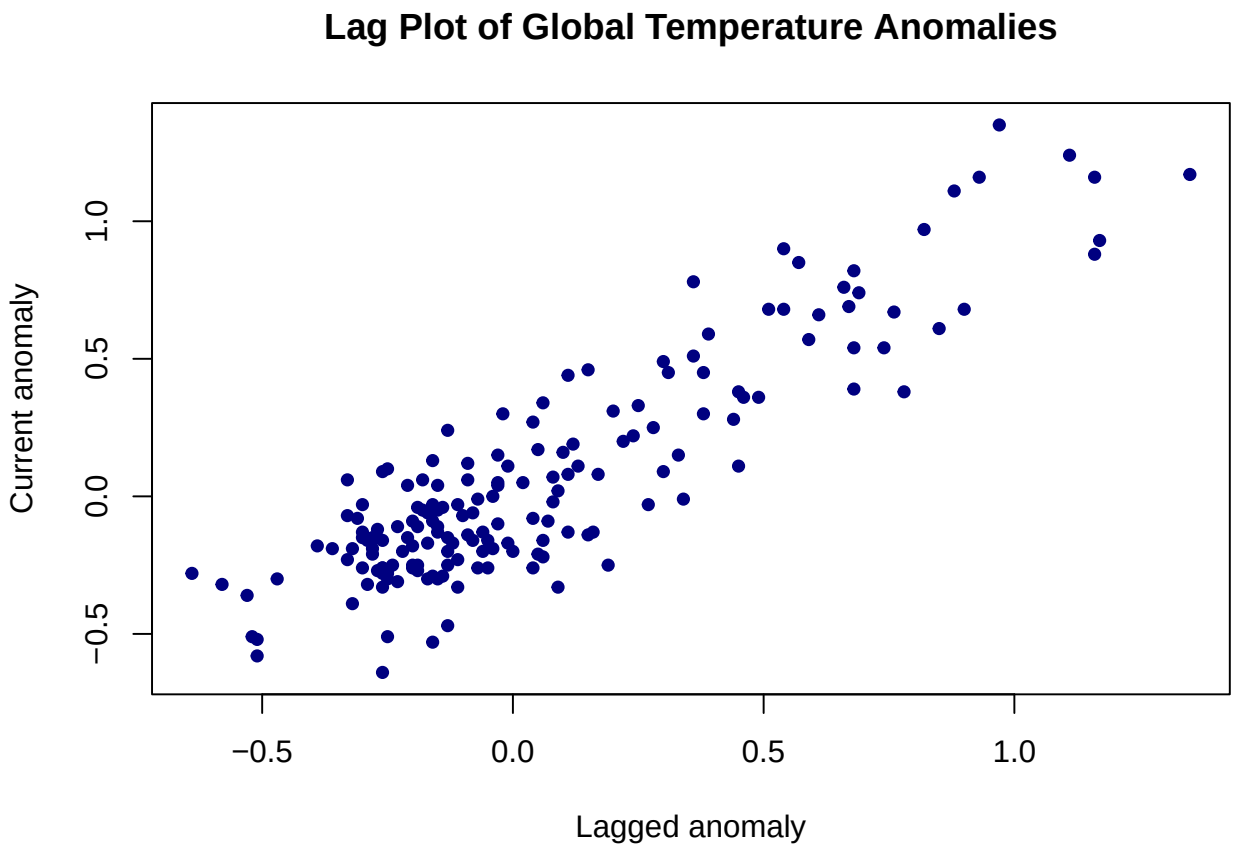


Figure 4.10.: A lag-style / phase-style plot obtained by passing two univariate `ts` objects to `plot.ts(x, y)`.

Discussion

`plot.ts()` is especially useful when working directly with `ts` and multivariate `ts` objects. Its built-in `plot.type`, `nc`, and x–y capabilities make it richer than many users initially expect.

4.4. `ts.plot()` for Common-Axis

Somehow similar to `plot.ts`, the `ts.plot()` method is well suited for plotting several time series on a common plot when they share a common frequency.

4.4.1. Multi-plots

```
ts.plot(
  gtemp_both_ts,
  gtemp_land_ts,
  gtemp_ocean_ts,
  col = c("steelblue", "firebrick", "darkgreen"),
  lty = 1:3,
  lwd = 2,
  xlab = "Year",
  ylab = "Temperature anomaly",
  main = "Temperature Series with ts.plot()"
)
legend(
  "topleft",
  legend = c("land_ocean", "land", "ocean"),
  col = c("steelblue", "firebrick", "darkgreen"),
  lty = 1:3,
  lwd = 2,
  bty = "n"
)
```

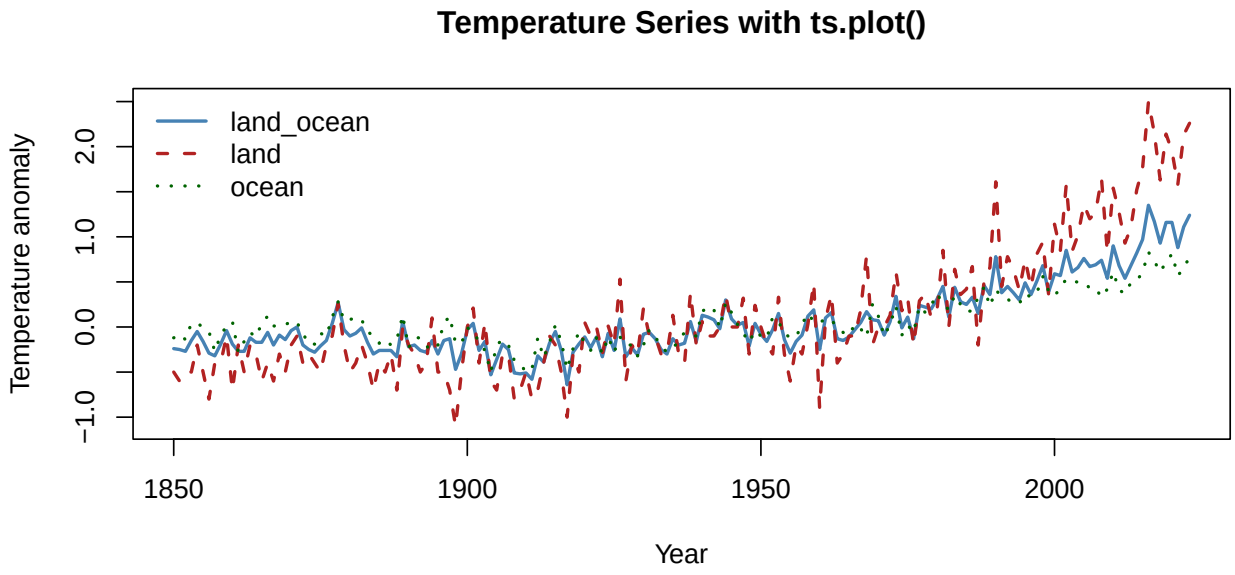


Figure 4.11.: Three temperature anomaly series plotted together with `ts.plot()`.

4.4.2. With `gpars`

```
ts.plot(
  gtemp_land_ts,
```

```
  gtemp_ocean_ts,
  gpars = list(
    col = c("firebrick", "darkgreen"),
    lty = c(1, 2),
    lwd = 2,
    xlab = "Year",
    ylab = "Temperature anomaly"
  ),
  main = "Land and Ocean Series with gpars"
)
legend(
  "topleft",
  legend = c("land", "ocean"),
  col = c("firebrick", "darkgreen"),
  lty = c(1, 2),
  lwd = 2,
  bty = "n"
)
```

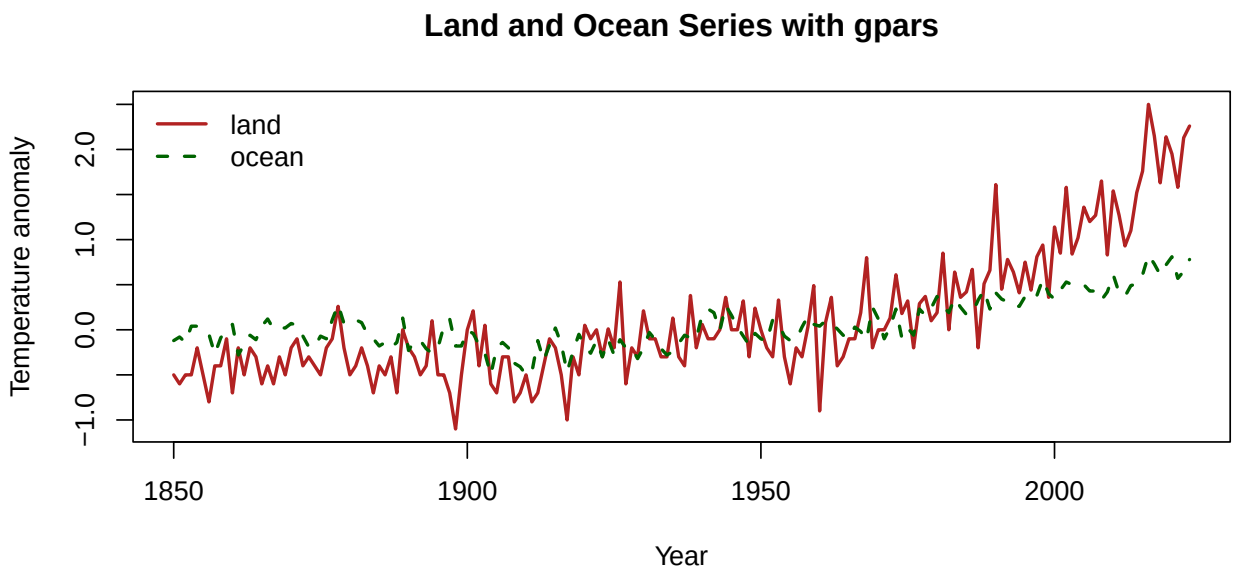


Figure 4.12.: Use of `gpars` inside `ts.plot()` to forward graphical options.

> **Comment**
>
> Compared with `plot.ts(plot.type = "single")`, `ts.plot()` is often the simpler choice when the main intention is to overlay separate series on a common axis.

4.5. `zoo` and `xts` indexed time series

This time, our focus is on the representation of time series where the index is explicit.

4.5.1. `zoo`

```
plot(
  gtemp_zoo,
  col = "firebrick",
  lwd = 2,
  main = "Global Temperature Anomalies as zoo",
  xlab = "Year",
  ylab = "Temperature anomaly"
)
```

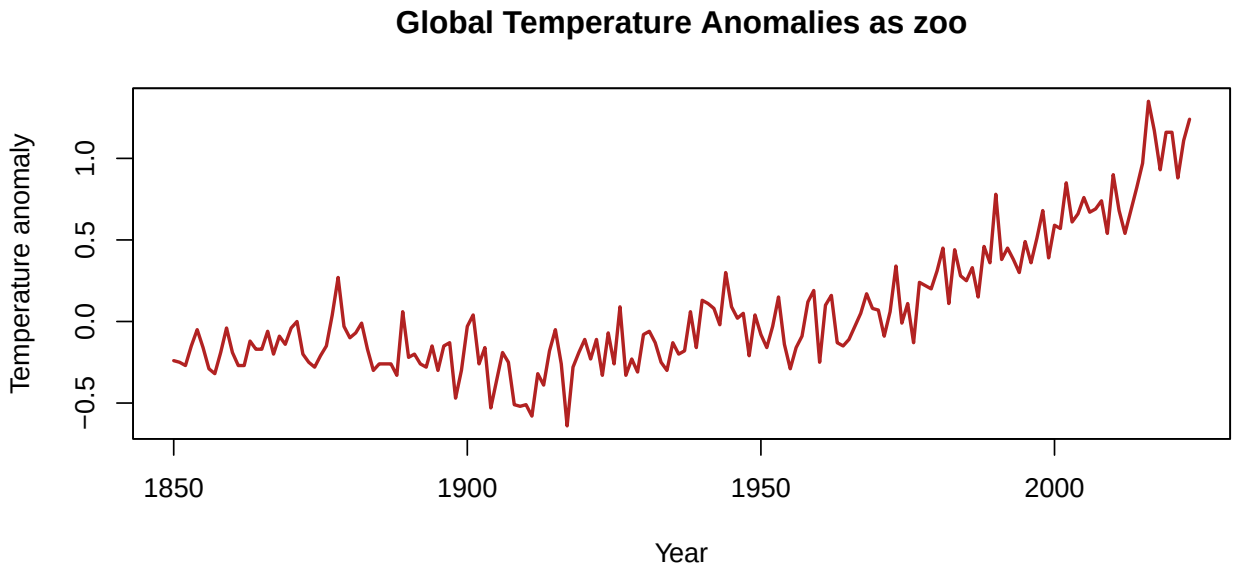


Figure 4.13.: Plot of the temperature anomaly series stored as a `zoo` object.

4.5.2. `plot.xts()`

```
plot.xts(
  gtemp_xts[, "land_ocean"],
  col = "steelblue",
  type = "l",
  lwd = 2,
  main = "Global Temperature Anomalies with plot.xts()",
```

```
  major.ticks = "years",
  ylab = "Temperature anomaly"
)
```

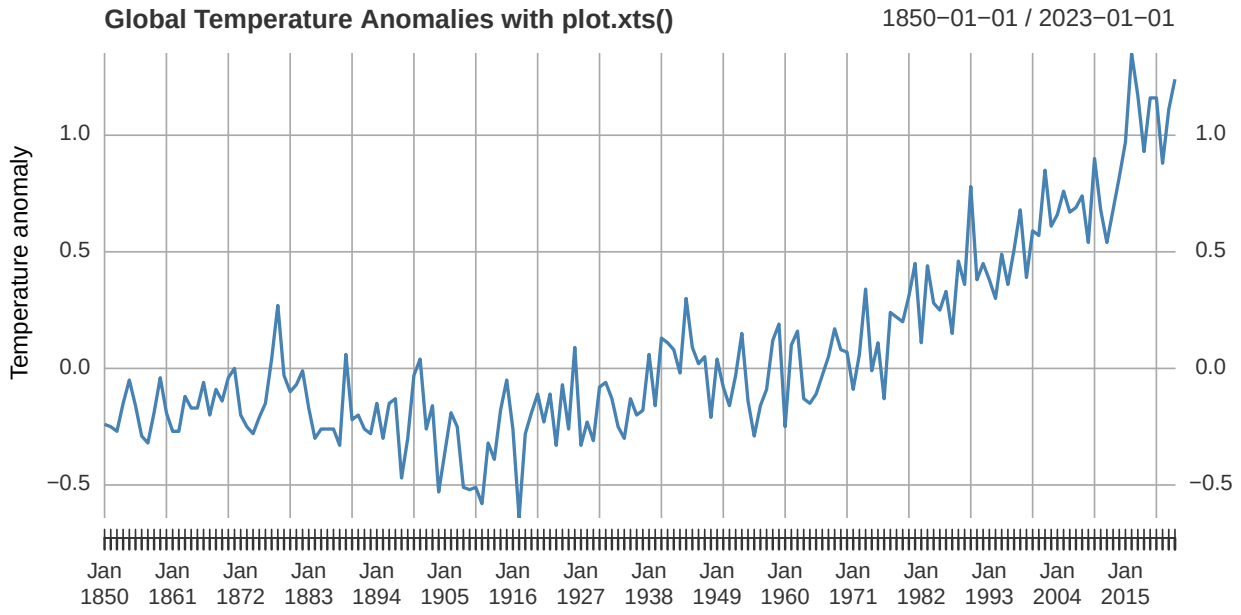


Figure 4.14.: Explicit use of `plot.xts()` for an indexed series.

4.5.3. `xts` (multiple TS)

```
plot.xts(
  gtemp_xts,
  col = c("steelblue", "firebrick", "darkgreen"),
  lwd = 2,
  main = "Temperature Series in a Single xts Panel",
  major.ticks = "years",
  legend.loc = "topleft"
)
```

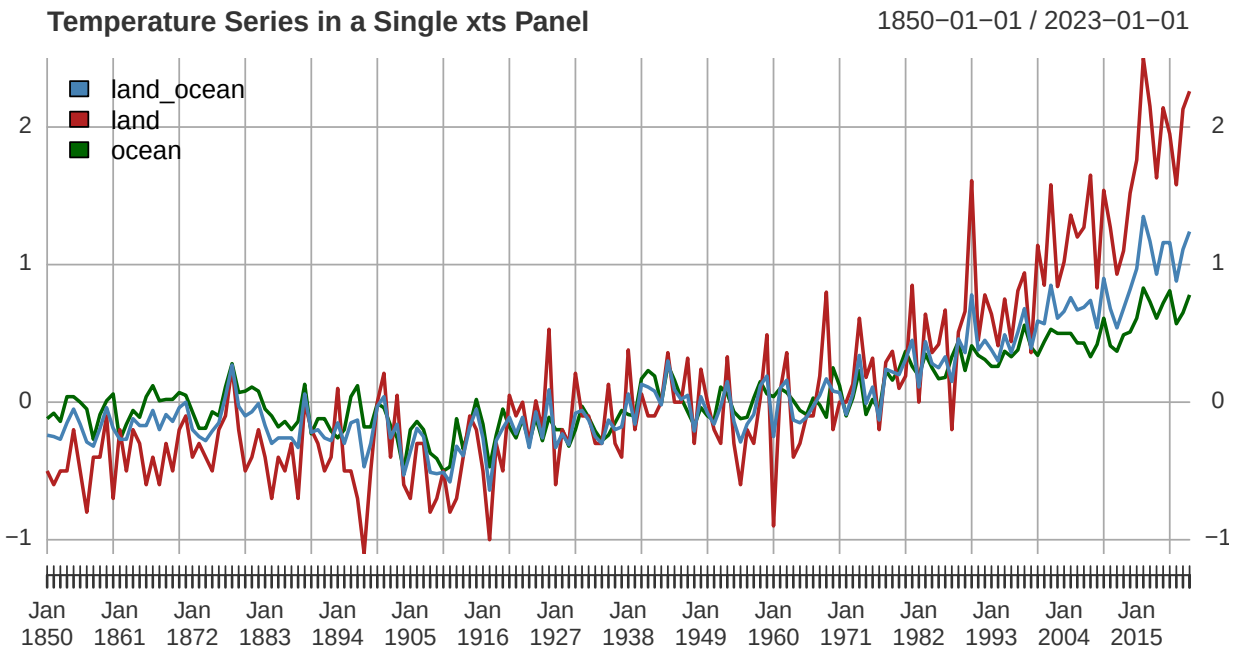


Figure 4.15.: Multiple xts series shown together in a single panel.

4.5.4. `multi.panel = TRUE`

```
plot.xts(
  gtemp_xts,
  multi.panel = TRUE,
  col = c("steelblue", "firebrick", "darkgreen"),
  lwd = 2,
  main = "Temperature Series in Separate xts Panels",
  major.ticks = "years"
)
```

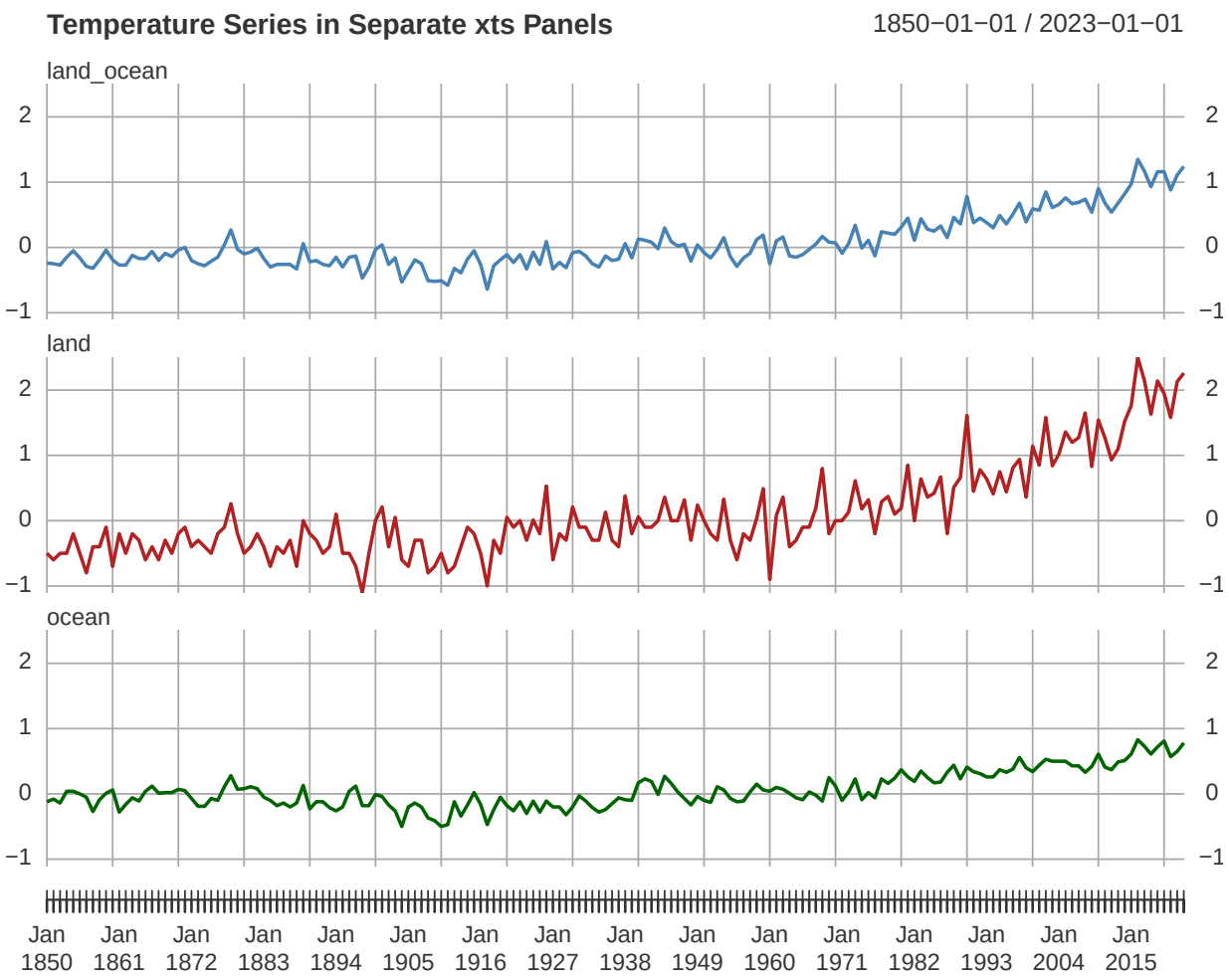


Figure 4.16.: Each xts column displayed in a separate panel using `multi.panel = TRUE`.

4.5.5. `multi.panel=value`

```
#eu_xts <- tsbox::ts_xts(EuStockMarkets)

plot.xts(
  eu_xts[, 1:4],
  multi.panel = 2,
  col = 2:5,
  lwd = 1.5,
  main = "EuStockMarkets Grouped Two Series per Panel",
  major.ticks = "years"
)
```

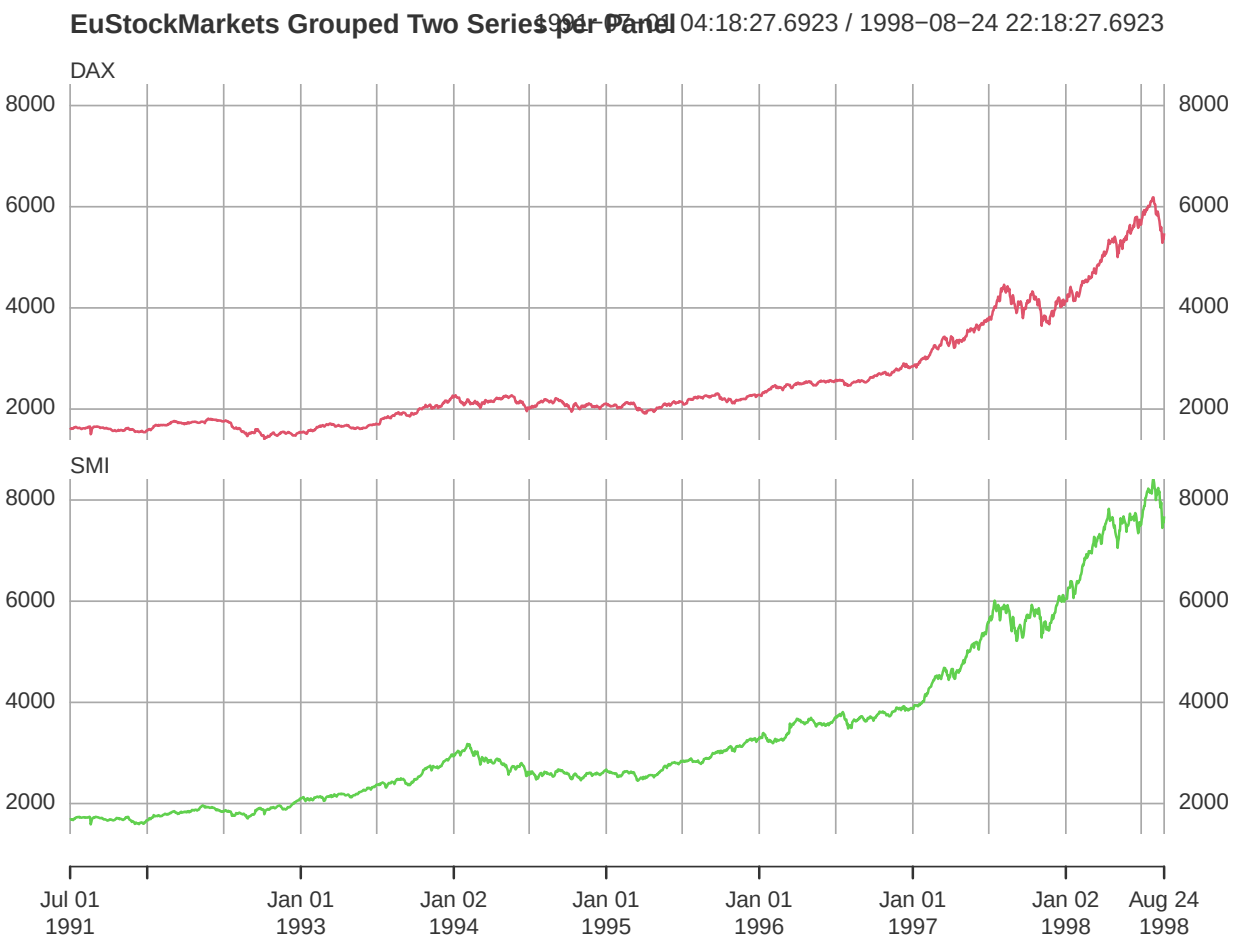


Figure 4.17.: An xts plot where columns are grouped by panel using an integer value for `multi.panel`.

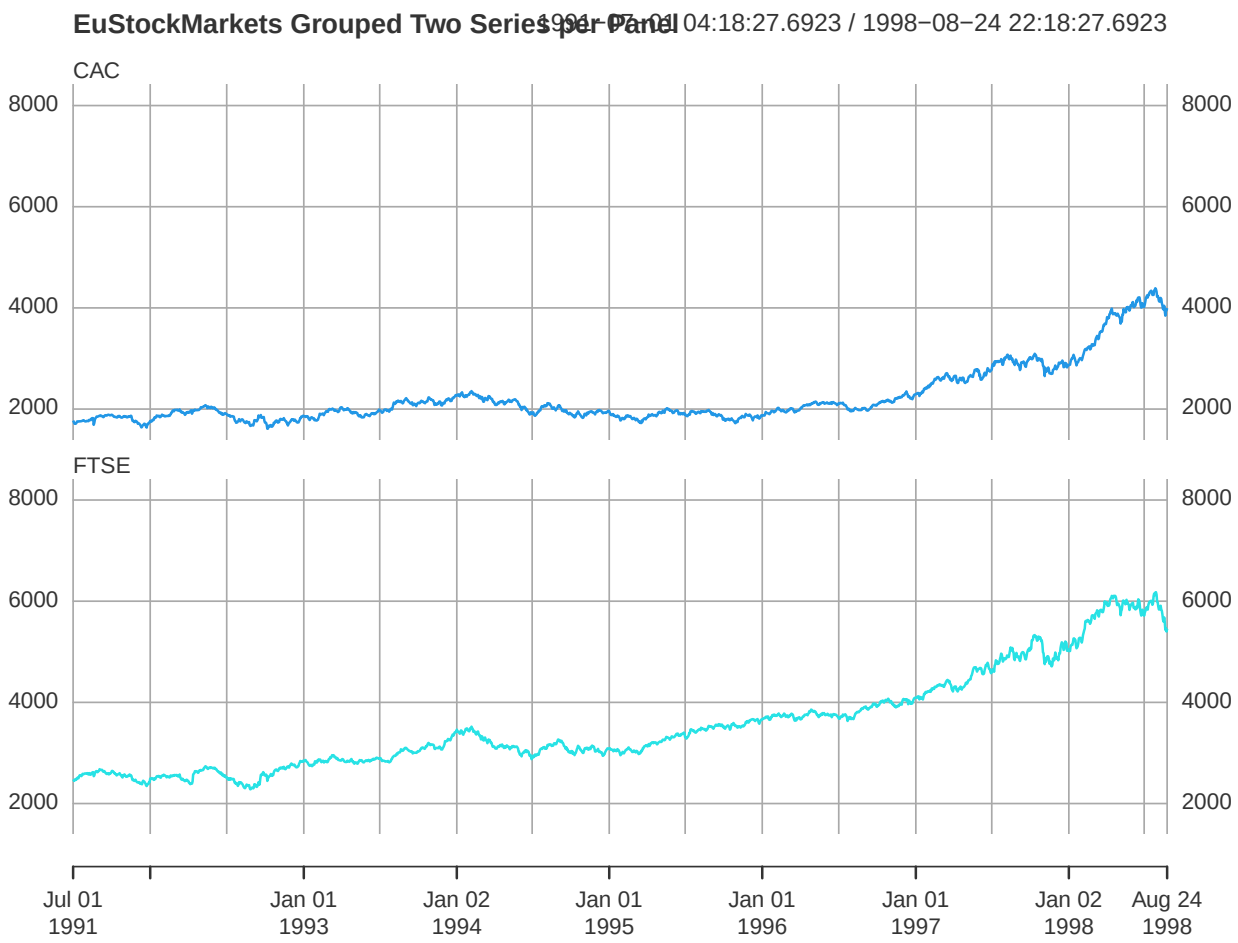


Figure 4.18.: An xts plot where columns are grouped by panel using an integer value for `multi.panel`.

4.5.6. time subset

```
plot.xts(
  eu_xts[, "SMI"],
  subset = "1996/1997",
  col = "darkorange",
  lwd = 2,
  main = "SMI Index, 1996-1997",
  major.ticks = "months",
  minor.ticks = NULL
)
```

Figure 4.19.: An xts plot restricted to a specific time window using the `subset` argument.

4.5.7. Alternative tick marks

```
plot.xts(
  eu_xts[, "DAX"],
  subset = "1996/1997",
  col = "purple",
  lwd = 2,
  main = "DAX with Observation-Based Spacing",
  major.ticks = "months",
  observation.based = TRUE
)
```

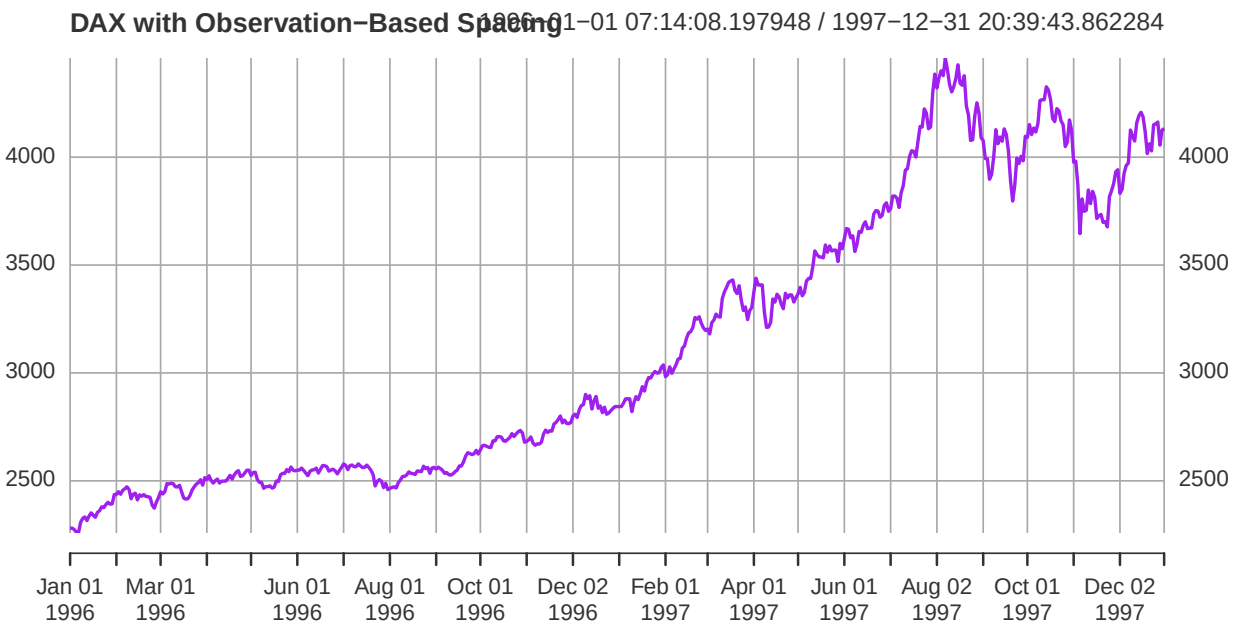


Figure 4.20.: An xts plot with explicit major ticks and observation-based spacing.

4.5.8. Log scale

```
plot.xts(
  eu_xts[, "SMI"],
  col = "brown",
  lwd = 2,
  main = "SMI on a Log Scale",
  major.ticks = "years",
  log = TRUE
)
```

Figure 4.21.: An xts plot with logarithmic scaling on the y-axis.

Notice

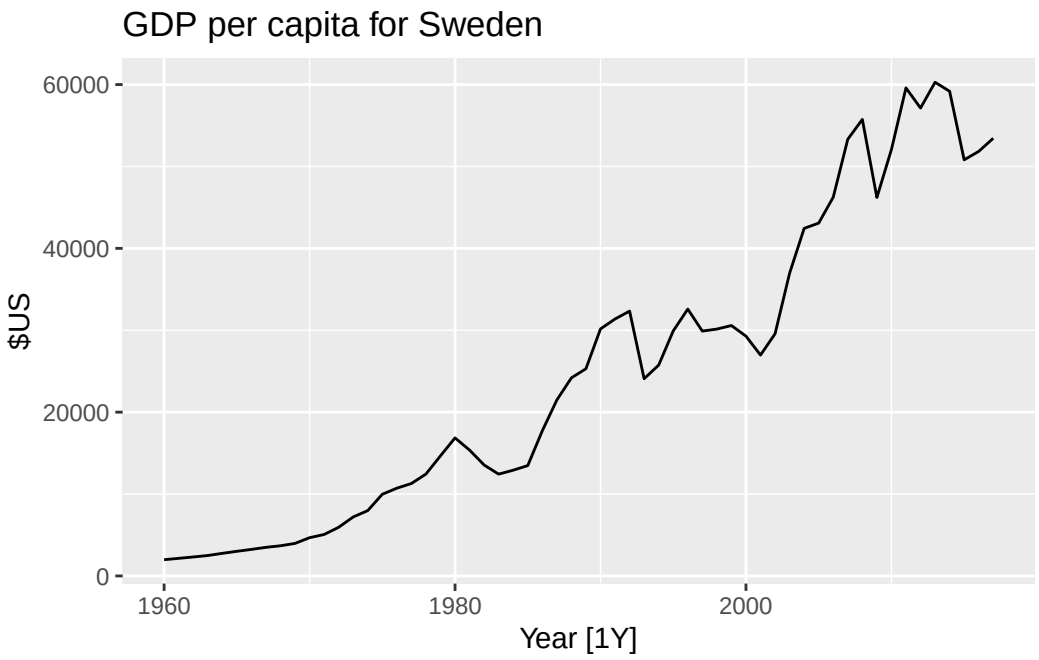
Indexed objects are especially useful when the time index itself is important. `plot.xts()` adds flexible control over temporal subsetting, tick marks, and multi-panel indexed displays.

4.6. Plot Tidy Time Series with `tsibble`

An introduction to the tidy workflows (which means that you may use tools from the `tidyverse` ecosystem, like tidy data (`tibbles`, data frames), pipe operators (`%>%`, for data processing chains), and various tools for data import, manipulation, representation, etc.

A simple example, from https://otexts.com/fpp3/a-tidy-forecasting-workflow.html,

```
# You'll need this library, for examples from the fpp3-supported book
library(fpp3)

gdppc <- global_economy |>
  mutate(GDP_per_capita = GDP / Population)

gdppc |>
  filter(Country == "Sweden") |>
  autoplot(GDP_per_capita) +
  labs(y = "$US", title = "GDP per capita for Sweden")
```



4.6.1. `tsibble` (ggplot2)

```
ggplot(gtemp_tsibble, aes(x = year, y = anomaly)) +
  geom_line(color = "purple", linewidth = 0.8) +
  labs(
    title = "Global Temperature Anomalies as tsibble",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

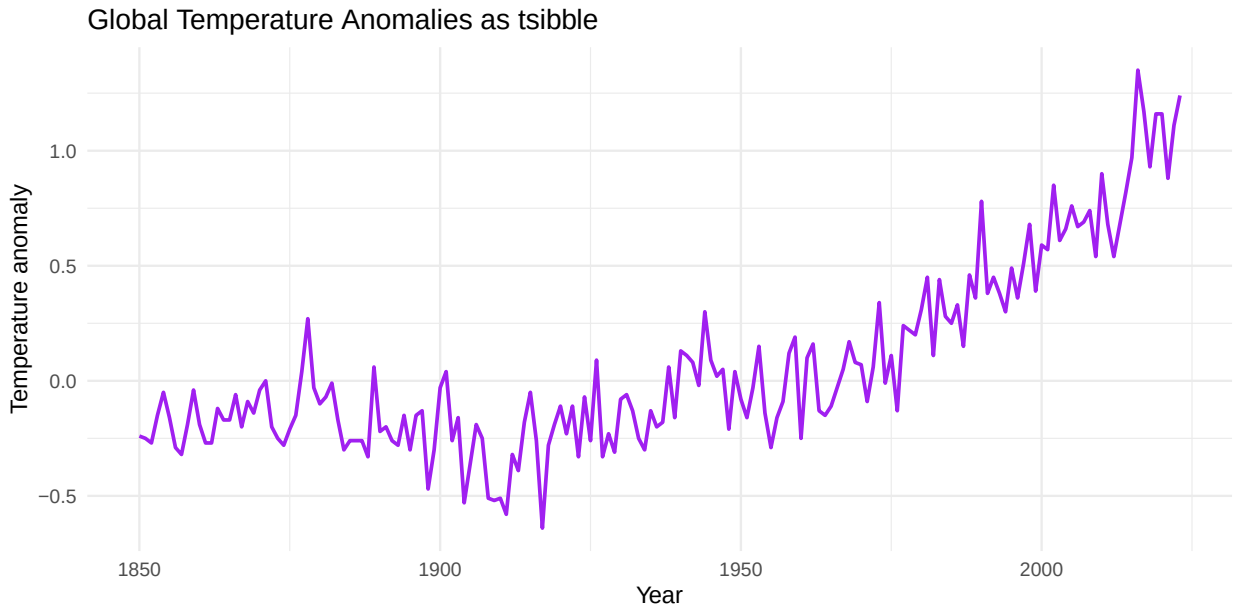


Figure 4.22.: Plot of the temperature anomaly series stored as a `tsibble` and rendered with `ggplot2`.

4.6.2. Faceted `tsibble` (1/2)

Please notice the `facet_wrap` construction. You can play with facets, for example, add `ncol=1` for variation.

```
# Check the availble facets. Should see land/ocean/land_ocean
gtemp_multi_tsibble |>
  dplyr::distinct(series)
```

```
# A tibble: 3 x 1
  series
  <chr>
1 land
2 land_ocean
3 ocean
```

```
gtemp_multi_tsibble |>
  mutate(
    year_band = case_when(
      year >= 1880 & year <= 1899 ~ "1880-1899",
      year >= 1900 & year <= 1919 ~ "1900-1919",
      year >= 1920 & year <= 1939 ~ "1920-1939",
```

```
      year >= 1940 & year <= 1959 ~ "1940-1959",
      TRUE ~ "Other"
    )
  ) |>
  filter(year_band != "Other") |>
  ggplot(aes(x = year, y = anomaly)) +
  geom_line() +
  facet_wrap(~ year_band)



ggplot(gtemp_multi_tsibble, aes(x = year, y = anomaly)) +
  geom_line(color = "steelblue", linewidth = 0.7) +
  facet_wrap(~series, scales = "free_y", ncol=2) +
  labs(
    title = "Facet Plot of Multiple Temperature Series",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

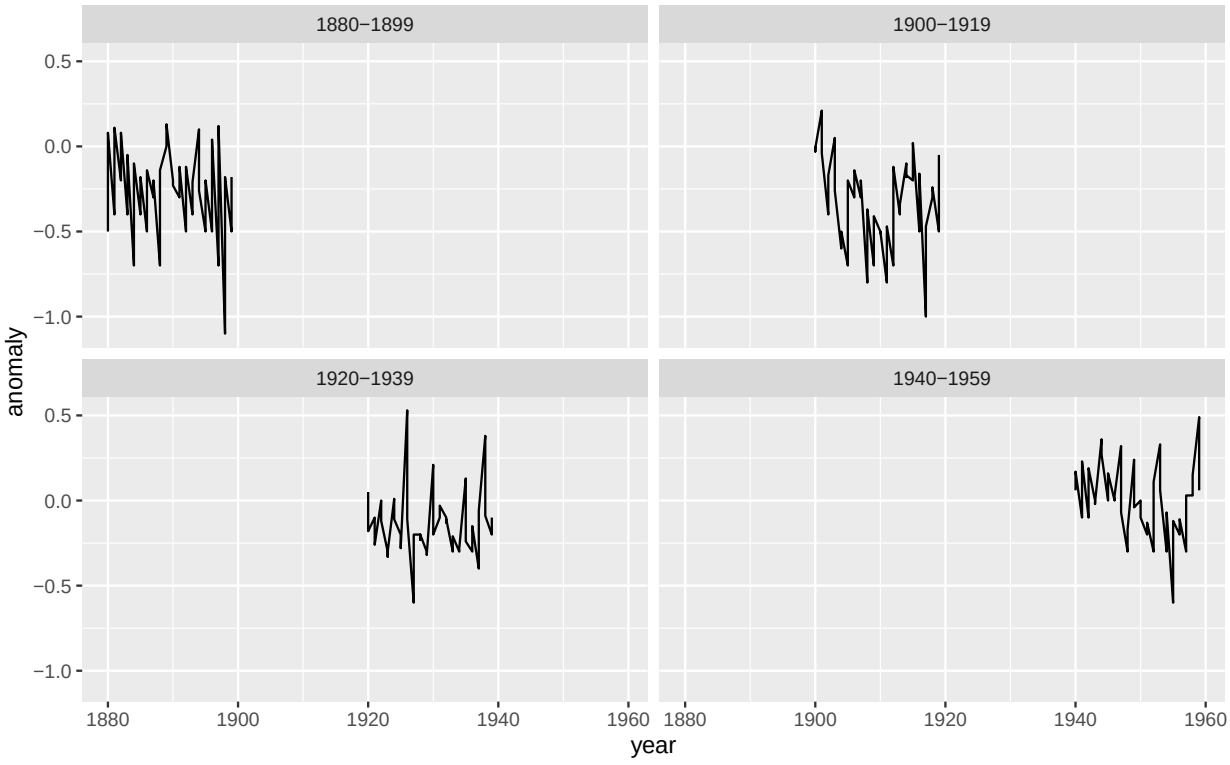


Figure 4.23.: A keyed `tsibble` plotted with one panel per series.

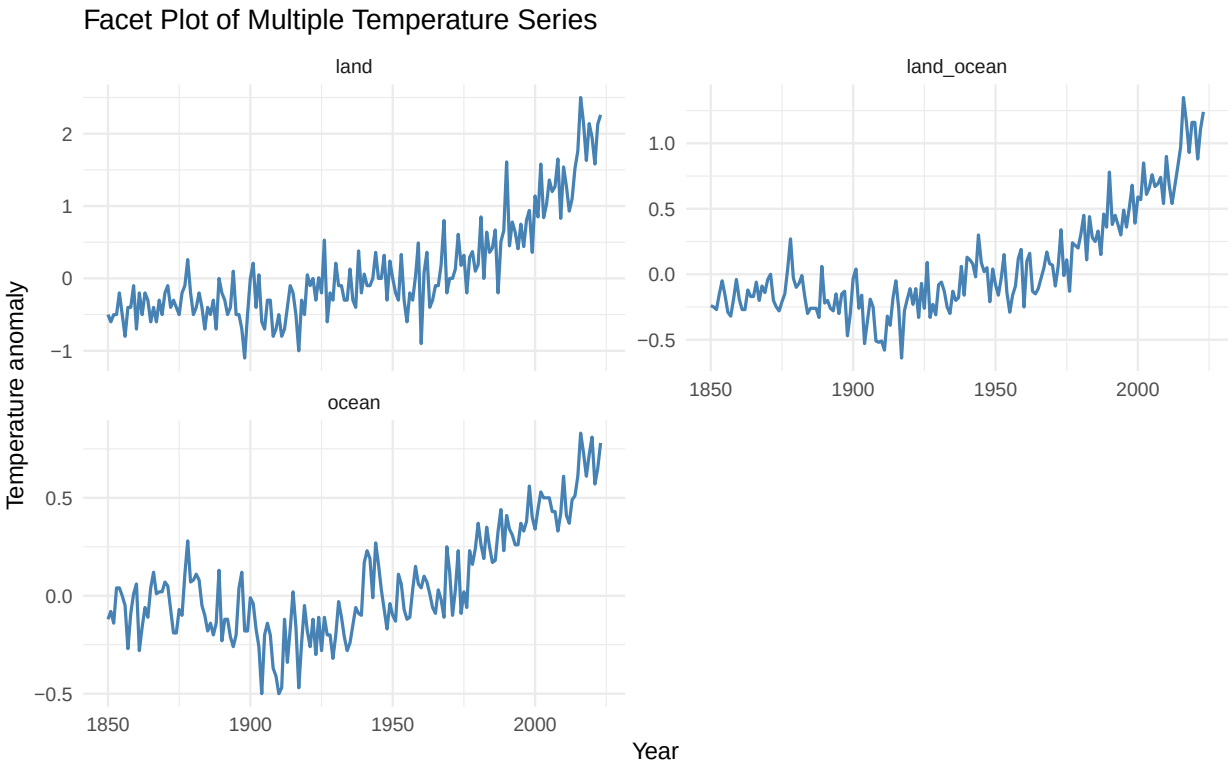


Figure 4.24.: A keyed `tsibble` plotted with one panel per series.

Faceted `tsibble` (2/2)

```
# Check the available names first
names(gtemp_multi_tsibble)
```

```
[1] "year"    "series"  "anomaly"
```

```
# The available values should be year, series, anomaly.

unique(gtemp_multi_tsibble$series)
```

```
[1] "land"       "land_ocean" "ocean"
```

```
# Do some filtering/transformations. For example, define some year bands
gtemp_multi_tsibble |>
  mutate(
    year_band = case_when(
      year >= 1880 & year <= 1899 ~ "1880-1899",
      year >= 1900 & year <= 1919 ~ "1900-1919",
```

```
      year >= 1920 & year <= 1939 ~ "1920-1939",
      year >= 1940 & year <= 1959 ~ "1940-1959",
      TRUE ~ "Other"
    )
  ) |>
  filter(year_band != "Other") |>
  ggplot(aes(x = year, y = anomaly)) +
  geom_line() +
  facet_wrap(~ year_band, ncol=1)
# And plot, at the end of the pipeline
```

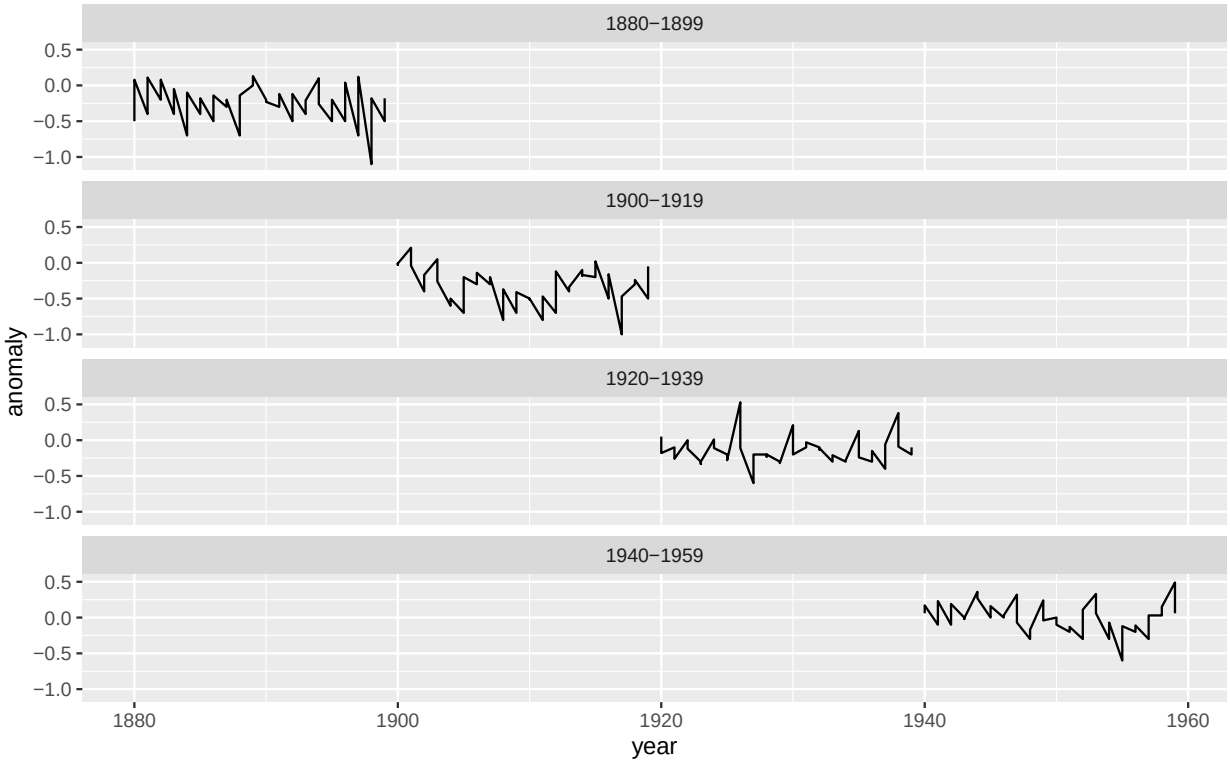


Figure 4.25.: A keyed `tsibble` plotted with one panel per series.

4.6.3. Multiple series together with color mapping

Plotting multiple datasets (series) together, using colormaps. There are different options, use the simplest.

```
ggplot(gtemp_multi_tsibble, aes(x = year, y = anomaly, color = series)) +
  geom_line(linewidth = 0.8) +
  labs(
```

```
    title = "Global Temperature Anomaly Series",
    x = "Year",
    y = "Temperature anomaly",
    color = "Series"
  ) +
  theme_minimal()
```

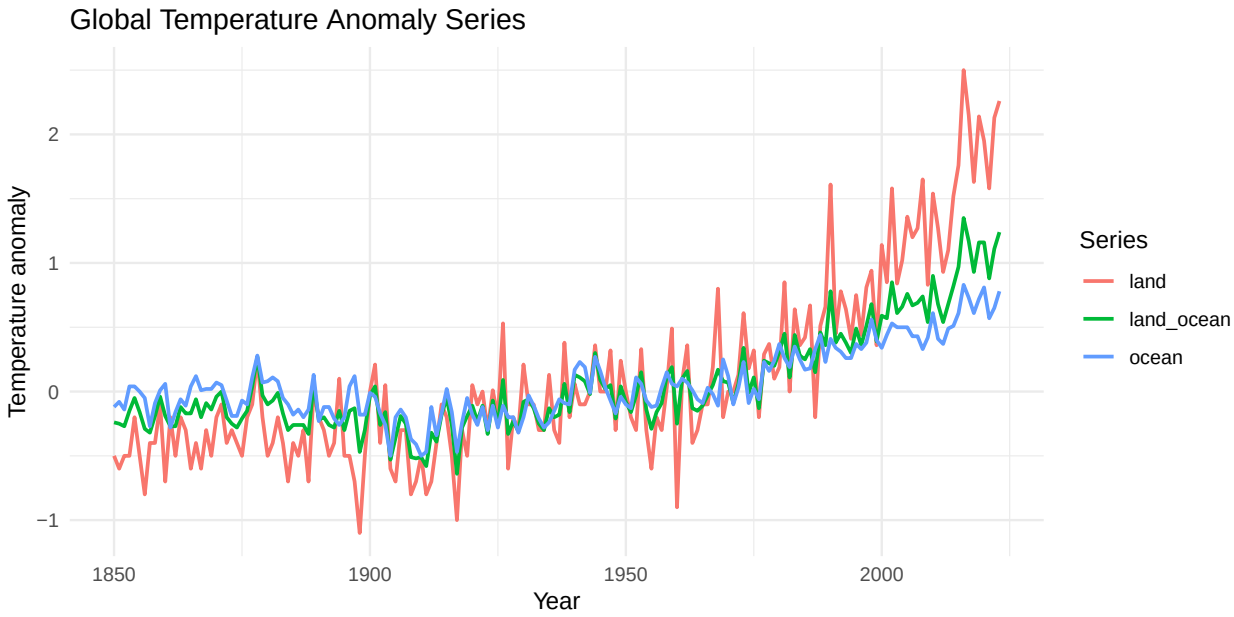


Figure 4.26.: A long-format `tsibble` plotted with explicit color mapping for multiple series.

Discussion

A `tsibble` integrates naturally with tidy plotting syntax and makes it easy to manage multiple series in a long-format structure.

4.7. Find a Trend

Trend (or smoothed pattern) identification, by some simple means. Derive a time series (check the `stats::filter` construct).

4.7.1. Simple moving average

```r
# A smoothed moving averages
gtemp_ma <- stats::filter(gtemp_both_ts, rep(1/10, 10), sides = 2)

plot(
  gtemp_both_ts,
  col = "gray50",
  lwd = 1,
  main = "Global Temperature Anomalies with Moving Average",
  xlab = "Year",
  ylab = "Temperature anomaly"
) # lines will add to exising plot
lines(gtemp_ma, col = "red", lwd = 2)
```

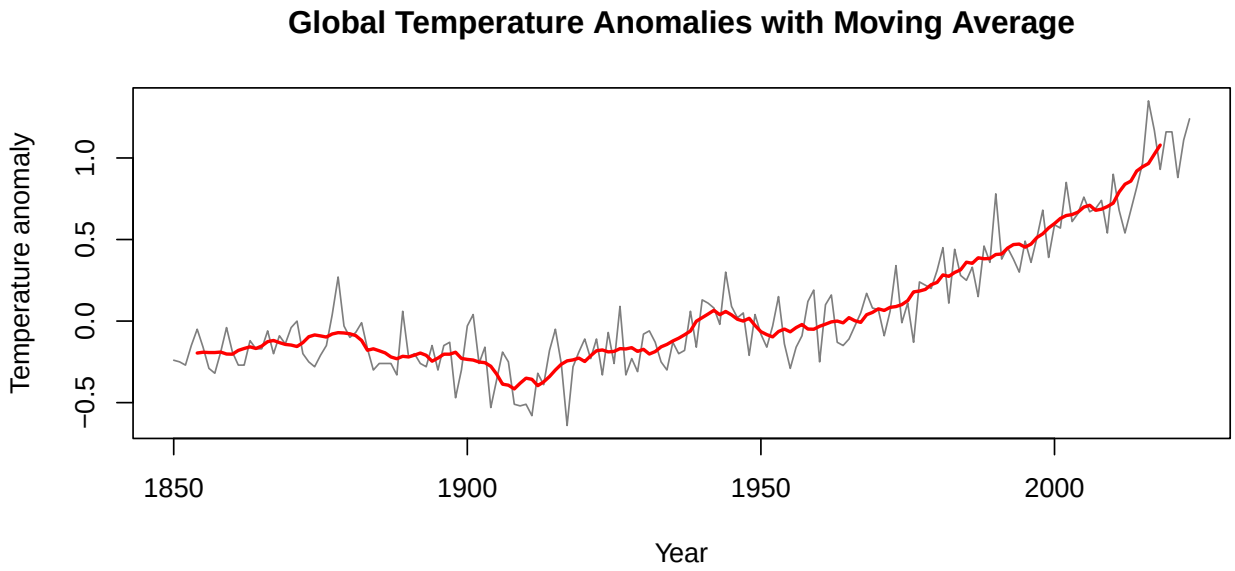


Figure 4.27.: A base plot with a moving average overlay to emphasize long-term change.

4.7.2. Smooth (ggplot2)

```r
ggplot(gtemp_df, aes(x = year, y = anomaly)) +
  geom_line(color = "gray50", linewidth = 0.7) +
  geom_smooth(se = FALSE, color = "red", linewidth = 1) +
  labs(
    title = "Global Temperature Anomalies with Smoother",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

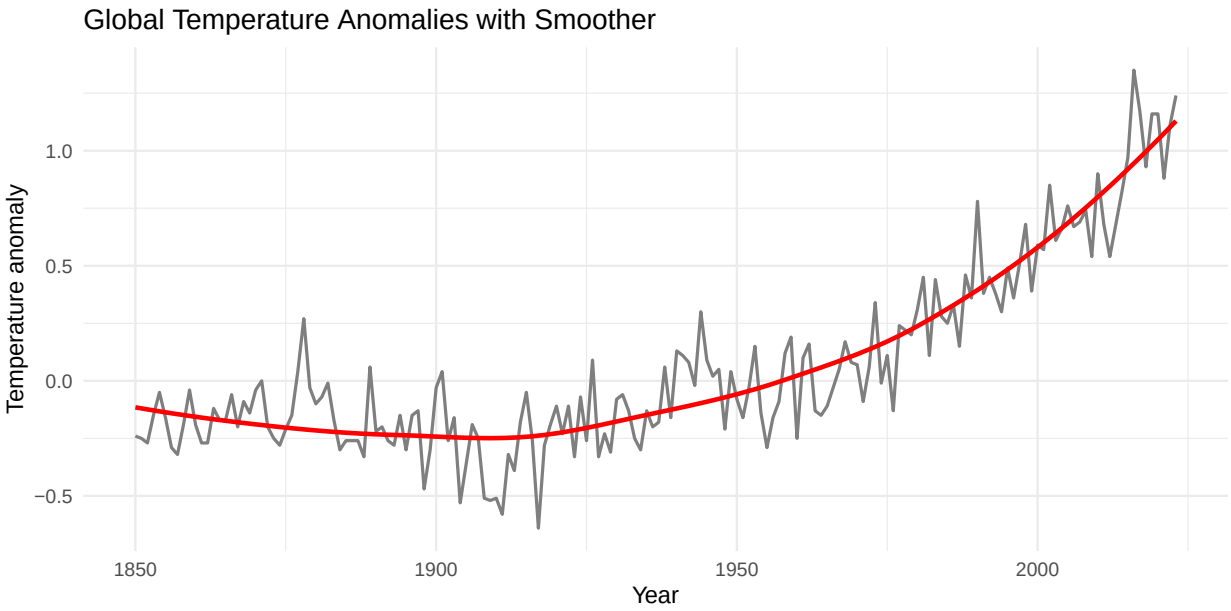


Figure 4.28.: A `ggplot2` line plot with a loess smoother to emphasize broad trends.

4.7.3. Original vs smoothed

```
smooth_df <- tibble(
  year = as.numeric(time(gtemp_both_ts)),
  original = as.numeric(gtemp_both_ts),
  smoothed = as.numeric(gtemp_ma)
) |>
  pivot_longer(
    cols = c(original, smoothed),
    names_to = "series",
    values_to = "anomaly"
  )

ggplot(smooth_df, aes(x = year, y = anomaly)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  facet_wrap(~ series, scales = "free_y") +
  labs(
    title = "Original and Smoothed Temperature Series",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

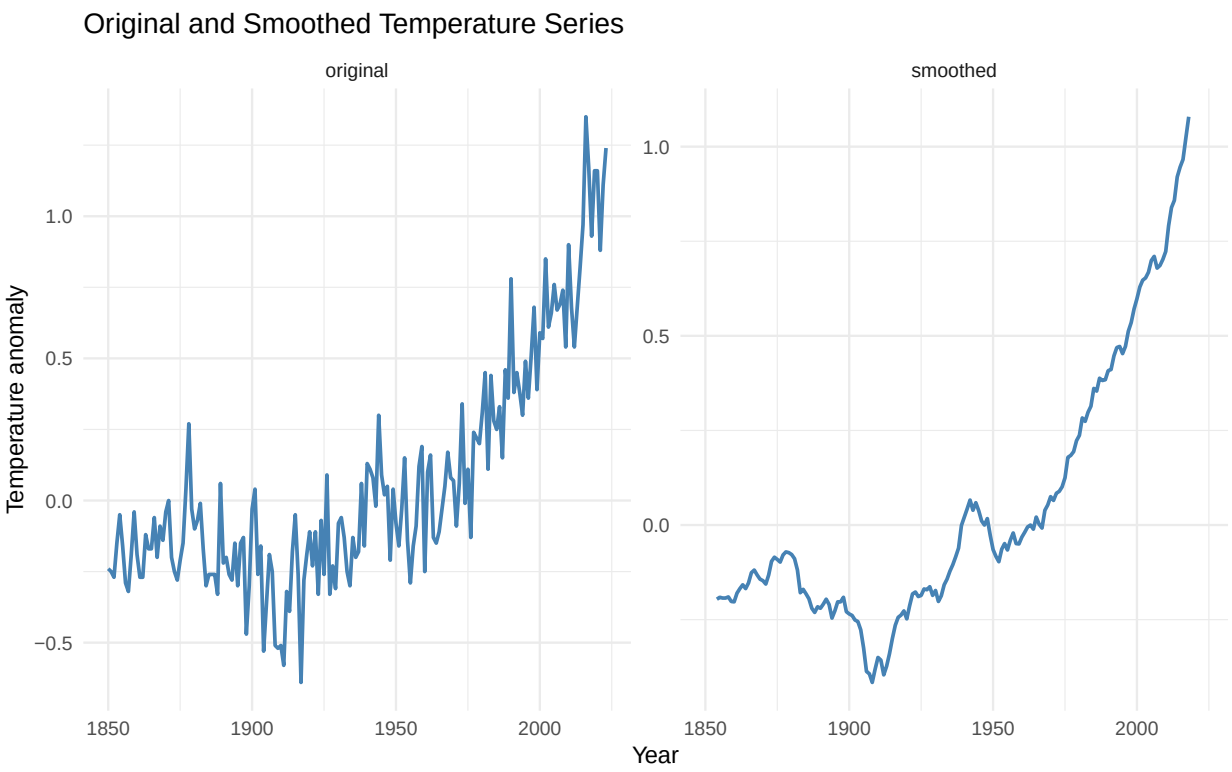


Figure 4.29.: A simple comparison between the original and smoothed versions of the same series.

> **Discussion**
>
> Smoothing helps readers distinguish broad patterns from short-term variation, but it should always be presented as an analytical aid rather than as a replacement for the observed data. There are various functions available in the various time-series libraries.

4.8. Visualize Distribution and Variation Over Time

A complementary approach, offering alternative views of temporal variation.

4.8.1. Bars

Use a bar chart to emphasize positive and negative values. You have two representations here: the easy one (left), and the custom one (right).

```
par(mfrow = c(1, 2)) # two column plots

ggplot(gtemp_df, aes(x = year, y = anomaly)) +
```

```
  geom_col(fill = "steelblue") +
  labs(
    title = "Yearly Temperature Anomalies",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()

ggplot(
  gtemp_df,
  aes(x = year, y = anomaly, fill = ifelse(anomaly < 0, "Negative", "Positive"))
) +
  geom_col() +
  scale_fill_manual(values = c("Negative" = "red", "Positive" = "steelblue")) +
  labs(
    title = "Yearly Temperature Anomalies",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal() +
  guides(fill = "none") # to remove fake legend titles

par(mfrow = c(1, 1)) # back to one
```

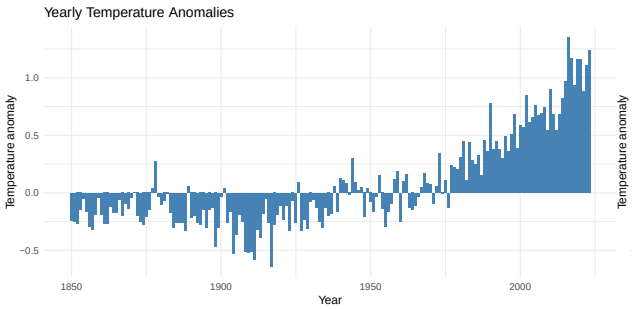


Figure 4.30.: A bar chart emphasizing positive and negative yearly temperature anomalies.

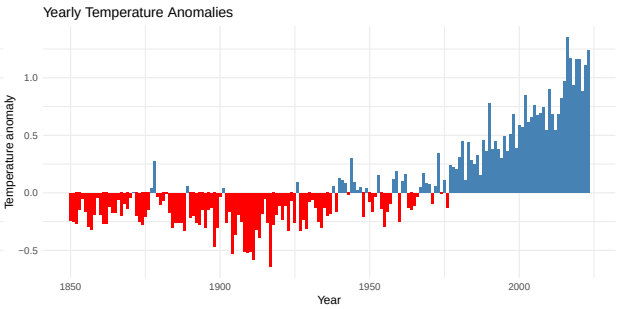


Figure 4.31.: A bar chart emphasizing positive and negative yearly temperature anomalies.

4.8.2. Histogram

Sometimes, a histogram can offer some hints on the available (observed) data. Play with the `bins` value for the number of represented bars.

```
ggplot(gtemp_df, aes(x = anomaly)) +
  geom_histogram(fill = "darkorange", color = "white", bins = 20) +
  labs(
    title = "Distribution of Temperature Anomalies",
    x = "Temperature anomaly",
    y = "Count"
  ) +
  theme_minimal()
```

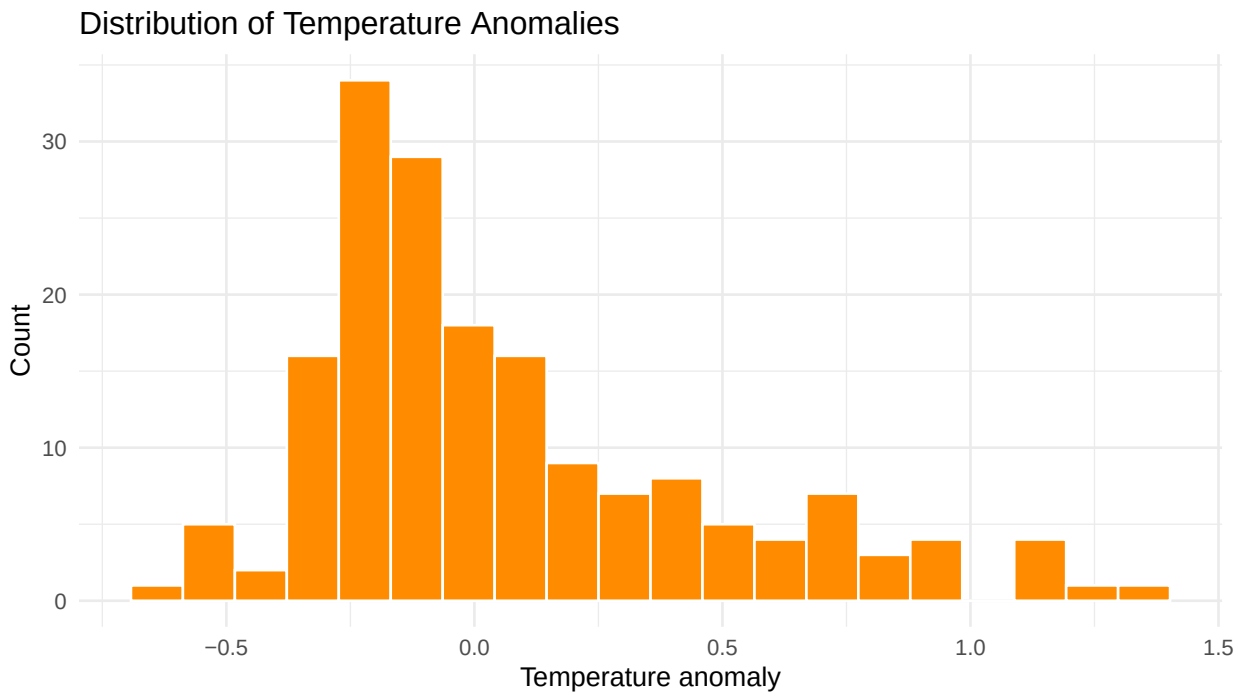


Figure 4.32.: A histogram showing the distribution of temperature anomaly values.

4.8.3. Boxplots

Or you can use boxplots, for a distribution-centric view.

```
period_df <- gtemp_df |>
  mutate(period = cut(year, breaks = c(1850, 1900, 1950, 2000, 2025)))

ggplot(period_df, aes(x = period, y = anomaly, fill = period)) +
  geom_boxplot(show.legend = FALSE) +
  labs(
    title = "Temperature Anomalies by Period",
    x = "Period",
```

```
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

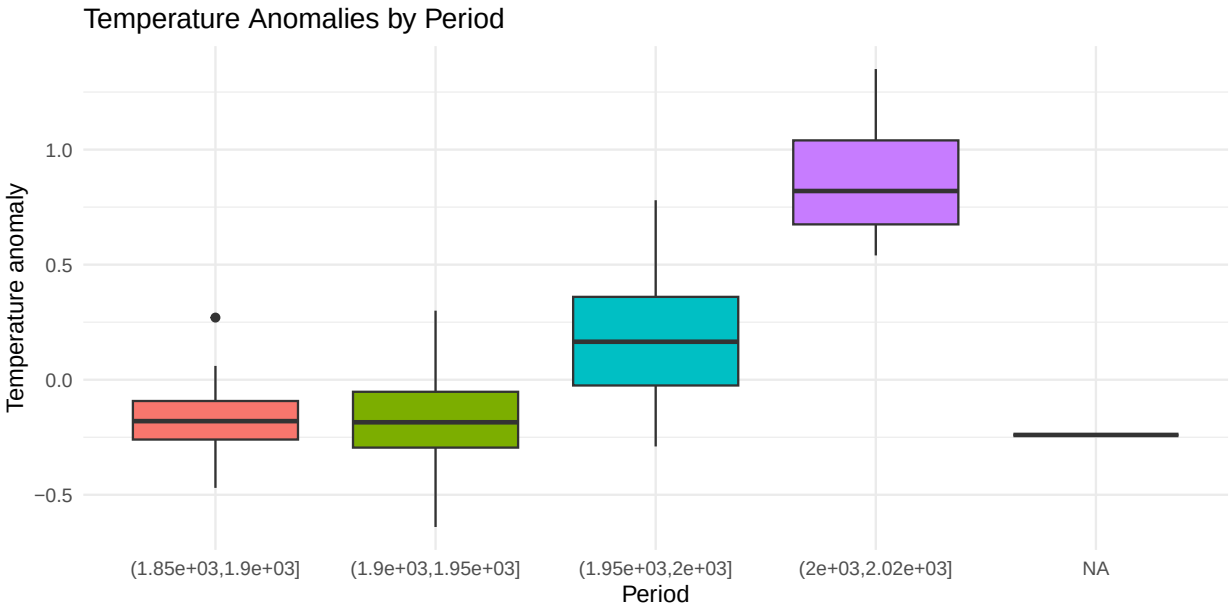


Figure 4.33.: A grouped view of temperature anomaly distributions across broad historical periods.

> **Remarks**
>
> Alternative plot types help reveal features that may be less obvious in line charts, especially changes in sign, spread, or distribution.

4.9. Compare Multiple Plots

Sometimes it is handy to place several plots together, for comparisons.

4.9.1. Basics

Check the `par(mfrow = c(...))` construction, for an easy setup.

```
par(mfrow = c(1, 2))
plot(gtemp_both_ts, col = "steelblue", main = "Land + Ocean")
plot(gtemp_land_ts, col = "firebrick", main = "Land only")
par(mfrow = c(1, 1))
```

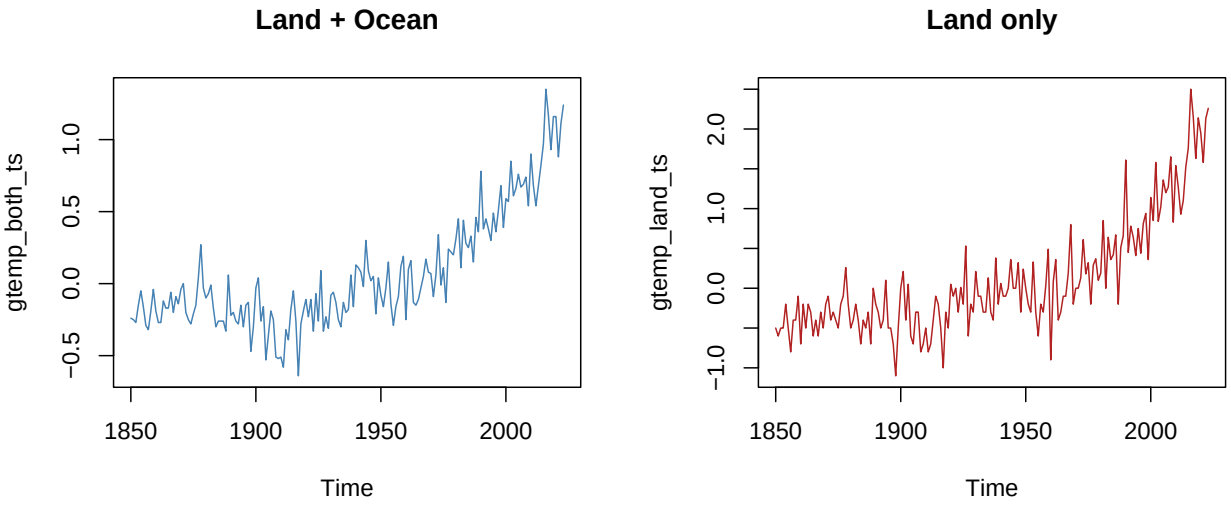


Figure 4.34.: A base R multi-panel layout using `par(mfrow = c(1, 2))`.

4.9.2. `facet_wrap()`

Use a faceted view, when you have the necessary details on your data (may require additional investigations).

```
ggplot(gtemp_multi_tsibble, aes(x = year, y = anomaly)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  facet_wrap(~ series, scales = "free_y", ncol = 2) +
  labs(
    title = "Comparison of Temperature Series",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()

# if you're unsure about the scale_color_manual values, use
unique(gtemp_multi_tsibble$series)
```

```
[1] "land"       "land_ocean" "ocean"
```

```
# at 1st level, head(unique(gtemp_multi_tsibble),0) will return avail. variables

ggplot(gtemp_multi_tsibble, aes(x = year, y = anomaly, color = series)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(~ series, scales = "free_y", ncol = 2) +
  scale_color_manual(values = c(
```

```
    "land" = "darkred",
    "land_ocean" = "orange2",
    "ocean" = "steelblue"
  )) +
  labs(
    title = "Comparison of Temperature Series",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal() +
  guides(color = "none")
```

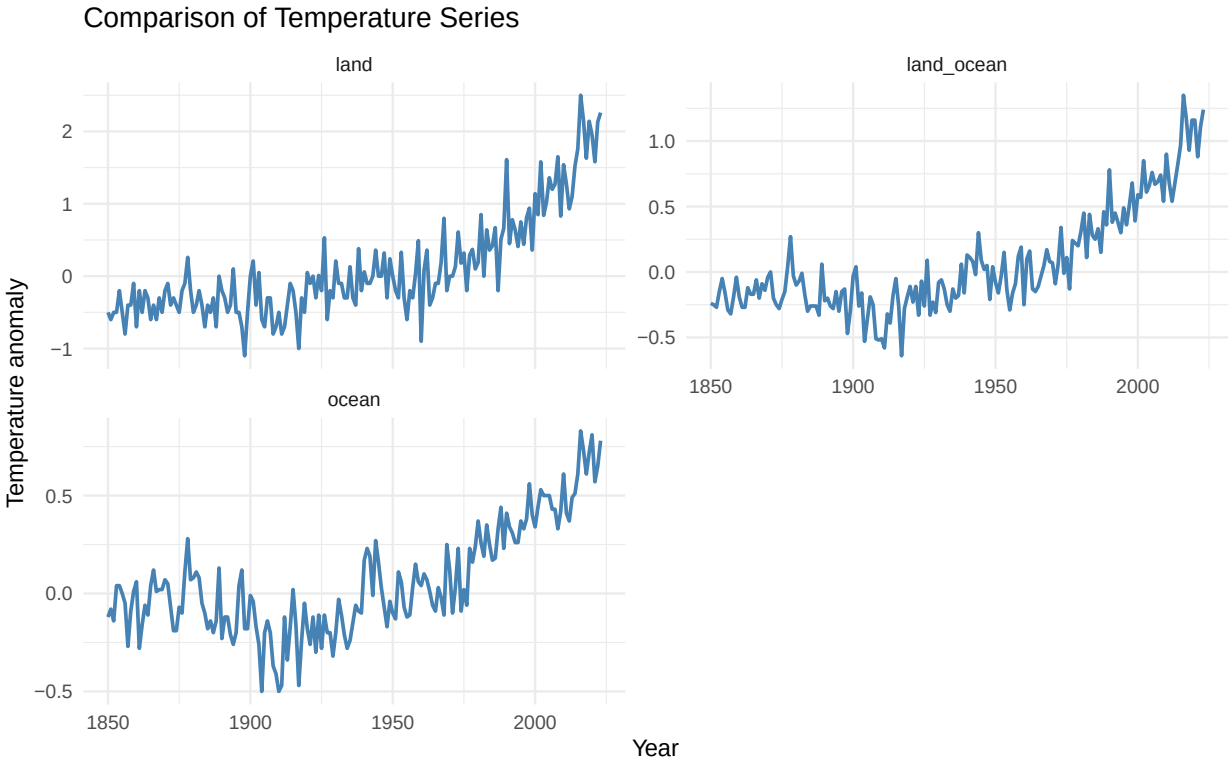


Figure 4.35.: A faceted layout using `ggplot2`, which might be more robust than base multi-panel state.

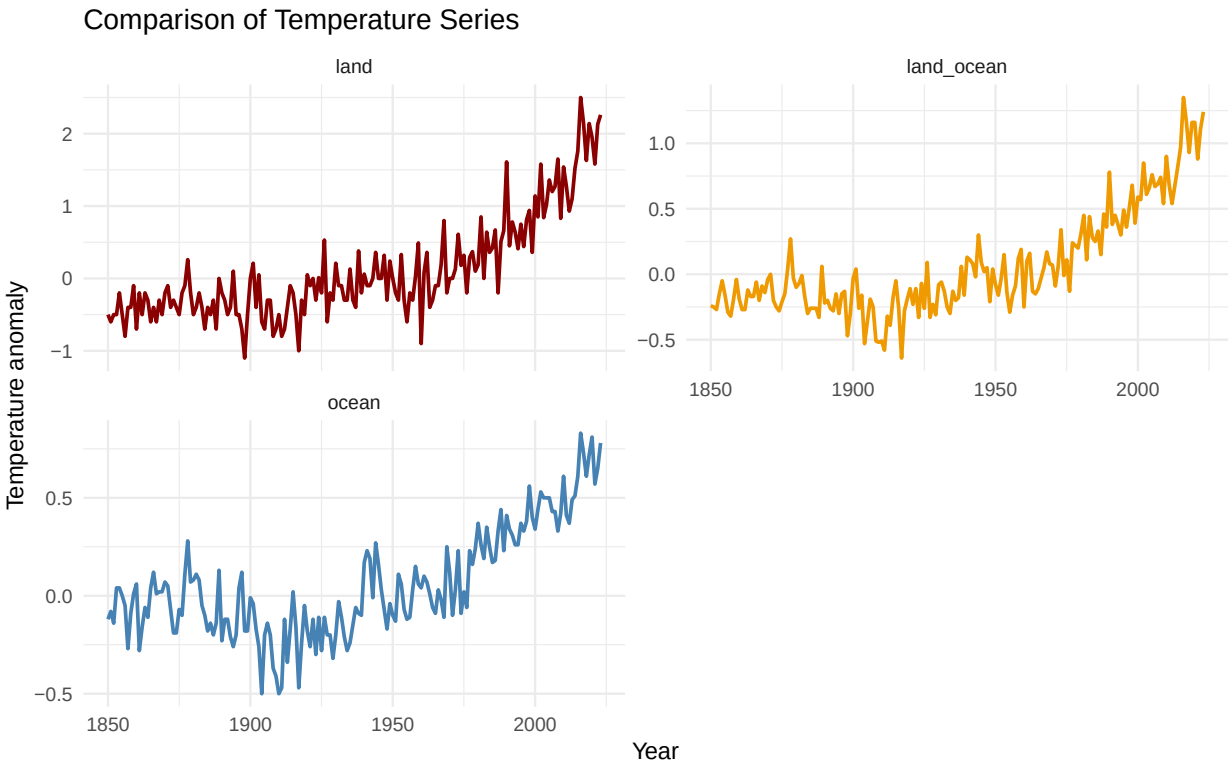


Figure 4.36.: A faceted layout using `ggplot2`, which might be more robust than base multi-panel state.

4.9.3. Option 8C: Vertical comparison by reshaping

```
subset_compare <- gtemp_multi_tsibble |>
  filter(series %in% c("land_ocean", "ocean"))

ggplot(subset_compare, aes(x = year, y = anomaly)) +
  geom_line(color = "darkgreen", linewidth = 0.8) +
  facet_grid(series ~ ., scales = "free_y") +
  labs(
    title = "Vertical Comparison of Two Series",
    x = "Year",
    y = "Temperature anomaly"
  ) +
  theme_minimal()
```

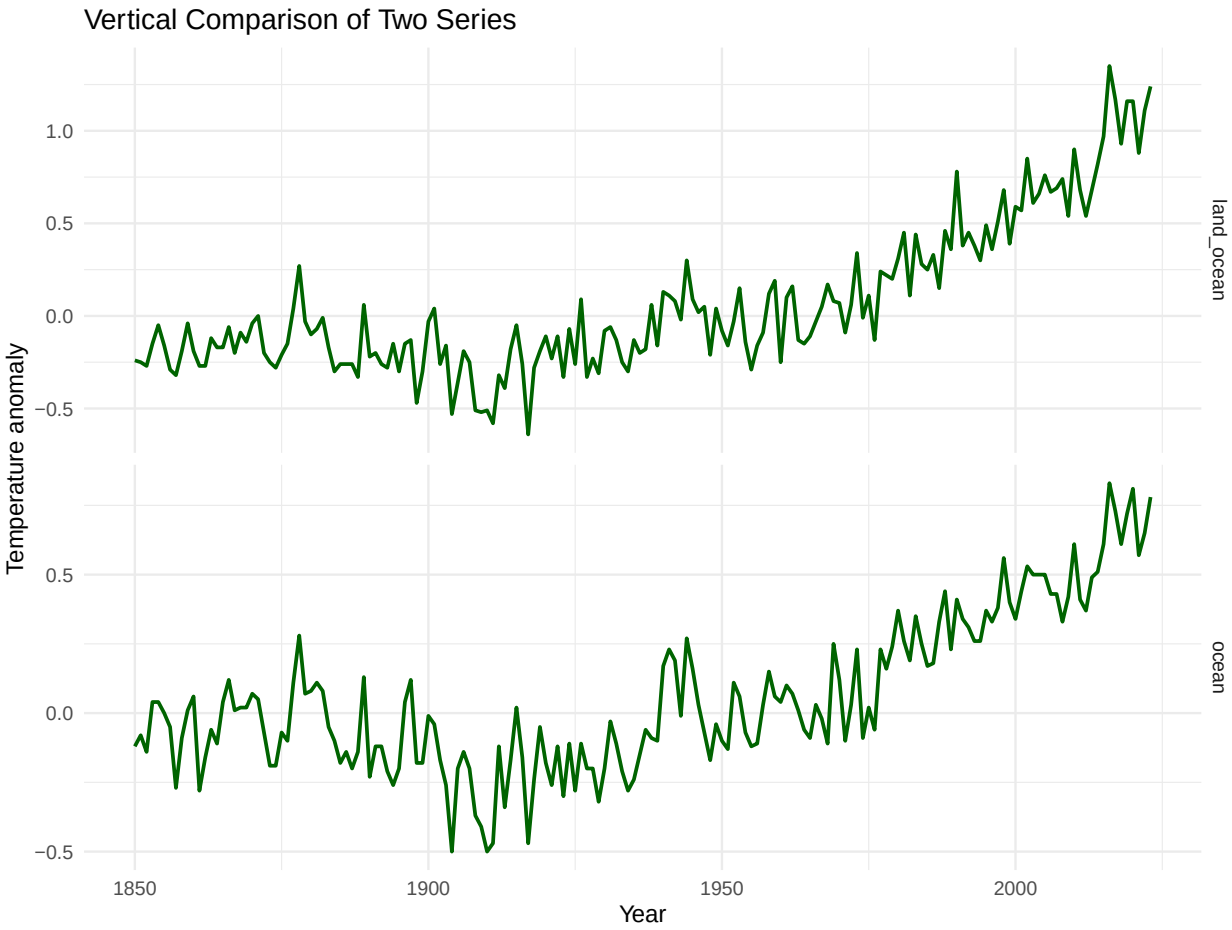


Figure 4.37.: A vertically stacked comparison strategy using a long-format table and faceting.

Notices

Plot arrangement is not only a technical issue: it affects comparison, readability, and the interpretability of scale differences.

4.10. Simple Plotting Guidelines

A simple list of plotting recommendations:

1. Use clear labels and captions
    - Always state what is being measured.
    - Keep axis labels specific.
    - Use captions to explain the content and the representation.

2. Choose scales deliberately

    - Decide whether multiple panels should share scales.
    - Use free scales with caution.
    - Be careful when visually comparing series of different magnitudes.

3. Prefer explicit data structures when needed

    - Use data frames or `tsibbles` when custom `ggplot2` control is needed.
    - Use base plotting when quick inspection is enough.
    - Use indexed objects when dates and irregularity are central.

4. Keep visual rendering in mind

    - Multi-panel base plots require state resets such as `par(mfrow = c(1, 1))`.
    - `ggplot2` facets are often more stable in rendered documents.
    - Explicit package loading helps avoid method-dispatch surprises.

4.10.1. Observations

Plotting is not just about choosing a function. It also involves choosing the representational level, the visual emphasis, and the narrative role of the figure within the chapter.

4.11. Summary

We had a simple investigation on how to plot time series across several major R frameworks: base `ts`, `ts.plot()`, indexed `zoo`/`xts`, and tidy pipelines based on `ggplot2`, `ggfortify`, and `tsibble`.

We covered various options, from first plots to explicit `plot.ts()` and `plot.xts()` variations, multi-series comparisons, smoothing overlays, alternative chart types, and multi-panel layouts.

Plotting heavily depends on representation: for the same time series data you can inspect them one form, customize in another one, and arrange into comparative displays in yet another. Choosing the plotting approach depends on data representation and analytical workflow of your choice.

! When to Use `ts.plot()`, `plot()`, `ggplot()`, and `autoplot()`

- Use **`plot()`** when you want the default plotting method associated with an object class. For example, `plot(ts_object)` dispatches to `plot.ts()`, while `plot(xts_object)` dispatches to `plot.xts()`.
- Use **`plot.ts()`** when you want explicit control over classical time-series plotting for `ts` objects, especially for multivariate displays with `plot.type = "single"` or `plot.type = "multiple"`, panel layouts via `nc`, and x–y / lag-style plots.
- Use **`ts.plot()`** when the main goal is to place several separate time series on a **common**

time axis with minimal syntax, especially when they share the same frequency.

- Use `plot.xts()` when working with indexed time series where explicit control over time subsetting, major tick marks, observation-based spacing, or multi-panel indexed views is important.
- Use `ggplot()` when you want full customization and are comfortable working from an explicit tabular representation such as a data frame, tibble, or `tsibble`.
- Use `autoplot()` when a package supplies a useful automatic plotting method for a class (for example, a `ts` object via `ggfortify`-style support). It is convenient, but usually less flexible than building the plot explicitly with `ggplot()`.
- In practice, **base plotting functions** (`plot.ts`, `ts.plot`, `plot.xts`) are often the quickest for exploratory work, while **ggplot2-based workflows** are usually the best choice when plots need to be highly customized, combined, or publication-ready.

Part II.

Exercises

5. Exercises

5.1. Discover datasets

There are many datasets available in R, adapted for time series use. Some are `ts`-native, others are data frames/`tsibble`. Take a look at the different options you have.

5.2. Base R

```r
# List datasets from the built-in datasets package
all_datasets <- data(package = "datasets")
all_datasets$results[, c("Package", "Item", "Title")]
```

```r
# Optional: search dataset names for possible time-related content
idx <- grepl(
  "air|temp|death|gas|stock|sun|rain|sales|passenger|co2|nile|market",
  all_datasets$results[, "Item"],
  ignore.case = TRUE
)
all_datasets$results[idx, c("Package", "Item", "Title")]
```

5.3. `timeSeriesDataSets`

```r
# Uncomment if needed:
# install.packages("timeSeriesDataSets")

if (requireNamespace("timeSeriesDataSets", quietly = TRUE)) {
  data(package = "timeSeriesDataSets")$results[, c("Package", "Item", "Title")]
} else {
  message("Package 'timeSeriesDataSets' is not installed. Install it with install.packages('ti
}
```

5.4. tsibbledata

```
# Uncomment if needed:
# install.packages("tsibbledata")

if (requireNamespace("tsibbledata", quietly = TRUE)) {
  data(package = "tsibbledata")$results[, c("Package", "Item", "Title")]
} else {
  message("Package 'tsibbledata' is not installed. Install it with install.packages('tsibbleda
}
```

! Tasks

1. Identify at least **three datasets** that appear to contain time series data.
    1. For each one, note whether it seems to be yearly, quarterly, monthly, or something else.
    2. State whether it looks univariate or multivariate.
2. Choose **two datasets** from `timeSeriesDataSets`.
3. Choose **two datasets** from `tsibbledata`.
    1. For each selected dataset, note the object class and what it appears to measure.
    2. Decide whether each dataset is univariate or multivariate.

5.5. Recognize time series objects

Inspect the following built-in datasets (or others, of your choice):

- `AirPassengers`
- `JohnsonJohnson`
- `nottem`
- `ldeaths`
- `EuStockMarkets`

```
series_list <- list(
  AirPassengers = AirPassengers,
  JohnsonJohnson = JohnsonJohnson,
  nottem = nottem,
  ldeaths = ldeaths,
```

```
  EuStockMarkets = EuStockMarkets
)

lapply(series_list, class)
lapply(series_list, start)
lapply(series_list, end)
lapply(series_list, frequency)
```

! Tasks

1. Determine which datasets are stored as `ts` objects.
2. Decide which are univariate and which are multivariate.
3. State the time frequency of each dataset.

5.6. Play with a time series dataset (basic)

Choose one dataset from the following list, or another built-in time series dataset you discover:

- `AirPassengers`
- `JohnsonJohnson`
- `nottem`
- `ldeaths`
- `Nile`
- `co2`
- `sunspot.year`
- `UKgas`

```
# Replace AirPassengers with your chosen dataset
x <- AirPassengers

class(x)
start(x)
end(x)
frequency(x)
summary(x)
plot(x, col = "steelblue", main = deparse(substitute(x)))
```

! Tasks

Write **2–4 sentences** describing:

1. whether the series has a visible trend;
2. whether it seems to have seasonality or repeated patterns;
3. whether any unusual values or abrupt changes are visible.

5.7. Play with a dataset (`timeSeriesDataSets`)

```r
if (requireNamespace("timeSeriesDataSets", quietly = TRUE)) {
  library(timeSeriesDataSets)

  # Replace gtemp_both_ts with another dataset from the package if you wish
  x <- gtemp_both_ts

  class(x)
  start(x)
  end(x)
  frequency(x)
  summary(x)
  plot(x, col = "firebrick", main = deparse(substitute(x)))
} else {
  message("Install 'timeSeriesDataSets' to complete this exercise.")
}
```

! Tasks

1. State what the dataset measures.
2. Determine whether it is univariate or multivariate.
3. Describe one visible feature of the plot.
4. Explain why this dataset qualifies as a time series.

5.8. Play with a dataset (`tsibbledata`)

```r
# Uncomment if needed:
# install.packages(c("tsibbledata", "ggplot2"))

if (requireNamespace("tsibbledata", quietly = TRUE)) {
  library(tsibbledata)
  library(ggplot2)

  # Replace gafa_stock with another dataset from tsibbledata if you wish
  x <- gafa_stock

  class(x)
  head(x)

  ggplot(x, aes(x = Date, y = Close, colour = Symbol)) +
    geom_line() +
    labs(
      title = "Example tsibbledata dataset",
      x = "Date",
      y = "Close"
    )
} else {
  message("Install 'tsibbledata' to complete this exercise.")
}
```

! Tasks

1. Identify the time index in the dataset.
2. Identify the measured variable(s).
3. Explain how this dataset differs from a basic `ts` object.
4. Write **2–4 sentences** describing what you observe.

5.9. Alternative representations, same same data

Create the same short series in two forms:

1. as a numeric vector;
2. as a `ts` object.

```r
values <- c(120, 135, 142, 160, 175)
values

values_ts <- ts(values, start = 2020, frequency = 1)
values_ts

class(values)
class(values_ts)

plot(values_ts,
     type = "o",
     pch = 16,
     col = "darkgreen",
     main = "Values as a Time Series",
     xlab = "Year",
     ylab = "Value")
```

! Tasks

1. Explain what information is present in the `ts` object that is not present in the plain vector.
2. State why the `ts` object is more useful for time-aware analysis.

5.10. Why time matters

Compare an ordered series with a shuffled version of itself.

```r
set.seed(123)

x <- AirPassengers
x_shuffled <- ts(
  sample(as.numeric(x)),
  start = start(x),
  frequency = frequency(x)
)

par(mfrow = c(1, 2))
plot(x, col = "steelblue", main = "Original series")
plot(x_shuffled, col = "tomato", main = "Shuffled series")
par(mfrow = c(1, 1))
```

```r
par(mfrow = c(1, 2))
acf(x, main = "ACF: original")
acf(x_shuffled, main = "ACF: shuffled")
par(mfrow = c(1, 1))
```

! Tasks

1. Do the two series contain the same values?
2. Why do they look different?
3. How does the autocorrelation structure change after shuffling?
4. What does this show about the importance of temporal order?

5.11. A simple moving average

Choose a monthly dataset such as `AirPassengers`, `nottem`, `co2`, or `ldeaths`.

```r
# Replace AirPassengers with your chosen series
x <- AirPassengers

x_ma <- stats::filter(x, rep(1/12, 12), sides = 2)

plot(x,
     main = "Series with 12-period moving average",
     xlab = "Time",
     ylab = "Value",
     col = "gray40")
lines(x_ma, col = "red", lwd = 2)
```

! Tasks

1. Explain what the moving average helps reveal.
2. State whether the moving average is smoother than the original series.
3. Describe what short-term variation becomes less visible.

5.12. A forecasting experiment

Use a simple forecasting function on a dataset of your choice.

```
# Uncomment if needed:
# install.packages("forecast")

library(forecast)

x <- AirPassengers

fc_naive <- naive(x, h = 12)
plot(fc_naive, main = "Naive forecast")
```

```
# Optional comparisons
fc_rw <- rwf(x, h = 12)
fc_snaive <- snaive(x, h = 12)

plot(fc_rw, main = "Random walk forecast")
plot(fc_snaive, main = "Seasonal naive forecast")
```

! Tasks

1. What does the naive forecast assume?
2. For your chosen dataset, does the forecast seem plausible?
3. Which of the three simple forecasting approaches seems most appropriate, and why?

5.13. Search and play with custom time series dataset

Use one of the CSV files from the public repository, via github, `jbrownlee/Datasets` (or other repositories)

Possible examples:

- `airline-passengers.csv`
- `daily-min-temperatures.csv`
- `daily-total-female-births.csv`

```
url <- "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
custom_df <- read.csv(url)
head(custom_df)
str(custom_df)
```

```
# A simple plot if the file has two columns: date/time and value
plot(custom_df[[2]],
     type = "l",
     col = "steelblue",
     main = "Custom dataset from jbrownlee/Datasets",
     xlab = "Observation index",
     ylab = names(custom_df)[2])
```

! Tasks

1. Identify the variable measured in the file.
2. Decide whether the data appear to be daily, monthly, yearly, or something else.
3. Convert the numeric column to a `ts` object if a regular frequency makes sense.
4. Write **2–4 sentences** describing the series.

5.14. Student choice mini-investigation (recommeded)

Choose **one dataset** from one of the following sources:

- base R datasets,
- `timeSeriesDataSets`,
- `tsibbledata`,
- a public CSV file such as one from `jbrownlee/Datasets`.

```
# Replace this with your chosen dataset or imported object
x <- JohnsonJohnson

class(x)
summary(x)
plot(x, col = "steelblue", main = deparse(substitute(x)))
```

! Tasks

Prepare a short investigation including:

1. the name of the dataset;
2. what it measures;
3. the class of the object;
4. its time frequency (if applicable);
5. one plot;
6. a short interpretation (**5–8 sentences**) mentioning, where relevant:
    - trend,
    - seasonality,
    - variability,
    - unusual observations,
    - suitability for forecasting.

5.15. Compare several time series from different sources

Choose **three** datasets from different sources (for example, one built-in dataset, one from `timeSeriesDataSets`, and one from `tsibbledata` or a web CSV).

```
par(mfrow = c(3, 1))
plot(JohnsonJohnson, col = "steelblue", main = "JohnsonJohnson")
plot(ldeaths, col = "firebrick", main = "ldeaths")
plot(nottem, col = "darkorange", main = "nottem")
par(mfrow = c(1, 1))
```

! Tasks

For each series:

1. state what it measures;
2. note whether it is yearly, quarterly, monthly, daily, or another frequency;
3. describe its most visible feature.

Then write a short comparison paragraph answering:

- Which one shows the clearest trend?
- Which one shows the clearest seasonality?
- Which one seems hardest to forecast visually?

Final questions

Answer the following in a few sentences each:

1. What distinguishes a time series from an ordinary unordered dataset?
2. Why does temporal ordering matter?
3. What is the difference between a numeric vector, a data frame with a time column, and a `ts` object?
4. What are some usual objectives of time series analysis?
5. Why can two datasets contain the same values but still convey very different information when ordered differently?